



ÉQUIPE DELTA / CADEV 2026

Cahier des charges

Un projet.

Un coup de main.

Une rencontre utile.

Proposez une aide concrète, engagez un échange consenti et partagez ce qui fait avancer les projets. L'atelier documente certaines améliorations.

COMMUNAUTÉ

Profils, projets
et connaissances

RÉALISATION

Laravel + React
Deux développeurs

LIVRAISON

Systalink + Vercel
Qodana + Actions

UNE SEULE RÉFÉRENCE ACTIVE

Sommaire

Le cadrage vient du mail fourni ; les modalités et exigences techniques sont les choix de conception HAAS.
Les identifiants de suivi sont conservés.

CADRAGE

01	La communauté HAAS	4
02	Parcours de lecture et traçabilité	5
03	Le concours comme contrainte	6
04	Droits, originalité et licences	7
05	Les usages réels à servir	8
06	Le périmètre contractuel du MVP	9

PRODUIT ET COLLABORATION

07	Rôles et autorisations	12
08	Les parcours de bout en bout	13
09	États et règles métier	14
10	Comptes, profils et notifications	16
11	Publier un blocage exploitable	17
12	Collaborer et résoudre	19
13	Capsules et versions	20
14	Recherche, réutilisation et reconnaissance	21
15	Le laboratoire contrôlé	22
16	Brique B1 · éviter les doublons	24
17	Brique B2 · reprendre après une coupure	25
18	Administration et protection des contenus	26

EXPÉRIENCE UTILISATEUR

19	Navigation et écrans	28
20	Direction artistique HAAS	29

ARCHITECTURE ET SÉCURITÉ

21	Architecture de référence	31
22	Dépôt et conventions de développement	31
23	Modèle de données · collaboration	33
24	Modèle de données · capsules et preuves	34
25	API · lecture et collaboration	36
26	API · preuves et erreurs	37
27	Sécurité intégrée au produit	39
28	Objectifs non fonctionnels	40

QUALITÉ ET LIVRAISON

29	Recette fonctionnelle · parcours central	42
30	Recette technique · preuves et exploitation	43
31	Qodana Ultimate et contrôles de qualité	45
32	GitHub Actions · de la PR à la release	45

33	Déploiement final · Systalink et Vercel	46
EXÉCUTION ET SOURCES		
34	Séquence de développement à deux	51
35	Backlog et réestimation.....	51
36	Budget simplifié et risques	52
37	Pilote et indicateurs honnêtes	54
38	Démonstration et présentation.....	55
39	Réception et traçabilité.....	56
40	Transmission et décisions à clôturer	57
41	Sources et portée du dossier	59
ATELIER DISTINCTIF		
42	Une communauté, un atelier distinctif.....	62
43	F13 · Proposer un cas concret.....	62
44	F14 · Comparer sans tricher	63
45	Données et API de l'atelier	64
46	F15 · Une preuve compréhensible	65
47	Architecture et écrans de l'atelier	66
48	Recette complémentaire AC33-AC52	68
49	Les 122 lots de réalisation	69
50	Utiliser le dossier et livrer	74
DÉPLOIEMENT		
51	Authentification · Vercel vers Systalink.....	76
52	Recette finale du déploiement.....	77
COMMUNAUTÉ ET CADRAGE		
53	Alignement sur le mail de l'organisateur	80
54	F16 · Présenter les projets	81
55	F17 · Découvrir les développeurs	83
56	Questions, données et contrats de communauté	83
57	Recette de l'alignement communautaire.....	85
LA RENCONTRE PAR LE COUP DE MAIN		
58	F18 · La rencontre par le coup de main	88
59	Contrats et architecture des coups de main.....	93
60	Une première collaboration facile à comprendre	97
61	Recette des coups de main AC69-AC90	98

01 / CADRAGE

La communauté HAAS

Identité et mission

HAAS — Help as a Service est le nom de concours retenu par l'équipe Delta. Une plateforme communautaire pour découvrir des développeurs et leurs projets, poser des questions, partager des connaissances et construire ensemble. Certains échanges aboutissent à des solutions documentées et à des vérifications que d'autres peuvent reprendre.

Présentez vos projets. Trouvez de l'aide. Construisez ensemble.

Le coup de main F18 facilite la première collaboration : projet ouvert volontairement, proposition limitée, accord explicite du propriétaire, puis échange dans le fil existant.

Le mail de l'organisateur met les usages communautaires au premier plan [M]. Notre atelier de cas, comparaison et fiche de vérification demeure la différence proposée, mais il ne conditionne ni une question, ni une discussion, ni la présentation d'un projet. L'utilisateur conserve ses outils et son assistant IA.

Décision	Application
Équipe	Deux développeurs Laravel/React ; revue croisée.
Communauté	F16 : projets et demandes liées ; F17 : découverte volontaire des développeurs.
Échanges	Questions de connaissance ou demandes techniques, commentaires et propositions.
Capitalisation	Capsules versionnées ; attribution du travail ; limites explicites.
Rencontres	F18 : première aide proposée, consentie, acceptée et liée à un échange utile.
Différence	Coup de main pour engager la rencontre ; F13-F15 pour documenter certaines améliorations.
Développement	Backend complet et testé → GO_FRONTEND humain → frontend → recette.
Hébergement	Un VPS Systalink/Datacloud pour Laravel/PostgreSQL et runner local restreint ; React sur Vercel ; sauvegarde distante.
Qualité	GitHub Actions, tests et Qodana Ultimate déclaré disponible ; accès projet à vérifier.

Statut et sources

Document de conception, pas attestation d'application développée, de tests produit réussis ou d'adoption. [M] : mail fourni ; [R] : règlement reçu ; [C] : capture du tableau de bord ; [U] : décisions utilisateur ; [H] : choix de conception HAAS. Les textes techniques antérieurs et références datées restent identifiés en section 41. Les nouvelles fonctionnalités ne sont pas présentées comme des obligations littérales du mail.

02 / CADRAGE

Parcours de lecture et traçabilité

Un dossier actif, des responsabilités explicites

Le mail précise la finalité ; le cahier décrit les choix retenus pour y répondre. Les compléments sont intégrés au produit, à la navigation, aux données, aux API, aux tests, aux instructions Codex et à la présentation. Les spécifications détaillées F16/F17 se trouvent aussi en sections 53-57.

Ensemble	Sections	Objet
Cadrage	01-06	Mission, sources, thème, droits, usages et périmètre.
Produit	07-18	Comptes, échanges, solutions, laboratoires et protection.
Expérience	19-20	Navigation, vingt-trois familles d'écrans et lisibilité.
Architecture	21-28	Laravel/React, données, API, sécurité et performance.
Qualité et livraison	29-33	Recette, Qodana, CI et cible Systalink/Vercel.
Exécution	34-41	Charge, pilote, présentation, réception et sources.
Atelier distinctif	42-50	Cas, comparaison, fiches de vérification et plan Codex.
Déploiement	51-52	Authentification inter-hôtes et contrôles d'ouverture.
Alignement communautaire	53-57	Mail, projets, découverte, contrats et nouveaux tests.
Coup de main	58-61	Ouverture, consentement, échanges liés, écrans et recette F18.

Repères stables

F01-F18 : exigences de conception. **AC01-AC90** : scénarios à exécuter ; AC47 s'applique uniquement si le P1 de copie de contexte est livré. **UX01-UX23** : familles d'écrans. **122 lots** : S2, B72, F40, R8 ; les 106 identifiants précédents sont conservés ; BH01-BH10/FH01-FH05/RH01 les complètent. Les douze sous-lots DEP et quatorze DEP-AC restent distincts de ce décompte.

Les identifiants historiques BV201/FV201 servent au suivi : ce ne sont pas des versions concurrentes du livrable. Les fichiers livrés gardent des noms sans suffixe de version. Ne pas écraser les statuts ou commits du dépôt réel par les TODO proposés dans ce pack.

Décisions applicables

ADR-001 fixe le backend-first ; ADR-004 fixe un VPS Systalink + Vercel ; ADR-005 place la communauté au premier plan. ADR-006 ajoute la rencontre par une aide bornée, sans modifier le mail source. ADR-002 conserve les règles de l'atelier sans imposer ce parcours à tout utilisateur. Toute divergence restante doit être arbitrée explicitement.

Priorité : recevabilité → usages communautaires complets → fiabilité → lisibilité → extensions. La différenciation ne remplace ni la pertinence au thème ni les tests de sécurité.

03 / CADRAGE

Le concours comme contrainte

Le règlement partagé est la base de travail. Les informations contradictoires restent visibles.

Thème et recevabilité

Le mail de cadrage précise le thème : « **Une plateforme d'échange pour les développeurs, par les développeurs et pour les développeurs.** » [M] Il cite se retrouver, échanger, poser des questions, partager ses connaissances, présenter ses projets et collaborer. La capture antérieure présente un ancrage africain [C] ; il ne devient pas un filtre technique d'exclusion dans HAAS. Le règlement impose une solution originale, essentiellement réalisée pendant le concours, accessible en ligne et mobilisant au moins un produit ou service Datacloud acquis selon ses conditions. Les équipes comptent deux à cinq membres ; chaque membre doit remplir les conditions d'âge, de pays, de compte et d'acceptation. [R, art. 4-6]

Deux calendriers à ne pas confondre

Échéance	Texte transmis	Décision de travail
Capture du tableau de bord	29 septembre-25 octobre 2026	Viser une version soumissible le 25 octobre.
Inscription et soumission	25 septembre-31 octobre 2026, 17 h Dakar	Demander confirmation écrite ; ne pas compter sur la marge.
Vote des pairs	1er-2 novembre 2026	Vérification individuelle du vote des deux membres.

Échéance	Texte transmis	Décision de travail
Public et jury	4-10 novembre 2026	Maintenir le service et les accès de démonstration.
Résultats	15 novembre 2026	Préparer le dossier de transmission, sans présumer du résultat.

Les dernières quatre lignes reprennent l'article 3 ; les heures sont celles de Dakar, GMT. **ARB01** : faire confirmer les dates, les modalités de dépôt et l'existence éventuelle d'exigences techniques complémentaires au mail. Aucune harmonisation silencieuse des deux sources. [R, art. 3 ; C]

La grille qui doit guider les efforts

Pertinence **25 %**, qualité technique **30 %**, ergonomie **20 %**, originalité **15 %**, vote public **10 % maximum**. Datacloud est une condition de recevabilité, non un critère de notation. [R, art. 6 et annexe 1]

La sélection par les pairs précède le jury ; le défaut de vote d'un membre exclut son équipe. Les trois projets les plus votés de chaque groupe national sont retenus, sous réserve du cas des groupes de trois projets recevables ou moins. [R, art. 7]

Conséquence pour HAAS : privilégier un échange réellement utile, une interface claire et une démonstration vérifiable plutôt qu'une accumulation de modules.

Ce que ce mail ne tranche pas

Le message laisse les équipes décider de la manière d'organiser les usages. Nos fiches projet, l'annuaire et le laboratoire sont des choix HAAS, non une liste de modules imposés. Le mail ne fournit pas de nouvelles dates, de budget ni de modification des articles du règlement. Les contradictions et autorisations restent ouvertes. Voir l'extrait exact et la correspondance en section 53.

04 / CADRAGE

Droits, originalité et licences

Le projet est construit pour le concours ; la possibilité d'une cession doit être anticipée dès le premier commit.

Ce que le règlement prévoit

Le prix de **5 000 000 FCFA brut** revient à l'équipe lauréate selon sa clé de répartition. Son versement est conditionné à une cession exclusive des droits patrimoniaux sur la solution gagnante. Le règlement précise que le lauréat ne peut ensuite exploiter ou commercialiser cette solution sans accord distinct. Ce n'est pas un financement ou une incubation. Les non-lauréats conservent leurs droits, selon les conditions de l'article 10. [R, art. 8-10]

ARB02 : solliciter le modèle de cession avant toute décision irréversible ; le règlement annonce sa communication lors des résultats. Ne pas présenter le présent cahier comme une analyse juridique de cet acte non communiqué.

Mesures de réalisation

Créer un dépôt neuf et conserver l'historique des contributions. Aucun code d'un client, d'un employeur ou d'une école n'est intégré sans autorisation adaptée. Les éléments personnels antérieurs sont déclarés ; les composants, leur provenance et leur licence sont inventoriés. [R, art. 6]

L'article 6 admet les composants existants sous licence permissive et exclut notamment GPL, AGPL et LGPL. Cette restriction est celle du concours : **ne pas la transformer en affirmation générale sur le droit des licences. ARB03** : demander son périmètre pour les dépendances de développement, services commerciaux et outils externes.

Séparer trois catégories

Catégorie	Politique proposée pour la version concours
Code et visuels créés par l'équipe	Inventaire nominatif ; dépôt privé ; aucun octroi public irréversible avant clarification.
Dépendances tierces	Licences et notices conservées ; aucune prétention à céder leurs droits de façon exclusive.
Contributions des testeurs	Retours de test et commentaires autorisés ; pas de code externe intégré au livrable sans autorisation compatible.

ARB05 : confirmer l'accès au dépôt, les modalités d'évaluation des kits téléchargeables et les droits de réutilisation. D'ici là, les sources des briques sont accessibles uniquement dans un cadre d'évaluation contrôlé ; la documentation publique ne promet pas une licence ouverte.

Les usages d'IA sont déclarés dans un registre : outil, date, fichiers concernés, vérification humaine et limites. Une sortie d'IA n'est jamais présumée libre de droits. [R, art. 6]

05 / CADRAGE

Les usages réels à servir

Trois situations, une même communauté

Présenter et trouver de l'aide. Un développeur décrit son projet et un besoin précis, ouvre une demande liée ou pose une question de connaissance sans code obligatoire.

Découvrir et contribuer. Un autre développeur trouve un projet, une personne ou un échange par technologie. Il commente, explique, relit, propose une correction ou documente un cas. Il n'est pas automatiquement recruté dans une équipe.

Apprendre et réutiliser. Un troisième membre consulte une discussion ou une capsule, comprend ce qui a été fait et en reprend la procédure. Toutes les solutions ne passent pas par un laboratoire.

Ces situations traduisent les usages du mail [M] en hypothèses de produit [H]. Aucune étude de marché ou métrique acquise n'est affirmée. Les profils publics se limitent aux informations volontairement partagées.

Objectifs proposés pour le pilote

Tâche	Cible qualitative à observer
Comprendre HAAS	4 personnes sur 5 identifient projets, personnes et échanges sans explication du facilitateur.
Présenter un projet	4 sur 5 remplissent une fiche et comprennent sa visibilité.
Trouver où contribuer	Un testeur extérieur retrouve une demande liée à un projet et y apporte un élément utile.
Poser une question	La tâche aboutit sans code, dépôt, résultat de test ni workflow de bug obligatoire.
Reprendre une solution	Au moins deux testeurs suivent une procédure et décrivent ce qu'ils ont obtenu.
Comprendre une preuve	4 sur 5 distinguent déclaration, acceptation et test exécuté.

Effectifs modestes et objectifs, pas résultats ou pourcentages représentatifs. Documenter aussi abandons, guidage et résultats négatifs. Tester d'abord les usages ordinaires, puis la branche du laboratoire.

Ancrage proposé

Démarrer avec des développeurs accessibles à l'équipe au Sénégal et, lorsque possible, dans un autre pays. Choisir les contenus à partir de leurs problèmes. Interface française P0 ; langue et pays facultatifs selon profils existants. La faible connexion est un cas technique à traiter, pas une caractéristique attribuée à tous les utilisateurs africains. Aucun classement de compétence par pays, âge ou identité.

06 / CADRAGE

Le périmètre contractuel du MVP

Deux développeurs doivent livrer une boucle complète, pas six produits partiellement construits.

P0 — indispensable à la réception

Code	Exigence
F01 / F11	Comptes, vérification du courriel, profils et attribution des contributions.
F02 / F03 / F04	Demande guidée, espace de discussion, proposition structurée et résolution par l'auteur.
F05 / F06	Capsule issue d'une résolution, publication relue, versions et limites explicites.
F07 / F12	Favoris, recherche simple et retour de réutilisation attaché à une version.

Code	Exigence
F08	Un laboratoire serveur à scénarios autorisés, résultat calculé et traçabilité.
F09 / F10	Signalement, modération minimale et notifications internes.

Deux briques éditorialisées sont prévues : **B1, traitement idempotent d'événements fictifs**, avec laboratoire serveur ; **B2, formulaire à reprise après coupure**, avec kit et protocole reproductible. Le kit B2 fonctionne réellement, mais n'impose pas un second moteur de laboratoire.

P1 — uniquement après validation des P0

Traduction anglaise relue, suggestion simple de contributeurs, abonnement à une discussion, courriel de notification, consultation hors connexion de capsules publiques choisies, assistance IA à la rédaction et second laboratoire serveur.

Hors concours

Exécution de code arbitraire, terminal en ligne, visioconférence, messagerie privée, paiements, place de marché, cours complets, application native, moteur de réputation complexe, génération de code autonome, import automatique de dépôts externes et reconnaissance de compétences par IA.

Règle de réduction du périmètre

En cas de retard, supprimer les P1 puis les raffinements du profil et les filtres secondaires. **Ne jamais supprimer les autorisations, la sincérité des résultats de laboratoire, la conformité des licences ou la recette du parcours principal.**

Si B2 menace la livraison du socle, documenter son retrait comme une modification de version, et réviser le pitch. Un bouton ou une carte non fonctionnelle ne peut pas être compté comme une fonctionnalité livrée.

Capacité historique, à réestimer : 216 heures-personnes proposées sur 18 jours de travail à 6 heures utiles par personne ; engagement initial de 180 heures et réserve de 36 heures. Disponibilité à confirmer par les deux membres [U ; hypothèse de planification].

P0 ajouté — atelier

Code	Livrable
F13	Cas documentaires proposés, révisés, examinés et liés à un scénario approuvé lors d'une release.
F14	Comparaison B1 des mêmes entrées sur deux implémentations approuvées ; deux observations réelles.
F15	Fiche versionnée et contributions reliées ; preuve humaine distincte du laboratoire.

La comparaison pédagogique auparavant facultative devient le parcours distinctif obligatoire de B1. Aucune généralisation à tous les langages. La copie de contexte pour un assistant reste P1, sans API IA. La capacité historique de 216 heures n'est pas une estimation de cette extension ; voir section 35.

Communauté — compléments indispensables retenus

F16 : présenter et découvrir un projet, publier une fiche, formuler un besoin et rattacher ses demandes. **F17** : trouver des développeurs par technologie dans un annuaire volontaire. **F02 précisé** : poser une question sans obliger à fournir du code ou un résultat de bug.

Ces choix donnent une réponse visible aux usages [M] ; leurs détails sont des prescriptions HAAS [H]. Les liens projet/demande réutilisent les fils existants. Pas de messagerie privée, d'invitations d'équipe, de gestion de tâches, de classement ni de galerie d'images à téléverser. Le laboratoire reste facultatif pour le parcours de chaque utilisateur, mais le démonstrateur B1 reste un livrable P0 du projet.

Le planning conserve BC01–BC08/FC01–FC04 et intègre BH01–BH10/FH01–FH05/RH01. Aucune conservation mécanique de l'estimation historique ne couvre ces ajouts.

Première rencontre — F18 indispensable

Un projet peut s'ouvrir volontairement aux coups de main. Un membre propose un apport limité ; le propriétaire accepte et autorise un échange public après les deux consentements, ou décline. F18 comprend les occasions de contribuer, l'offre, son cycle et la projection de progrès réellement documentés. Les règles détaillées sont en sections 58–61. Pas de messagerie générale, équipe, paiement ou recommandation IA. Une acceptation n'est pas un travail achevé.

07 / PRODUIT ET COLLABORATION

Rôles et autorisations

Le rôle ne remplace pas la propriété de la ressource : les deux sont vérifiés côté serveur.

Matrice d'accès

Action	Visiteur	Membre vérifié	Modérateur	Administrateur
Lire les contenus publics publiés	Oui	Oui	Oui	Oui
Créer une demande / contribuer	Non	Oui	Oui	Oui
Modifier sa demande ouverte	Non	Propriétaire	Propriétaire	Propriétaire
Accepter une solution	Non	Auteur de la demande	Non, sauf auteur	Non, sauf auteur
Publier une capsule relue	Non	Proposition seulement	Oui, après revue	Oui, après revue
Lancer un laboratoire	Non	Oui, sous quota	Oui, sous quota	Oui, sous quota
Masquer un contenu / traiter un signalement	Non	Non	Oui	Oui
Suspendre un compte / gérer les rôles	Non	Non	Non	Oui

Les modérateurs peuvent masquer ou archiver avec motif, mais ne peuvent pas fabriquer une validation au nom de l'auteur. Une intervention administrative laisse une trace d'audit.

Contraintes de propriété

Un membre n'édite pas les messages ni les propositions d'autrui. La propriété n'est jamais acceptée depuis le corps JSON : `author_id`, rôles, état de revue et résultats de tests sont fixés par le serveur.

Un membre suspendu conserve l'accès à son écran d'information et à la procédure de contact, mais ne publie plus et ne lance plus de laboratoire. Les sessions sont invalidées lors d'une suspension ou d'un changement sensible de rôle.

Mode démonstration

Trois personnages peuvent illustrer la collaboration, même si l'équipe de concours compte deux personnes. Leurs comptes portent explicitement « **Données de démonstration** ». Ils ne sont ni présentés comme des utilisateurs acquis, ni utilisés pour gonfler les indicateurs du pilote.

Le jury utilise un compte de démonstration limité et renouvelable ; aucun accès administrateur partagé publiquement. Les vrais testeurs ne se connectent pas avec les comptes de personnages.

Recette associée : AC03, AC04, AC25 et AC26. Chaque ressource privée ou retirée est aussi testée par accès direct à son URL et à son API.

Droits supplémentaires [A]

Le membre actif/vérifié peut proposer un cas sur un parent accessible, puis réviser ses brouillons. Le réviseur habilité est distinct de l'auteur. Un statut « examiné » ne donne aucun droit d'exécution. Le comparateur exige profil approuvé, version visible et quota commun ; les conclusions machine ne sont jamais modifiables par une route publique. Les restrictions de retrait s'étendent aux cas, comparaisons et fiches liés.

Projets et annuaire

Seul un propriétaire actif vérifié édite/publie/archive son projet et lui rattache sa propre demande ; un autre membre contribue dans les échanges publics autorisés. La modération peut masquer avec motif, pas agir comme propriétaire. L'annuaire exige compte actif vérifié et consentement `directory_visible` ; disponibilité auto-déclarée, courriel jamais public. Les précisions de visibilité liée sont en section 54.

08 / PRODUIT ET COLLABORATION

Les parcours de bout en bout

A — Présenter un projet et recevoir une aide

A crée une fiche en brouillon, la relit puis la publie. Il peut ouvrir une demande liée ou activer les coups de main. B découvre le besoin et, sans fil ouvert, propose une offre F18 ; A accepte après aperçu/consentement et crée ou choisit le fil. Si une demande publique existe déjà, B peut la rejoindre directement. B commente, propose une réponse ou apporte un cas. A accepte une proposition si elle a réellement aidé ; sinon la discussion continue.

Échecs traités : droits insuffisants, fiche incomplète, brouillon privé, projet archivé/masqué, réseau interrompu et édition concurrente. Un lien de démonstration externe n'est pas exécuté par HAAS.

B — Rencontrer des développeurs et apprendre

Un membre choisit d'être visible dans l'annuaire, indique ses technologies et éventuellement son ouverture à une contribution. Un visiteur le découvre, consulte ses projets ou contributions publiques et rejoint un échange existant. Pas de demande de contact privé ou de faux statut en ligne.

Un membre peut aussi poser une question de connaissance : titre, contexte, question et technologies. Les réponses sont des commentaires du fil existant ; aucun code ni laboratoire n'est obligatoire pour discuter ou apprendre.

C — Transformer un échange en ressource

L'auteur d'une résolution ou un contributeur autorisé propose une capsule. Une autre personne habilitée la relit. La version publiée conserve la procédure, les auteurs et les limites. C peut ensuite consulter, enregistrer et déclarer son retour de réutilisation. L'absence de test machine est indiquée sans bloquer la lecture.

D — Mettre certaines solutions à l'épreuve

Un cas documentaire peut être proposé, relu puis intégré à un scénario uniquement par la procédure de release approuvée. Pour B1, le jury choisit un scénario existant et compare deux implémentations contrôlées. Les observations sont recalculées ; une panne ou une régression est visible. Le démonstrateur B2 reste distinct.

E — Protéger la communauté

Un membre signale un contenu ou projet. La modération examine son contexte, masque avec motif si nécessaire et invalide les accès/caches liés. La suspension est une action administrative. Aucun nombre de signalements ne fabrique une sanction irréversible.

Recette centrale : projet de A → découverte par B → échange utile → contribution attribuée → solution lisible par C. L'atelier prolonge ce parcours lorsqu'il est applicable ; il ne remplace pas le thème communautaire.

A, B et C sont des personnages de recette, pas trois membres déclarés dans l'équipe de deux.

E — Première collaboration sans demande préalable

Projet volontairement ouvert → offre pending privée → acceptation et aperçu public → fil créé/choisi → apport utile. Les deux acteurs confirment la projection du résumé. Une demande ouverte reste directement accessible aux contributions ; aucune offre n'est exigée pour toute discussion. Voir F18 et AC69-90.

09 / PRODUIT ET COLLABORATION

États et règles métier

Les transitions autorisées sont explicites ; un simple changement de statut envoyé par le navigateur ne suffit pas.

Machines d'états

Objet	Transitions principales
Demande	Brouillon → Ouverte → En cours → Résolue ; Résolue → Ouverte par réouverture motivée ; vers Archivée avec motif.
Proposition	Proposée → Acceptée ou Non retenue ; une résolution référence une seule proposition acceptée active.

Objet	Transitions principales
Version de capsule	Brouillon → En revue → Publiée ; En revue → À corriger → En revue ; Publiée → Retirée.
Exécution de laboratoire	En file → En cours → Réussie / Échouée / Erreur / Délai dépassé.
Signalement	Nouveau → En examen → Traité / Sans suite, toujours avec motif.

Une première proposition structurée fait passer une demande ouverte à « En cours ». Un commentaire seul ne change pas l'état. Une réouverture annule la résolution active, conserve l'historique et marque les capsules liées « Source rouverte — revue nécessaire ».

Règles transversales

RM01 — Une acceptation autorisée. Seul l'auteur de la demande accepte une proposition lui appartenant indirectement par cette demande. Une résolution de sa propre demande est admise mais étiquetée « auto-résolution », sans validation indépendante.

RM02 — Concurrence. Accepter ou rouvrir passe par une transaction avec verrou sur la demande. Deux acceptations concurrentes ne créent jamais deux résolutions actives.

RM03 — Version figée. Une version publiée ne s'écrase pas. Une correction produit une nouvelle version ; ses preuves ne sont pas héritées automatiquement de l'ancienne.

RM04 — Trois signaux distincts. « Accepté par l'auteur », « Retour de réutilisation déclaré » et « Tests de laboratoire réussis » restent trois informations différentes. Aucun badge universel de sécurité.

RM05 — Traçabilité. Une modification significative, un retrait, une acceptation ou une action de modération crée un événement horodaté avec acteur et motif éventuel.

RM06 — Visibilité. Brouillons, contenus retirés et informations de sécurité ne sont pas exposés par les listes, recherches, caches ni liens directs. Les identifiants publics ne donnent aucun droit d'accès.

États complémentaires [A]

Objet	États
Cas de vérification	draft, submitted, reviewed, declined, integrated, withdrawn ; révisions conservées.
Comparaison	queued, running, completed, error, timed_out.
Conclusion	improved, unchanged, regressed, mixed, inconclusive.

Une comparaison terminée peut conclure à une régression. « Examiné » ne signifie pas « testé ». Le profil, les cas, les entrées et les digests sont figés au lancement. Les règles détaillées figurent en sections 43–46.

États complémentaires de la communauté

ProjectStage : idea/prototype/in_development/launched ; déclaration d'avancement.

ProjectPublicationState : draft/published/archived ; contrôle de publication séparé de hidden_at. Une restauration de modération ne publie pas un brouillon. HelpAvailability : not_specified/available/unavailable ; aucune connexion temps réel impliquée. HelpIntent ajoute ask_question ; validation conditionnelle précisée en section 56.

États du coup de main

pending → accepted / declined / withdrawn / expired. Délai serveur 7 jours ; accepted garde l'accord historique, pas un état « terminé ». L'expiration ne donne pas un refus attribué à l'auteur. Consentements, courses et retrait de visibilité : sections 58–59.

10 / PRODUIT ET COLLABORATION

Comptes, profils et notifications

F01, F10 et F11 — une identité minimale, sans collecte inutile ni complexité de réseau social.

Compte et authentification

Inscription par pseudonyme, courriel et mot de passe. Pseudonyme de 3 à 30 caractères ; courriel unique normalisé ; mot de passe de 12 à 128 caractères, confirmation requise, collage autorisé. Courriel non exposé publiquement. Acceptation des conditions d'usage versionnée.

Le compte non vérifié peut lire mais pas publier, télécharger un kit contrôlé ou lancer un laboratoire. Vérification par lien signé expirant après 60 minutes ; nouvelle demande limitée. Réinitialisation par lien à usage unique, expirant après 60 minutes ; réponse identique que le courriel existe ou non.

Choix technique : authentification SPA par session Laravel Sanctum et protection CSRF, plutôt qu'un jeton stocké dans localStorage. Sanctum prévoit ce mode par cookies pour une SPA de confiance. [S02]

Profil public

Pseudonyme, biographie de 500 caractères, jusqu'à 8 technologies, langue principale, pays facultatif, lien GitHub HTTPS facultatif. Avatar généré à partir des initiales en P0 : pas d'envoi de photo à sécuriser.

Les contributions proviennent des événements métier, pas d'un nombre envoyé par le client. Afficher les liens et la nature du travail ; ne pas calculer un classement de compétence. Les contributions sur jeux de démonstration sont séparées des contributions réelles.

Notifications internes

Événement	Destinataire	Contenu minimal
Proposition publiée	Auteur de la demande	Auteur, titre et lien vers la proposition.

Événement	Destinataire	Contenu minimal
Proposition acceptée / demande rouverte	Contributeur concerné	Nouvel état et lien vers l'historique.
Revue de capsule terminée	Auteur du brouillon	Publication ou corrections demandées.
Rapport de laboratoire disponible	Demandeur du test	État du rapport, version et lien.

Marquage lu/non lu, liste paginée, aucune copie de secret ou de code dans une notification. L'émission suit la transaction validée ; un incident de notification n'annule pas une résolution enregistrée.

Cibles d'abus proposées : 5 essais de connexion/minute par combinaison IP-compte ; 3 demandes de lien/heure par compte. Ajustement documenté après test pour ne pas bloquer arbitrairement un réseau partagé.

Notifications et attributions atelier [A]

Un cas soumis, une revue et une comparaison terminée génèrent uniquement les notifications utiles et autorisées. Ne pas copier du code sensible dans le message. Le profil relie le rôle de chaque contributeur (cas, diagnostic, correctif, documentation), sans convertir ces actions en score d'expertise.

Découverte volontaire

F17 ajoute `directory_visible=false` par défaut, une disponibilité auto-déclarée, son horodatage de mise à jour et un texte de contribution facultatif. L'annuaire ne remplace pas le profil courant et n'expose ni courriel ni dernière connexion. Un retrait de l'annuaire invalide la découverte, pas automatiquement l'attribution d'un travail public. Les projets publics du membre sont visibles sans ses brouillons. Voir section 55.

11 / PRODUIT ET COLLABORATION

Publier un blocage exploitable

F02 — le formulaire améliore la qualité du contexte sans demander à l'utilisateur d'écrire un rapport technique.

Contrat du formulaire pour une demande technique

Champ	Règle de validation proposée
Titre	15-140 caractères, formulé comme un problème concret.
Objectif / résultat attendu	30-2 000 caractères chacun.
Comportement observé	30-4 000 caractères ; message d'erreur distinct si utile.

Champ	Règle de validation proposée
Tentatives déjà réalisées	20-3 000 caractères ; « aucune » explicite autorisé.
Technologies	1-5 étiquettes du référentiel ; version texte de 40 caractères maximum par technologie.
Extrait de code	Facultatif, 12 000 caractères maximum, texte inerte ; langage choisi.
Environnement / langue	Environnement court ; français par défaut ; langue visible.

Un lien de reproduction HTTPS peut être ajouté sans récupération automatique de son contenu. Pas de téléversement de ZIP, de PDF ni de dépôt complet en P0. Aucun `iframe` externe.

Parcours d'édition

Trois étapes : **Contexte** → **Problème** → **Vérification avant publication**. Résumé relisible avant validation ; erreurs sous les champs, focus sur la première erreur. Un bandeau explique que le contenu publié est visible par d'autres personnes.

Une détection indicative repère des motifs tels que `PRIVATE KEY`, variables de secret et jetons courants. Le membre doit retirer les valeurs signalées. Cette détection ne garantit pas l'absence de secret et ne remplace pas la modération.

Les brouillons enregistrés restent réservés à leur auteur. Après une contribution, l'auteur peut clarifier le texte mais ne change pas silencieusement le problème : une note de modification et un horodatage sont conservés. Les suppressions destructrices de discussions sont réservées au traitement de modération.

Actions et états d'interface

« Enregistrer le brouillon », « Publier », « Modifier », « Archiver avec motif ». Pendant l'envoi, le bouton est désactivé ; une clé de requête prévient la création accidentelle en double. Sur erreur, la saisie est conservée et un bouton de reprise apparaît.

Recette : AC05-AC07. Le formulaire mobile doit permettre de relire et corriger les champs sans défilement horizontal de la page.

Accueillir le travail déjà réalisé [A]

Champ `help_intent` : résoudre un blocage / faire relire une solution / faire reproduire un comportement. Même formulaire et même cycle de demande. L'aide sous « Tentatives » invite à expliquer une piste déjà obtenue, avec ou sans IA. Ne pas demander une conversation complète ni un dépôt privé. Un cas à vérifier peut ensuite être proposé dans le parcours F13.

Variante « Poser une question »

F02 est précisé : `help_intent=ask_question` n'exige ni code, ni comportement logiciel attendu, ni tentatives préalables. Réutiliser `goal` pour le contexte et `observed` pour la question ; `expected/attempts` facultatifs et nullables pour ce mode. Les autres intentions conservent leurs règles. Une demande peut rester autonome ou être liée au projet publié de son propriétaire. Les champs `project_id` et `help_intent` sont validés dans le même contrat, sans second fil de discussion. Voir AC64 et AC68.

12 / PRODUIT ET COLLABORATION

Collaborer et résoudre

F03 et F04 — une discussion reste lisible et débouche sur une décision attribuée.

Espace de résolution

L'en-tête rassemble titre, auteur, technologies et état. Une colonne principale affiche contexte, discussion et propositions. Le panneau de synthèse indique les tentatives, la résolution éventuelle et la capsule liée. Sur mobile, ce panneau suit le contexte.

Commentaire : 1–4 000 caractères, Markdown restreint, historique d'édition. **Proposition** : diagnostic, correctif, mode de vérification et limites ; chacun de 20–4 000 caractères, code facultatif limité à 12 000 caractères. Une proposition ne consiste pas seulement en un lien extérieur.

Règles de contribution

Un membre peut commenter ou proposer plusieurs approches, mais il ne peut accepter sa réponse à la place du demandeur. Les mentions et pièces jointes sont exclues du P0. Une requête répétée avec la même clé ne crée pas deux messages.

Le rafraîchissement est explicite ou par interrogation légère toutes les 15 secondes tant que l'écran est actif. Suspendre les requêtes en arrière-plan et en cas de coupure. Aucun WebSocket n'est nécessaire au MVP.

Clôture guidée

L'auteur sélectionne une proposition, précise son environnement et explique ce qui a fonctionné. Le serveur crée la résolution, l'événement de contribution et la notification après contrôle d'autorisation et transaction.

Si la proposition ne fonctionne pas, la demande reste ouverte ou en cours. Une action « Non retenue » demande un court motif utile, sans sanction automatique du contributeur.

Réouverture et archivage

Une résolution peut être rouverte par l'auteur avec motif. Les anciennes preuves restent dans l'historique ; la capsule n'est pas automatiquement effacée, mais son lien de contexte signale la réouverture. Un modérateur peut demander une revue ou retirer une version trompeuse.

L'archivage n'est pas une résolution. Il porte un motif tel que doublon, hors périmètre, auteur sans besoin actif ou modération. L'auteur est informé.

Accepté signifie « cela a aidé le demandeur dans son contexte », pas « cette solution est correcte et sûre dans tous les projets ».

Recette : AC08–AC11 ; tests de double clic, accès interdit et acceptations concurrentes obligatoires.

Une contribution ne se limite pas au correctif [A]

Proposer un cas oublié, reproduire des étapes ou expliquer une limite constitue une contribution identifiable. Le bouton « Proposer un cas à vérifier » ouvre F13, distinct d’un commentaire. Une intégration au scénario ne s’effectue qu’après revue technique et release, jamais à partir du texte soumis exécuté directement.

Échanges liés à un projet ou une question

Le contexte peut indiquer le projet publié de l’auteur. Ses contrôles de visibilité s’appliquent aussi aux lectures du fil. Pour une question de connaissance, les commentaires suffisent à échanger : aucune proposition technique ni passage au laboratoire n’est imposé. Si une résolution est acceptée, les règles d’attribution restent les mêmes. « Proposer une aide » depuis un projet renvoie vers une demande ouverte existante, pas vers un formulaire d’invitation fictive.

13 / PRODUIT ET COLLABORATION

Capsules et versions

F05 et F06 — le résultat de la collaboration devient un objet documenté, versionné et retrouvable.

Structure d’une capsule

Une capsule contient titre, résumé, problème, cause, correction, procédure de vérification, limites, technologies, versions compatibles déclarées, contributeurs et provenance. Elle est reliée à une demande résolue ou à une origine éditoriale clairement signalée pour les briques de démonstration.

La capsule est le contenant documentaire ; la **brique** est son module de code facultatif ; le **laboratoire** est une vérification contrôlée facultative. Ces trois objets ne sont pas assimilés.

Revue avant publication

Le membre propose un brouillon. Un autre membre habilité vérifie la cohérence et les droits. Pour les contenus créés par A, B relit ; pour ceux de B, A relit. L’auteur ne valide pas seul sa propre revue éditoriale. Un administrateur conserve cette séparation même s’il détient tous les droits techniques.

Contrôle	Condition de publication
Compréhension	Objectif, procédure et limites renseignés.
Attribution	Auteurs et origine identifiés ; pièces d’autorisation au registre si nécessaire.
Code	Aucun secret, aucun exécutable téléversé ; licence et périmètre de diffusion validés.
Vérification	Nature de la preuve indiquée ; aucun succès fictif.
Version	Identifiant unique, journal de changement et empreinte de l’artefact lorsqu’il existe.

Version publiée immuable

Numérotation lisible 1.0.0, 1.0.1, etc., sans promesse automatique de compatibilité. Une nouvelle version crée un enregistrement distinct. Les anciens rapports restent attachés à leur version, au digest du code et à la suite de tests correspondante.

Le retrait d'une version demande un motif visible et désactive son téléchargement et le lancement de tests nouveaux. Les anciennes preuves peuvent rester consultables si cela ne révèle pas de contenu interdit ; sinon seul l'historique de retrait est affiché.

Distribution contrôlée

Le kit peut contenir code source de la brique, README, cas de test, dépendances verrouillées et notices. Le téléchargement nécessite des conditions d'évaluation approuvées et une autorisation serveur ; ses droits ne sont jamais déduits du simple fait qu'un fichier est disponible. Voir ARB05, section 04.

La version comme point d'ancrage [A]

Chaque cas et comparaison cible une version explicite. La fiche de vérification distingue observations et exécutions. Le manifeste de la référence pédagogique incorrecte est séparé du kit recommandé et n'est pas distribuable. Une correction du candidat crée une nouvelle version et de nouveaux tests ; aucun héritage implicite de résultat.

14 / PRODUIT ET COLLABORATION

Recherche, réutilisation et reconnaissance

F07, F11 et F12 — rendre les connaissances accessibles sans fabriquer une réputation artificielle.

Catalogue et recherche

Recherche dans les titres, résumés et étiquettes ; filtres P0 : technologie, type « demande/capsule », état de résolution et présence de laboratoire. Tri explicite par pertinence textuelle ou date. Pagination de 20 éléments, maximum 50 par requête.

Le catalogue n'expose que les capsules publiées et les demandes publiques autorisées. Une recherche vide propose les derniers contenus éditoriaux, sans fausse personnalisation. Si rien ne correspond, proposer de reformuler ou de créer une demande ; ne pas générer une réponse fictive.

Les cartes affichent titre, version, technologies, statut de preuve et dernière mise à jour. Un badge de laboratoire ne s'applique pas à toutes les versions d'une capsule.

Retour de réutilisation

Un membre choisit la version utilisée, renseigne environnement et date, indique « Reproduit », « Partiellement reproduit » ou « Non reproduit », puis décrit son observation. Un retour reste une déclaration humaine, pas une certification indépendante.

Un retour actif maximum par utilisateur et version ; modification possible avec historique. Un auteur peut documenter son propre test mais son retour est marqué comme interne à la contribution. Aucun point n'est accordé pour une auto-validation.

Favoris et profils

Un favori est privé et peut être supprimé. Les compteurs de profil s'appuient sur les contributions effectivement liées à une résolution ou à une publication. Un contenu retiré ne conserve pas un badge de validation actif.

Pas de classement global, de score de compétence, de monnaie virtuelle ni de récompense financière en P0. La reconnaissance vient des liens vers les travaux et du rôle : diagnostic, correction, documentation, test.

Garde-fous de mesure

Les téléchargements répétés et favoris ne sont pas présentés comme des projets effectivement aidés. Les métriques de démonstration sont exclues des statistiques d'adoption. Le tableau d'administration distingue **événement technique**, **déclaration de résultat** et **observation utilisateur**.

Recette : AC14, AC15 et AC27. Tester explicitement la recherche d'une version retirée et l'envoi de plusieurs retours identiques.

Chercher une preuve sans inventer une réputation [A]

Une carte peut signaler « comparaison disponible » uniquement pour une version et un profil réellement autorisés. Les contributions de cas sont reliées à leur révision. Ne pas trier les personnes par un score calculé à partir de résultats de démonstration. La navigation privilégie Explorer, Demander de l'aide et Mes contributions.

Découverte complémentaire

Le catalogue de capsules reste inchangé ; les projets disposent de leur catalogue et les développeurs de leur annuaire volontaire. Réutiliser recherche/pagination/visibilité, sans unifier artificiellement des objets aux règles différentes. Les contributions affichées sur un projet ou profil proviennent d'actions réelles sur des ressources accessibles, jamais d'une liste de coéquipiers déduite d'un commentaire.

15 / PRODUIT ET COLLABORATION

Le laboratoire contrôlé

F08 — démontrer un comportement réel sans ouvrir une plateforme d'exécution de code arbitraire.

Contrat de sécurité et de sincérité

Le laboratoire n'accepte ni source exécutable, ni commande shell, ni URL à charger, ni dépendance à installer depuis une requête utilisateur. Les seules entrées sont un identifiant de version approuvée et un scénario appartenant à une liste autorisée.

Le code testé est écrit ou intégré par l'équipe, relu et déployé via la CI. Le registre associe `runner_key`, digest du module, digest de la suite et versions d'exécution. Un rapport ne peut pas être chargé ou marqué réussi par le navigateur.

Exécution et isolation

L'API crée une exécution et la met en file. Le job Laravel orchestre ; un service local standalone sous une identité restreinte appelle un adaptateur approuvé sans charger le `.env` Laravel principal. Les données de cas sont dans une base de laboratoire distincte, réinitialisées par exécution. Son rôle SQL n'a pas accès aux comptes et contenus privés ; seul l'orchestrateur applicatif contrôle la queue et enregistre les rapports. Le service de test ne lit pas la base métier ni sa queue.

Aucune clé de paiement, aucun accès cloud administrateur et aucun secret de production n'entre dans le service exécuter. Les sorties sont limitées à un schéma JSON, sans stack trace publique.

Cette architecture n'est pas présentée comme une sandbox de code tiers.

Limites proposées

Limite	Valeur P0
Exécutions actives	Une opération par membre ; un seul run actif globalement.
Quota	Cinq lancements par heure et par membre ; seuils IP à ajuster pour réseaux partagés.
Temps de traitement	15 secondes par exécution ; abandon technique au-delà de 20 secondes.
Rapport	64 Ko maximum ; pas de fichier généré par du code utilisateur.
Données de travail	Purge après le test ou par nettoyage de secours sous 24 heures.

Rapport minimal

Identifiant, version, digests, scénario, date UTC, durée, liste des assertions, résultat attendu/observé et état terminal. **Échouée** signifie assertion non satisfaite ; **Erreur** signifie incident technique ; **Délai dépassé** signifie absence de conclusion. Un ancien rapport reste daté et n'est jamais affiché comme un test qui vient d'être lancé.

Quotas communs au comparateur [A]

La limite de cinq lancements est exprimée en cinq unités d'exécution par heure : un run simple consomme une unité ; une comparaison en réserve deux. Les enfants sont exécutés séquentiellement, une opération par membre, au maximum un run actif globalement. Les deux voies partagent les mêmes réservations atomiques. Les 20 secondes maximum par enfant restent valables ; budget total proposé 50 secondes après démarrage de la comparaison. Voir section 44.

Compromis Déploiement retenu : le service et les deux bases partagent un VPS. Comptes système, fichiers et rôles SQL sont séparés et testés ; cela ne garantit pas une isolation de noyau ni de ressources PostgreSQL. Une seule exécution globale, charge mesurée et arrêt d'urgence sont requis.

16 / PRODUIT ET COLLABORATION

Brique B1 · éviter les doublons

Le démonstrateur principal : un même événement fictif ne doit pas créer deux commandes.

Ce que la brique fait réellement

Un petit module Laravel reçoit un événement de démonstration composé de `event_id`, `order_ref`, `amount_minor` et `currency`. Aucun paiement réel ni service bancaire n'est intégré. La monnaie et le montant sont des données fictives, pas un calcul financier.

Le traitement validé repose sur une transaction et une contrainte unique (`run_id`, `event_id`). La clé n'est pas seulement contrôlée par un `SELECT` applicatif : l'unicité de base protège aussi les accès concurrents. La création de la commande et l'enregistrement de l'événement sont atomiques.

Scénarios fournis

Cas	Entrée	Résultat attendu
B1-01 · nominal	Un événement valide	Une commande et un événement traité.
B1-02 · doublon	Deux occurrences identiques	Une seule commande ; seconde occurrence reconnue comme doublon.
B1-03 · distincts	Deux identifiants différents	Deux commandes distinctes.
B1-04 · invalide	Champ obligatoire absent	Rejet de validation ; aucune écriture partielle.
B1-05 · concurrence	Deux traitements réellement simultanés	Une seule commande ; pas d'exception non gérée.

B1-01 à B1-04 sont accessibles dans le laboratoire web. B1-05 est un test d'intégration concurrente dans la CI ; son rapport est étiqueté « exécuté en CI », pas « rejoué dans le navigateur ».

Comparaison pédagogique

Le démonstrateur inclut un module pédagogique distinct reproduisant volontairement un traitement naïf qui crée deux commandes. Il porte la mention « Exemple incorrect ». Il n'est jamais utilisé pour l'authentification, les contenus ou le traitement normal de HAAS.

Le jury observe le défaut, la contribution qui l'explique, la version corrigée puis les assertions recalculées. La CI vérifie aussi que l'exemple incorrect reproduit exactement le défaut attendu ; elle ne masque pas une régression du produit.

Limites à afficher

Le module n'est ni un connecteur fournisseur, ni une intégration de paiement certifiée. Il ne couvre pas les signatures réelles, les remboursements, tous les ordres de livraison ni tous les incidents distribués. Un adaptateur de production nécessiterait une analyse supplémentaire.

Comparateur central [A]

L'exemple incorrect et le candidat sont enregistrés sous un même profil de comparaison, avec suite commune et données identiques. Le résultat attendu du cas doublon est une commande ; les deux observations doivent être recalculées. Le candidat n'est pas déclaré conforme par anticipation. Les autres scénarios et les échecs restent visibles ; l'illustration n'est pas un benchmark de vitesse ni une intégration fournisseur.

17 / PRODUIT ET COLLABORATION

Brique B2 · reprendre après une coupure

Une seconde brique utile, sans imposer un deuxième moteur de laboratoire à une équipe de deux personnes.

Fonctionnement livré

Un mini-formulaire React de démonstration prépare une commande fictive. Chaque demande reçoit une clé stable avant le premier envoi et est conservée dans IndexedDB. L'écran distingue « **Conservée sur cet appareil** », « **Envoi en cours** », « **Confirmée par le serveur** » et « **Échec à reprendre** ».

Au retour du réseau, l'utilisateur peut cliquer sur « Réessayer ». Le serveur de démonstration gère la répétition de la même clé. Une action automatique à l'ouverture de la page peut compléter ce parcours, mais **Background Sync n'est pas une dépendance obligatoire**. Les mécanismes de service worker permettent des usages hors connexion, avec des capacités dépendantes du navigateur. [S08]

Protocole de vérification

Étape	Preuve attendue
Ouvrir la démo en ligne	Version visible ; fichiers nécessaires déjà disponibles.
Couper réellement le réseau	Une saisie reste enregistrable localement et n'est pas annoncée « envoyée ».
Fermer puis rouvrir la démo	Brouillon retrouvé lorsque le stockage local n'a pas été effacé.
Rétablir le réseau et reprendre	Réponse serveur visible ; passage à « Confirmée ».
Répéter l'envoi	Pas de seconde commande pour la même clé.
Effacer les données locales	Avertissement préalable ; aucune promesse de récupération ensuite.

Conditions de réalisation

Le kit comprend le composant, la procédure d'installation, le contrat API et les tests essentiels. Son téléchargement suit les droits définis en section 04. La preuve de recette est un protocole daté et, si disponible, une trace de test navigateur ; pas un faux résultat de laboratoire serveur.

Utiliser un sous-domaine de démonstration sans cookie d'authentification HAAS partagé. Les données sont exclusivement fictives. Ne jamais cacher en service worker les profils privés, sessions, endpoints d'authentification ou réponses de modération de la plateforme principale.

Limites connues

Le stockage local peut être effacé ou indisponible ; le navigateur doit l'indiquer. Le mode privé n'est pas garanti. Hors connexion, aucune synchronisation n'est promise tant qu'un accusé serveur n'a pas été reçu. Ce kit ne rend pas toute la plateforme HAAS utilisable hors ligne.

Hébergement Déploiement retenu : B2 peut utiliser un projet Vercel distinct (`demo.example.com`) et un endpoint fictif distinct (`demo-api.example.com`) sur le VPS. Ils sont hors du domaine de cookie `.haas.example.com` réservé à la plateforme. Les noms restent des exemples, pas des sites configurés.

18 / PRODUIT ET COLLABORATION

Administration et protection des contenus

F09 — un back-office réduit mais opérationnel, avec responsabilité et historique.

Écrans d'administration P0

Une file de signalements, une file de revue des capsules, une liste de comptes et une liste de laboratoires en incident. Les filtres sont simples : état, date et catégorie. Aucun tableau de bord commercial ni outil de surveillance comportementale.

Un signalement référence une ressource et un motif : secret exposé, contenu abusif, droits incertains, solution trompeuse ou autre. Texte de 20-1 000 caractères ; un signalement actif par membre et ressource limite les doublons.

Traitement

Le modérateur consulte le contexte, choisit « Sans suite », « Demander une correction », « Masquer » ou « Retirer la version ». L'action contient un motif et une date. L'auteur reçoit une notification. Une contestation peut être transmise par le canal de contact indiqué dans l'interface ; aucune suspension irréversible automatique sur la seule base d'un nombre de signalements.

Secret ou donnée personnelle publié par erreur

Masquer immédiatement la ressource et révoquer les caches applicatifs associés. Avertir l'auteur de révoquer le secret à sa source ; supprimer l'affichage ne rend pas un secret à nouveau sûr. Éviter d'inclure la valeur dans les journaux, courriels et captures de recette.

Données et conservation proposées

Courriel privé ; aucune pièce d'identité collectée par HAAS. Les vérifications d'identité du concours relèvent de l'organisateur et ne sont pas réimplémentées dans l'application. [R, art. 4 et 12]

Politique opérationnelle à valider avant ouverture : journaux techniques 30 jours, traces d'audit et rapports de tests 90 jours pour l'évaluation, sauvegardes glissantes 7 jours. Ce sont des durées de conception, **pas des délais légaux affirmés**.

Prévoir une demande de suppression ou d'export du compte ; le traitement peut être manuel en P0, suivi par ticket et journalisé. La conservation d'une contribution après anonymisation dépend des conditions d'usage validées ; ne pas la présumer. Les obligations de HAAS et la politique de confidentialité de l'organisateur ne sont pas interchangeables.

Recette : AC25-AC27 ; le retrait doit être contrôlé par accès direct, recherche, ancien lien et téléchargement.

Modérer les cas, pas programmer le worker [A]

Ajouter une file de cas soumis, la lecture de leurs révisions et une décision motivée par un réviseur distinct. L'intégration exécutable dépend d'un manifeste de release relu. Aucun champ d'administration ne fournit une commande, une URL à charger ou une classe PHP à instancier.

Modération des projets

Ajouter le type project à la liste de ressources signalables. Réutiliser la file et le journal existants. Masquer un projet protège l'accès public à ses demandes et descendants ; expurger les liens de source dans les capsules, et revoir tout contenu sensible reproduit. La modération de visibilité n'édite pas le texte au nom du propriétaire. Une suspension retire aussi le membre de l'annuaire. Tests AC59/AC62 obligatoires.

19 / EXPÉRIENCE UTILISATEUR

Navigation et écrans

Une communauté facile à parcourir

Navigation principale **Explorer · Projets · Développeurs**, avec **Demander de l'aide** comme action. Dans le compte : Mon espace, Mes contributions, Notifications. Sur mobile, un menu accessible regroupe les liens sans multiplication de barres. Lire les contenus publics reste possible sans compte ; l'authentification est demandée pour participer.

L'accueil conserve la promesse de construire ensemble et donne deux entrées : « **Faire avancer mon projet** » et « **Donner un coup de main** ». Explorer les projets, poser une question et trouver des membres restent accessibles sans tunnel. Trois aperçus éditoriaux explicitement étiquetés suffisent ; pas de statistiques d'adoption inventées. Le laboratoire apparaît ensuite comme exemple de collaboration approfondie.

Vingt-trois familles d'écrans

Repère	Écran ou groupe	Action principale
UX01	Accueil	Comprendre et découvrir.
UX02	Explorer demandes/capsules	Chercher une discussion ou une solution.
UX03	Connexion, inscription et récupération	Entrer et gérer l'accès.
UX04	Mon espace	Reprendre projets, demandes, brouillons et favoris.
UX05	Question ou demande guidée	Formuler une aide utile sans code imposé.
UX06	Discussion et propositions	Échanger dans le contexte.
UX07	Accepter ou rouvrir	Exprimer une décision attribuée.
UX08	Édition/revue capsule	Documenter et soumettre à un autre réviseur.
UX09	Capsule publiée	Comprendre et réutiliser.
UX10	Rapport laboratoire	Lire une observation réelle.
UX11	Profil, favoris et retours	Découvrir le travail et conserver une solution.
UX12	Notifications	Retrouver une action dans son contexte.
UX13	Administration	Modérer, revoir et tracer.
UX14	B2 isolé	Tester la reprise après coupure.

Repère	Écran ou groupe	Action principale
UX15	Cas documentaire	Décrire un comportement à vérifier.
UX16	Comparaison	Examiner deux observations comparables.
UX17	Fiche de vérification	Identifier version, contributions et limites.
UX18	Projets : liste et détail	Découvrir un projet et ses échanges.
UX19	Éditeur de projet	Présenter/publier son travail et ouvrir une demande liée.
UX20	Développeurs et préférences	Découvrir les membres volontaires par technologie.
UX21	Occasions de contribuer	Choisir un petit besoin concret.
UX22	Proposer un coup de main	Relire le résumé et consentir avant l’envoi.
UX23	Offres et premier échange	Décider, rejoindre le fil et consulter les progrès.

Chaque famille traite chargement, vide, succès, erreur et accès interdit. Champs et listes sont utilisables à 360–430 px, au clavier et au zoom ; code dans son propre conteneur. Les nouveaux écrans réutilisent les composants, tokens, focus et messages du système existant. Spécifications détaillées : docs/design/SCREEN_SPECIFICATIONS.md.

20 / EXPÉRIENCE UTILISATEUR

Direction artistique HAAS

Une identité calme, nette et technique, sans reprendre les logos des organisateurs comme s’ils validaient le produit.

Principes visuels

Confiance sans froideur. Fond clair, texte sombre, espaces généreux et une couleur d’action. L’interface met en évidence la personne qui aide, la version concernée et le résultat, plutôt que des effets décoratifs.

Jeton	Valeur proposée	Usage
Encre	#102A2E	Titres, navigation, texte principal.
Papier	#F5F7F4	Arrière-plan général.
Surface	#FFFFFF	Cartes et formulaires.
Action	#087F73	Bouton principal et focus renforcé.

Jeton	Valeur proposée	Usage
Menthe	#BFE8D5	Accents doux ; texte sombre uniquement.
Vigilance	#8A4B10	Avertissements avec icône et libellé.
Ligne	#DCE5DF	Séparateurs et bordures.

Typographie d'interface proposée : pile système `system-ui`, sans téléchargement obligatoire de police. Texte de base 16 px, interligne 1,5 ; grille d'espacement de 8 px ; arrondis de 12 px. Le nom reste remplaçable par configuration, sans chaîne « HAAS » disséminée dans le code métier.

Accessibilité à vérifier

Cible WCAG 2.2 AA pour les écrans du parcours principal : contraste textuel d'au moins 4,5:1 pour le texte courant, focus visible, formulaires étiquetés, erreurs explicites et information jamais portée par la couleur seule. Il s'agit d'une cible de recette, pas d'une certification. [S09]

Microcopie

« Votre brouillon est enregistré » seulement après confirmation du serveur. « Conservé sur cet appareil » pour une sauvegarde locale. « Tests réussis pour cette version » plutôt que « Code sécurisé ». « Aucun résultat pour ces filtres » plutôt que « Erreur inconnue ».

Écran de principe — capsule

En-tête : titre + version + technologies. **Colonne principale** : contexte, correction, code, procédure, limites. **Panneau de preuve** : statut précis, date, scénario, bouton « Rejouer ». **Bas de page** : contributeurs et retours de réutilisation.

Cette planche définit une hiérarchie d'information ; elle ne prouve pas qu'une interface a déjà été développée. Les composants doivent être testés sur contenu long, zoom 200 % et appareil mobile.

Précisions de lisibilité [A]

Texte secondaire #506367 ; bordure de contrôle #718782 ; erreur #B42318 sur #FFF1F0. La ligne #DCE5DF reste décorative et n'identifie pas seule un champ. Corps 16 px et cible tactile 44 px sont des choix HAAS. Ne pas mettre un texte blanc sur menthe ni du texte d'action clair sur menthe. Les tokens et leur script de contrôle sont dans le ZIP.

Les états sont nommés et pas seulement colorés. Détails techniques secondaires repliables ; résultats et limites restent visibles. La palette contrôlée ne constitue pas une certification WCAG de l'application.

Priorité du contenu

Les personnes, projets et échanges précèdent les compteurs techniques. Les phases de projet et disponibilités ont des libellés, pas seulement des pastilles colorées. « En ligne » pour un projet est une déclaration de l'auteur, jamais une disponibilité automatiquement vérifiée. La fiche « Testé » n'existe que pour une version réellement exécutée. Aucun nouveau thème ou famille de couleurs n'est ajouté pour l'annuaire.

21 / ARCHITECTURE ET SÉCURITÉ

Architecture de référence

Un monolithe modulaire Laravel et une SPA React. Une infrastructure légère, explicable à deux.

Topologie finale [U]

React/TypeScript est servi sur Vercel ; Laravel, PostgreSQL, jobs et service LAB local tournent sur un seul VPS Systalink. Le navigateur appelle l'API directement en HTTPS. Les sauvegardes sont chiffrées et distantes. Le laboratoire ne dispose pas des secrets applicatifs et ne charge pas le bootstrap Laravel principal.

Deux origines, même parent de confiance : `app.haas.example.com` et `api.haas.example.com` sont des exemples. Les cookies sont bornés à `.haas.example.com`. B2 reste hors de ce périmètre. CORS, TLS, sessions et CSRF sont testés dans le vrai navigateur, pas supposés corrects parce que la connexion locale marche. [D03, D06]

Versions et découpage

La cible Laravel 13 / PHP 8.4 du dossier est conservée, sous réserve de compatibilité et de verrouillage dans VERSIONS.md. PostgreSQL et les versions React/Vite/Node sont choisies et testées ensemble. Pas de dernière version flottante. [S01]

Identity, HelpRequests, Collaboration, Capsules, Lab, Moderation et Notifications restent les domaines du monolithe. VerificationCases, Comparisons et Evidence complètent la boucle sans deuxième moteur de test. Controllers / Requests / Data / Services / Politiques / Ressources gardent leurs responsabilités ; React est organisé par fonctionnalités.

Limites explicites

Un seul run laboratoire actif globalement ; comparaisons séquentielles. Pas de Kubernetes, base managée ou Redis obligatoire. Pas de préproduction distante permanente : recette locale/CI et, si nécessaire, environnement temporaire approuvé. Le VPS est un point unique de panne, pas une architecture haute disponibilité. Les seuils de charge doivent être mesurés avant ouverture.

Extensions sans nouvelle infrastructure

Projects possède ses Requests/Data/Services/Politiques/Queries/Ressources. Identity porte la découverte volontaire des membres. Les demandes gardent le même fil et une référence projet facultative. React ajoute features/projects et features/developers. Aucun nouveau serveur, websocket, moteur social ou synchronisation Git n'est nécessaire à ces choix. Les performances réelles restent à mesurer.

22 / ARCHITECTURE ET SÉCURITÉ

Dépôt et conventions de développement

Une organisation que les deux développeurs comprennent et peuvent maintenir l'un sans l'autre.

Monorepo proposé

haas/	
backend/	Laravel : app, routes, tests, migrations
frontend/	React : src/features, components, tests
modules/	B1 et B2, cas et notices de diffusion
docs/	architecture, API, recette, droits, IA
ops/	deployment, backup, restore, health
.github/	workflows, templates, CODEOWNERS
README.md	installation et parcours de démonstration

Dans Laravel : Controllors pour l'orchestration HTTP, FormRequests pour validation, Policies pour accès, Services ou Actions pour les règles métier, Resources pour les réponses, Jobs pour les traitements différés. Aucun contrôleur ne décide seul des droits ou ne fabrique un rapport de laboratoire.

Dans React : fonctionnalités séparées, composants de formulaire partagés, client API unique, état de requête explicite. Pas de logique d'autorisation définitive côté frontend. Un schéma d'erreur commun évite les notifications incohérentes.

Règles d'équipe

Branches courtes issues de `main` ; une pull request par objectif cohérent ; revue par l'autre développeur avant fusion. La séquence active est backend complet et vérifié, puis frontend après `GO_FRONTEND`. Le contrat des écrans est préparé pendant la phase backend, sans générer React avant le gate.

Les identifiants d'exigence et de recette figurent dans la PR. Le réviseur vérifie droits, tests, limites et provenance du code, pas seulement le formatage. Une modification de schéma contient migration, stratégie de retour et mise à jour du dictionnaire.

Documents vivants

`ARCHITECTURE.md`, `OPENAPI.yaml`, `THIRD_PARTY_NOTICES.md`, `AI_USAGE.md`, `DECISIONS.md`, `RUNBOOK.md` et `RELEASE_CHECKLIST.md`. Les fichiers `.env.example` contiennent des noms de variables et des valeurs fictives, jamais les secrets de démonstration.

Le code du concours reste privé sauf décision explicite compatible avec le règlement et les droits de diffusion. Aucun dépôt existant n'est renommé pour faire croire qu'il a été créé pendant l'événement.

Définition de prêt : besoin compris, règle documentée, droits clarifiés, test d'acceptation identifié et charge compatible avec le lot.

Pack Codex actif

122 lots ordonnés ; 106 identifiants précédents préservés et compléments BH/FH/RH. Lire le prochain lot et les consignes locales, créer un petit commit cohérent avec ses tests, conserver le SHA réel et la reprise de session. Ne pas pousser, acheter ou déployer sans autorisation. Préserver un dépôt déjà entamé et son suivi ; le pack n'autorise aucun reset. Les fichiers de référence initiaux ne sont pas inclus dans ce dossier.

23 / ARCHITECTURE ET SÉCURITÉ

Modèle de données · collaboration

Des relations explicites et des contraintes de base protègent les règles métier.

Tables principales

Table	Champs et relations structurants
users	id UUID, email unique, handle unique, password, role, status, verified_at, is_demo.
profiles	user_id unique, bio, country nullable, primary_language, github_url nullable.
technologies / user_technologies	Référentiel et relation utilisateur-technologie unique.
help_requests	id, author_id, title, goal, expected, observed, attempts, environment, state, lock_version.
request_technologies	request_id, technology_id, version_label ; paire unique.
comments	id, request_id, author_id, body, edited_at, hidden_at ; édition historisée.
proposals	id, request_id, author_id, diagnosis, fix, verification, limits, state.
resolutions	id, request_id, proposal_id, accepted_by, validation_note, accepted_at, revoked_at.

Les contenus de code sont stockés comme texte inerte, avec langage et taille contrôlés. Un champ utilisateur ne devient jamais du SQL, une commande shell ou une expression à évaluer.

Cardinalités et intégrité

Un utilisateur possède plusieurs demandes ; une demande possède plusieurs commentaires et propositions. Une résolution référence une proposition **de cette même demande**. Le service vérifie cette appartenance ; une contrainte relationnelle ou un contrôle transactionnel testé empêche la référence croisée.

Une seule résolution non révoquée par demande : index unique partiel sur request_id lorsque revoked_at IS NULL. Les réouvertures conservent les enregistrements précédents.

Historique et timestamps

Création et modification en UTC, affichage localisé ; le calendrier du concours est toujours rappelé en heure de Dakar. content_revisions conserve auteur, type, ressource, numéro de révision et valeurs modifiées nécessaires, sans stocker de secrets volontairement supprimés.

Index de départ

Demandes sur (state, created_at), (author_id, created_at) ; propositions sur (request_id, created_at) ; commentaires idem ; unicité des étiquettes et handles. La recherche P0 utilise les champs textuels indexés ou une stratégie simple mesurée ; aucun index n'est ajouté sans besoin de requête identifié.

Les tables techniques Laravel — sessions, jobs, failed_jobs, notifications et tokens de réinitialisation — sont documentées séparément, avec accès et rétention adaptés.

Demandes et cas [A]

Ajouter help_intent aux demandes avec compatibilité des enregistrements antérieurs.
verification_cases possède exactement un parent (demande ou version) ;
verification_case_revisions conserve les textes et la revue propre à une révision. Les contraintes, index et nouvelles données sont décrits en section 45.

Données communautaires

Ajouter projects, pivot project_technologies unique, FK help_requests.project_id nullable sans cascade destructive, préférences directory_visible/help_availability/contribution_interests/availability_updated_at dans profiles. Les états de publication et d'avancement du projet restent distincts. Expected/attempts deviennent nullable pour ask_question uniquement, avec règles conditionnelles de FormRequest. Dictionnaire et index détaillés : section 56.

Données F18

HelpOffer, états typés, consentements datés, expiration et lien à un fil validé ; projets help_open=false et catégories ; index pending unique et projection publique source_offer_id unique. L'acceptation seule ne produit aucune preuve de correction. Le dictionnaire complémentaire est en section 59.

24 / ARCHITECTURE ET SÉCURITÉ

Modèle de données - capsules et preuves

Une preuve est toujours rattachée à un contenu, une version et un contexte d'exécution.

Tables complémentaires

Table	Champs et contraintes structurants
capsules	id, slug unique, source_request_id nullable, owner_id, editorial_origin, visibility.
capsule_versions	id, capsule_id, version_label, body, limits, state, reviewer_id, published_at ; version unique par capsule.
capsule_contributors	version_id, user_id, contribution_role ; unicité du triplet.

Table	Champs et contraintes structurants
artifacts	version_id, private_path, sha256, size, distribution_status, notices_path.
lab_definitions	version_id, runner_key autorisé, code_digest, suite_digest, enabled.
lab_runs / lab_results	user_id, definition_id, scenario, request_key, state, timestamps ; assertions attendues et observées.
reuse_reports / favorites	Retour unique par membre-version ; favori unique par membre-capsule.
reports / audit_events	Ressource signalée, motif, traitement ; acteur, action, ressource et métadonnées autorisées.

Idempotence des commandes API

Une table `api_idempotency` conserve utilisateur, route, clé, empreinte de charge utile, statut et réponse, avec unicité de l'ensemble utilisateur-route-clé. La même clé avec un contenu différent reçoit un conflit 409. Conservation proposée : 24 heures pour les commandes de création.

Les doublons de laboratoire sont aussi empêchés par la clé utilisateur-requête. Ce mécanisme évite deux lancements accidentels ; il ne réutilise pas silencieusement un ancien rapport lorsque l'utilisateur demande volontairement un nouveau test avec une nouvelle clé.

Séparation des données de test

Dans la base de laboratoire : `test_events` et `test_orders` incluent systématiquement `run_id`. Une contrainte unique protège (`run_id`, `event_id`). La purge supprime uniquement les fixtures d'une exécution identifiée, jamais des données métier de l'application.

Suppression et retrait

Un retrait documentaire n'est pas une suppression technique du rapport. Les copies de contenu contenant un secret doivent toutefois être expurgées suivant la procédure de sécurité. Une suppression de compte révoque sessions et favoris ; le sort des contributions suit les conditions d'usage approuvées et le traitement documenté.

Le dictionnaire détaillé précise pour chaque champ type, nullabilité, source, niveau de confidentialité et index. Toutes les migrations sont testées sur une base neuve puis sur un jeu représentant une version précédente.

Comparaisons [A]

`comparison_profiles` est aligné sur le registre de release. `comparison_runs` fige le profil, les entrées, les versions, les digests et les deux enfants `lab_runs`. `case_scenario_links` associe une révision humaine au scénario livré. L'unicité d'un côté par comparaison et les réservations communes protègent contre les créations concurrentes ; l'API publique n'écrit jamais le rapport.

25 / ARCHITECTURE ET SÉCURITÉ

API · lecture et collaboration

Une API versionnée avec des commandes métier, pas une mise à jour libre de tous les champs.

Convention

Préfixe `/api/v1`, JSON UTF-8, identifiants UUID, dates ISO 8601 UTC. Listes sous `data` et `meta`.
Validation serveur indépendante du frontend ; pagination maximale 50. Les routes d'authentification sont séparées des ressources publiques.

Méthode et route	Usage	Accès / résultat
GET <code>/me</code>	Compte et permissions courantes	Session requise.
GET <code>/technologies</code>	Référentiel	Public ; cache limité.
GET <code>/requests</code>	Recherche des demandes	Filtrage de visibilité serveur.
POST <code>/requests</code>	Brouillon ou publication validée	Membre vérifié ; 201 ; clé de requête.
GET <code>/requests/{id}</code>	Détail et contexte	Public si publié ; sinon propriétaire.
PATCH <code>/requests/{id}</code>	Clarifier les champs autorisés	Propriétaire ; verrou de version.
POST <code>/requests/{id}/comments</code>	Ajouter un échange	Membre vérifié ; 201.
POST <code>/requests/{id}/proposals</code>	Proposer une résolution	Membre vérifié ; 201.
POST <code>/requests/{id}/resolve</code>	Accepter une proposition	Auteur ; transaction ; 200.
POST <code>/requests/{id}/reopen</code>	Réouvrir avec motif	Auteur ; transaction ; 200.
POST <code>/requests/{id}/archive</code>	Archiver avec motif	Auteur ou modération selon règle.

Authentification

POST `/register`, POST `/login`, POST `/logout`, POST `/forgot-password`, POST `/reset-password`, confirmation de courriel signée et renouvellement du lien. La SPA initialise la protection via `/sanctum/csrf-cookie` puis transmet les cookies et l'en-tête CSRF suivant la configuration Sanctum. [\[S02\]](#)

Aucune route d'API ne rend le mot de passe haché, les jetons, l'adresse IP ou le courriel privé dans un profil public. L'absence d'autorisation sur une ressource privée peut être rendue comme 404 pour éviter d'en confirmer l'existence.

Gestion de concurrence

Les modifications éditoriales fournissent `lock_version` ; une version périmée déclenche 409 et une invitation à recharger. Une commande de résolution ne peut pas cibler une proposition d'une autre demande.

L'OpenAPI livré décrit les corps, erreurs, scopes d'accès, états et exemples fictifs. Les contrats sont vérifiés dans les tests d'intégration, pas laissés comme documentation décorative.

Intention et ressources complémentaires [A]

Le champ `help_intent` précise le besoin sans modifier le cycle. Les routes de cas, comparaisons et fiche sont détaillées section 45. Elles utilisent les mêmes erreurs, permissions, UUID, `lock_version` et clés d'idempotence. Un valide UUID n'établit ni la visibilité du parent ni la compatibilité de deux versions.

Contrats de communauté à ajouter

GET/POST `/projects` ; GET/PATCH `/projects/{project}` ; POST `/projects/{project}/publish` ; POST `/projects/{project}/archive` ; GET `/projects/{project}/requests` ; GET `/developers` ; extension de la route de mise à jour du profil courant. POST `/requests` admet `project_id` facultatif et `ask_question`, sans bypass de propriété. Les profils/demandes liés sont filtrés pour chaque acteur. Aucun identifiant propriétaire ni URL externe exécutée n'est accepté. Voir section 56.

API F18

Occasions d'aider, réglages du projet, offre privée, acceptation explicite create/attach, refus/retrait, progression depuis les contenus autorisés. Contrat, exemples et erreurs : section 59 et docs/api/COUPS_DE_MAIN_API.md. Réutiliser les contrôles de demande ordinaires, pas une seconde API concurrente.

26 / ARCHITECTURE ET SÉCURITÉ

API · preuves et erreurs

Le client peut demander un test ; seul le serveur peut produire son résultat.

Routes complémentaires

Méthode et route	Règle
GET <code>/capsules</code> et <code>/capsules/{slug}</code>	Versions publiées et visibilité autorisée uniquement.
POST <code>/capsules</code> ; POST <code>/capsules/{id}/versions</code>	Brouillon ; auteur ou contributeur autorisé.
POST <code>/versions/{id}/submit-review</code>	Déclenche la revue, sans publication immédiate.
POST <code>/admin/versions/{id}/publish</code>	Réviseur distinct ; contrôle de droits et contenu.

Méthode et route	Règle
GET /versions/{id}/artifact	Contrôle de distribution ; lien signé court ou transfert autorisé.
POST /lab-runs ; GET /lab-runs/{id}	Demande bornée ; accès à la preuve publique ou à son propre run.
POST /versions/{id}/reuse-reports	Retour humain versionné, pas résultat machine.
PUT /capsules/{id}/favorite ; DELETE idem	Action idempotente réservée au membre.
POST /reports ; PATCH /admin/reports/{id}	Signalement puis décision motivée.

Exemple de lancement

```
{
  "capsule_version_id": "UUID_DE_LA_VERSION",
  "scenario": "duplicate_event"
}
```

En-tête Idempotency-Key obligatoire. Réponse **202** avec `data.id`, `data.state`: "queued" et une URL de suivi. Une requête demandant `code`, `command`, `url` ou un scénario non autorisé est rejetée, pas ignorée silencieusement.

Erreurs normalisées

`error.code`, `error.message`, `error.fields` facultatif et `request_id`. Codes HTTP : 401 session absente, 403 action interdite, 404 introuvable, 409 conflit, 419 session/CSRF expiré, 422 validation, 429 quota et 503 laboratoire indisponible.

L'interface traduit ces cas et conserve les données saisies quand c'est sûr. Une réponse 500 affiche un message neutre avec identifiant de support ; jamais les chemins serveur, SQL, cookies ou traces internes.

Important : aucune route publique de type PATCH /lab-runs/{id} n'autorise à écrire `passed`, `observed` ou les digests. Le worker dispose d'un accès interne limité ; le rapport associe le même digest au code livré et au test annoncé.

Réponse de comparaison [A]

POST /comparisons reçoit un profil et un scénario approuvés, retourne 202 avec URL de suivi. GET expose la progression et les enfants disponibles. L'état technique `completed` est distinct de `outcome`. Aucun calculateur frontend n'invente une amélioration ; l'erreur d'un côté donne une conclusion inexploitable. Rapports anciens explicitement datés.

27 / ARCHITECTURE ET SÉCURITÉ

Sécurité intégrée au produit

N01, N06 et N07 — protéger les comptes, les contenus et la chaîne de fabrication.

Menaces prioritaires et réponses

Menace	Contrôle exigé
Lecture ou modification d'un autre compte	Politiques Laravel, tests négatifs et champs explicitement autorisés.
Injection de script dans le code ou Markdown	HTML brut désactivé ; rendu sûr ; URL contrôlées ; aucun eval.
Requête forgée / vol de session	Cookies sécurisés, CSRF, renouvellement de session, HTTPS.
Secret publié dans une demande	Avertissement, détection indicative, signalement et retrait rapide.
Exécution serveur détournée	Scénarios autorisés, aucun code utilisateur exécuté, worker et base restreints.
Coût ou saturation du laboratoire	Quotas, concurrence bornée, délais et arrêt d'urgence.
Compromission de dépendance ou workflow	Fichiers verrouillés, revue des versions, permissions minimales et actions épinglées.

Paramètres d'exploitation

APP_DEBUG=false, cookies de session HttpOnly et Secure, politique SameSite=Lax pour les deux origines same-site de confiance, avec CSRF et CORS explicites, hôtes autorisés définis. Ne pas appliquer HttpOnly au cookie CSRF que le client doit lire selon le mécanisme choisi.

Politique CSP testée avec les assets compilés ; aucune ouverture générale à des scripts tiers. Limites de taille côté proxy et Laravel. Connexions PostgreSQL non exposées sur Internet. Les fichiers privés ne sont jamais servis par un chemin public prévisible.

Contrôles administratifs

Comptes d'administration individuels, secrets distincts par environnement et changement de rôle audité. L'accès d'exploitation passe par clés SSH ; pas de mot de passe partagé dans un document ou une discussion.

Un incident impose : contenir, conserver les seules preuves nécessaires, révoquer les secrets touchés, corriger puis documenter. La disponibilité de la démonstration ne justifie pas de contourner une alerte critique.

Licences et attributions

Nomenclature des dépendances directes et transitives, outils de développement séparés du code distribué, composants sans licence identifiée bloqués jusqu'à revue. L'audit automatique aide à repérer ; il ne prononce pas à lui seul la recevabilité au concours. [R, art. 6]

Recette : AC03, AC04, AC07, AC18-AC20, AC25-AC28 et contrôles CI.

Risques propres à l'atelier [A]

Cas malveillant pris pour du code, sélection d'un runner arbitraire, paire incompatible, dépassement du quota par une deuxième route, faux résultat partiel et fuite après retrait : chacun exige un test négatif. Un cas ou une capsule donné à un assistant reste du contenu non fiable, pas une instruction de confiance. Aucun appel à une IA externe n'est requis dans le P0.

Menaces communautaires complémentaires

IDOR sur projet, association trompeuse d'une demande au projet d'un tiers, fuite de brouillon par compteurs, récupération serveur d'un lien externe, exposition du courriel dans l'annuaire et cache conservé après opt-out. Réponses : champs whitelist, Politiques sur le parent, aucune récupération automatique, Ressources minimales et invalidations testées. AC53-AC68 complètent la recette.

Sécurité F18

Les deux consentements, la visibilité pending, l'identité des acteurs, le lien au bon fil, l'idempotence et les courses fermeture/retrait/suspension sont obligatoires. Une modération n'usurpe ni l'accord ni le commentaire du propriétaire ou du proposant. Aucun droit de dépôt, rôle ou statut d'équipe acquis après acceptation.

28 / ARCHITECTURE ET SÉCURITÉ

Objectifs non fonctionnels

Des seuils de recette proposés, à mesurer sur l'environnement réel ; aucun niveau de service commercial n'est promis.

Cibles mesurables

Code	Exigence	Méthode proposée
N01	Aucun accès interdit accepté	Batterie de tests de politiques et endpoints.
N02	p95 lecture API ≤ 800 ms ; écriture $\leq 1\,200$ ms, hors laboratoire	20 utilisateurs virtuels pendant 10 min, base fictive de 1 000 demandes.
N02	Écran principal exploitable ≤ 3 s	Profil mobile de test consigné : 4 Mbit/s, latence 150 ms, appareil défini.
N03	Parcours clavier complet ; zoom 200 %	Vérification manuelle des écrans principaux.
N04	Restauration en moins de 2 h ; perte cible ≤ 24 h	Restauration d'une sauvegarde sur environnement vierge.

Code	Exigence	Méthode proposée
N05	Fusion uniquement après contrôles requis	PR témoin volontairement en échec.
N06	Aucun composant à licence inconnue accepté sans décision	Registre et revue des fichiers de verrouillage.
N07 / N08	Contributions attribuées ; aucune preuve simulée présentée comme réelle	Contrôle de données de démo, digests et rapports.

Performance raisonnable

Chargement différé des écrans lourds ; édition de code limitée à un composant léger. Objectif initial de JavaScript transféré au premier écran : 300 Ko compressés maximum, hors assets différés. Images non indispensables supprimées du P0.

Les seuils incluent le nom de la machine, le commit, le jeu de données et les outils employés. En cas d'écart, fournir la mesure et la décision d'amélioration plutôt qu'un pourcentage de performance inventé.

Robustesse et disponibilité

Vérification automatique du service toutes les cinq minutes proposée, alertes vers les deux développeurs. Aucun engagement « 99,99 % » sans architecture et historique correspondants. Une file de laboratoire indisponible ne doit pas empêcher la lecture des capsules ni la publication de demandes.

Compatibilité

Recette principale sur Chrome desktop et Android réel, puis vérification Firefox et Safari si disponibles. La compatibilité n'est déclarée que pour les environnements effectivement testés. Le protocole B2 rappelle ses limites de stockage et de travail hors connexion.

Toute dégradation acceptée doit être documentée dans les notes de version et validée par les deux développeurs. Les exigences de sécurité et de sincérité ne sont pas dérogeables pour la démonstration.

Mesures de l'atelier

Conserver les conditions de test indiquées ci-dessus. Mesurer séparément attente en file et temps de comparaison, avec deux runs réels. Le délai de 50 s après démarrage est une borne proposée à implémenter, pas une performance acquise. La mesure du parcours comprend aussi compréhension des types de preuve et contribution d'un cas sans assistance pas à pas.

29 / QUALITÉ ET LIVRAISON

Recette fonctionnelle · parcours central

Des scénarios vérifiables à transformer en tests d'intégration ou en tests navigateur.

Identité et demandes

Test	Situation et résultat attendu
AC01	Inscription valide : compte créé, courriel privé, lien de vérification utilisable une fois selon son état.
AC02	Compte non vérifié : lecture autorisée, publication et laboratoire refusés.
AC03	Un membre tente de modifier une demande d'autrui : 403/404, aucune mutation.
AC04	Le client envoie role=admin ou author_id tiers : rejet, aucune élévation.
AC05	Champ obligatoire absent : 422, erreur liée au champ, saisie conservée.
AC06	Double soumission avec même clé : une seule demande, même résultat.
AC07	Extrait contenant HTML/script : texte affiché sans exécution ; alerte de secret si motif détecté.

Collaboration et capitalisation

Test	Situation et résultat attendu
AC08	Une proposition structurée est publiée : demande en cours, notification unique.
AC09	Une personne non auteur accepte : refus ; l'auteur accepte : résolution créée.
AC10	Deux acceptations concurrentes : une seule résolution active.
AC11	Réouverture motivée : historique conservé, capsule liée signalée à revoir.
AC12	Auteur tente sa propre publication éditoriale : revue indépendante requise.
AC13	Correction d'une version publiée : nouvelle version ; ancien rapport inchangé.
AC14	Retour humain attaché à une version : visible comme déclaration, pas résultat machine.
AC15	Version retirée : absente des téléchargements et recherche normale, motif disponible si autorisé.

Preuve à conserver

Pour chaque test : version du logiciel, environnement, préconditions, étapes, attendu, observé, résultat, date et identifiant d'incident éventuel. Une capture seule ne remplace pas la vérification de l'état enregistré en base.

Les tests négatifs sont prioritaires : ne pas seulement confirmer qu'un administrateur peut tout faire. L'autre développeur exécute au moins une fois le parcours sans connaître le chemin exact d'implémentation.

Extension de recette

AC01-AC32 sont préservés. AC33-AC52 (section 48) couvrent les nouveaux parents, révisions, profils, quotas, exécutions, conclusions, retraits, attributions et écrans. AC47 est non applicable si le P1 de copie de contexte n'est pas livré ; il ne doit jamais être marqué réussi sans exécution.

Parcours communautaires supplémentaires

AC53-AC68 couvrent les projets, l'annuaire volontaire, les questions sans code et la chaîne projet→aide→connaissance. Les tableaux détaillés sont en section 57 et dans ACCEPTANCE_MATRIX.md. Il s'agit de cas à exécuter, pas de résultats déjà acquis.

30 / QUALITÉ ET LIVRAISON

Recette technique · preuves et exploitation

La fiabilité se démontre aussi quand une ressource est indisponible, qu'un accès est interdit ou qu'une étape échoue.

Laboratoires et résilience

Test	Situation et résultat attendu
AC16	B1 rejoué : assertions recalculées, identifiant neuf et digests exacts.
AC17	Un scénario connu échoue : état Échouée, attendu/observé conservés.
AC18	Code ou commande envoyé au laboratoire : 422, aucune exécution.
AC19	Quota ou concurrence dépassé : 429 ou mise en file bornée ; pas de saturation.
AC20	Worker interrompu : état Erreur/Délai dépassé après contrôle ; jamais un faux succès.
AC21	Deux utilisateurs lancent un test : aucune donnée de fixture partagée.
AC22	B2 hors connexion : enregistrement local explicite ; aucun accusé serveur inventé.
AC23	B2 répète l'envoi après reprise : une seule commande de démonstration.

Test	Situation et résultat attendu
AC24	Stockage local B2 indisponible : avertissement, aucune garantie trompeuse de sauvegarde.

Produit et livraison

Test	Situation et résultat attendu
AC25	Contenu masqué : inaccessible via URL directe, recherche et ancien lien de kit.
AC26	Compte suspendu : sessions révoquées, actions interdites.
AC27	Jeu de démonstration : exclu des indicateurs d'usage réel.
AC28	PR introduisant une alerte bloquante : fusion/déploiement empêchés.
AC29	Sauvegarde restaurée ailleurs : comptes de test, demandes et versions cohérents.
AC30	Retour au build précédent : service fonctionnel sans migration inverse destructrice.
AC31	Parcours mobile/clavier : aucune action essentielle inaccessible.
AC32	Recette complète A→B→C : résolution, publication, test et réutilisation reliés.

Stratégie de test proposée

PHPUnit pour les services et API ; tests React de formulaires et états ; Playwright pour le parcours navigateur et B2 ; vérification concurrente B1 sur PostgreSQL réel en CI. Les cas de sécurité et d'unicité ne sont pas remplacés par des mocks de base.

Les composants de test et leurs licences sont audités comme le reste de la chaîne, suivant le périmètre confirmé par l'organisateur. Une cible initiale de couverture de 80 % des lignes des services métier critiques est proposée ; elle ne dispense jamais d'un scénario d'acceptation important.

Paires et pannes [A]

La recette doit faire échouer la référence pédagogique, observer le candidat et provoquer une panne d'un côté. Vérifier un résultat mitigé et une régression sur des fixtures de test contrôlées. Une comparaison interrompue ou incompatible n'affiche pas d'amélioration. API puis navigateur sont testés dans leurs phases respectives.

31 / QUALITÉ ET LIVRAISON

Qodana Ultimate et contrôles de qualité

Un outil d'analyse complète les tests ; il n'est ni une certification de sécurité ni une validation juridique.

Accès retenu [U]

L'utilisateur déclare disposer d'Ultimate. Aucun nouvel achat Qodana n'est inclus dans le budget. Vérifier le projet, le token, la durée et les contributeurs couverts avant activation ; aucun compte n'a été inspecté dans cette livraison.

Qodana PHP est disponible avec Ultimate et prend en charge PHP, JavaScript/TypeScript ainsi que Laravel et React. La documentation distingue explicitement certaines fonctions Ultimate Plus : audit de licences, analyse de contamination et vérification de vulnérabilités ne sont pas attribuées automatiquement à Ultimate. Maintenir les inventaires et audits complémentaires requis. [D10, D11]

Configuration et seuils

Installer les dépendances depuis les verrous, fixer l'image/linter et le profil réellement compatibles. Analyser backend et frontend dans des configurations séparées si cela rend les rapports plus lisibles. Exclure les sorties générées, pas les services métier. Secrets Qodana uniquement dans la CI autorisée ; pas de token dans VITE_*, logs, captures ou dépôt.

Bloquer les nouvelles anomalies pertinentes de gravité élevée confirmées, l'échec du moteur d'analyse et le dépassement des seuils adoptés. Consigner la correspondance des sévérités et toute exception relue. Une baseline ne cache pas une régression. Une absence d'analyse ne produit jamais un contrôle vert.

Preuves

Rapport privé, SHA, profil, version du linter, date et résultat réellement obtenu. Tests unitaires, API, concurrence, navigateur et audit de dépendances restent distincts. Une licence disponible ne prouve ni l'exécution de la CI ni la qualité d'un dépôt non encore construit.

32 / QUALITÉ ET LIVRAISON

GitHub Actions · de la PR à la release

La chaîne livre la version testée, sans publication Vercel anticipée.

Étape	Résultat attendu
Préparer	Installer les versions verrouillées ; aucun secret dans les logs.
Vérifier	Pint, PHPStan/Larastan, TypeScript strict, lint et build.
Tester	PostgreSQL réel, règles métier, concurrence, cookies et parcours navigateur.

Étape	Résultat attendu
Auditer	Qodana Ultimate réellement exécuté ; inventaires/licences et recherche de secrets séparés selon outils disponibles.
Construire	Artefacts API, runner et React, empreintes et manifeste associés au commit accepté.
Relire	Autre développeur ; recette locale/CI et accord humain GO_PRODUCTION.
Livrer	API Systalink compatible d'abord, puis frontend Vercel, smoke tests et contrôle du domaine final.

L'ancien prérequis de préproduction permanente est supprimé. Un environnement distant temporaire peut être autorisé et chiffré ; les données et secrets de test restent distincts. Le frontend n'est développé qu'après GO_FRONTEND.

Sécurité et coordination

Permissions minimales, actions épinglées sur références vérifiées, aucune exécution de code non approuvé avec des secrets de production. Vérifier l'offre GitHub pour les protections et approbations réellement disponibles ; jamais de faux statut requis. [D09]

Ne pas laisser Vercel publier automatiquement la branche principale avant les contrôles. Choisir une promotion explicitement approuvée ou une branche de release protégée dont la mise à jour attend le backend. Relier chaque déploiement à son manifeste ; une nouvelle compilation requiert vérification de sa provenance. [D04]

Une clé SSH dédiée et un hôte vérifié sont utilisés pour le VPS ; aucun `StrictHostKeyChecking=no`. Sérialiser les releases, conserver les logs expurgés et recharger les workers après changement de code. Les modifications de schéma restent compatibles avec le frontend encore actif. [D07]

33 / QUALITÉ ET LIVRAISON

Déploiement final · Systalink et Vercel

Architecture cible active. Elle remplace le choix antérieur à deux serveurs ; aucune infrastructure n'est provisionnée par ce document.

Topologie retenue

```
Utilisateur / navigateur
|-- HTTPS --> Vercel : React + TypeScript + Vite
|
|       app.haas.example.com
|
|-- HTTPS --> Systalink : api.haas.example.com
|       Nginx -> PHP-FPM -> API Laravel
|               |-- PostgreSQL : haas_app
|               |-- sessions / queue / cache de base
|               |-- orchestrateur -> service LAB local restraint
|                               PostgreSQL : haas_lab
|
|-- sauvegarde chiffrée -> stockage distant
```

GitHub Actions + Qodana Ultimate -> contrôles -> accord humain
-> API Systalink -> frontend Vercel

Les hôtes `example.com` sont des EXEMPLES ; ils ne sont ni disponibles ni provisionnés par ce dossier. Le navigateur charge React depuis Vercel, puis appelle directement l'API HTTPS. Les réponses privées ne transitent pas par un cache public de Vercel. Vercel ne sert ni PHP ni PostgreSQL dans cette architecture. [D03, D06]

Ressources et limites

Élément	Cible retenue	Portée
VPS Systalink	Business : 2 vCPU, 8 Go RAM, 100 Go NVMe	Référence publique de départ ; éligibilité, région et panier à confirmer.
React/Vite	Un projet Vercel principal	Offre et droits des deux contributeurs à valider ; pas de fonctions serveur indispensables.
PostgreSQL	Bases distinctes <code>haas_app</code> , <code>haas_lab</code> ; B2 distincte	Même instance possible, rôles non super-utilisateurs et accès réellement restreints.
Exécuteur LAB	Un run actif global ; comparaison séquentielle	Service local sous un autre compte, sans <code>.env</code> Laravel et sans base métier.
Sauvegarde	Stockage distant ; chiffrement côté client	Le panier est distinct du VPS ; restauration à tester.
Recette	Environnements locaux/CI jetables	Pas de préproduction distante permanente facturée. Un environnement temporaire supplémentaire demande accord et budget.

Systalink affiche la référence Business ; le dimensionnement reste une hypothèse à éprouver, pas une garantie de débit. La gestion du VPS n'est pas présumée incluse. [D01]

Processus et responsabilités

`haas-app` exécute PHP-FPM et les jobs métier. Il utilise uniquement les permissions requises sur `haas_app`. Une identité de migration distincte applique les changements de schéma. Nginx n'expose que le dossier `public/` de Laravel, jamais la racine du dépôt.

L'orchestrateur autorise chaque opération, crée ses identifiants, réserve le quota et conserve les résultats dans la base métier. L'exécuteur `haas-lab` reçoit seulement des identifiants de scénarios et de manifestes approuvés, des identifiants de run et les fixtures canoniques autorisées. Le résultat est un JSON borné et validé. Le code de la brique est livré par la CI, pas téléchargé à la demande.

Frontière locale proposée : service séparé, canal Unix local et permissions de groupe permettant au seul orchestrateur d'émettre une demande. Aucun port public, shell arbitraire ou nom de classe transmis par le navigateur. Le runner est un paquet d'exécution minimal, séparé du bootstrap Laravel principal : lancer `php artisan` dans le projet principal sous un autre nom d'utilisateur ne suffit pas.

Le runner n'a ni APP_KEY, ni accès au code/configuration privés de l'API, ni identifiants SMTP, GitHub, Vercel ou sauvegarde. Son rôle SQL n'accède qu'aux fixtures ; il ne lit pas la queue applicative. Il renvoie ses observations à l'orchestrateur via le canal déjà ouvert. Les fichiers, sockets et identités effectifs figurent dans le runbook.

Droits et ressources du laboratoire

Séparer les répertoires /srv/haas-api, /opt/haas-lab et les espaces de travail temporaires. Retirer au compte laboratoire tout groupe donnant accès aux secrets APP. La configuration du service limite les fichiers accessibles, les familles réseau, la mémoire, le CPU et le nombre de processus ; vérifier ces options sur l'OS retenu. Pas de droit Docker, sudo générique ou montage du socket d'administration.

Les rôles PostgreSQL ne sont ni propriétaires de la base métier ni super-utilisateurs. Contrôler CONNECT, les schémas et les droits accordés par défaut à PUBLIC. Tester explicitement la lecture de users, la connexion à haas_app, la lecture du .env et l'ouverture des sauvegardes sous l'identité laboratoire : ils doivent échouer. La seule création d'un deuxième nom de base ne constitue pas cette preuve.

Une borne de ressources est définie après mesure ; point de départ proposé : au plus 512 Mio pour le runner, consommation CPU limitée et un processus d'exécution à la fois. Les traitements PostgreSQL, même isolés par rôle, consomment les ressources du même serveur : limiter aussi les connexions et durées de requêtes. Mesurer la latence de l'API pendant une comparaison avant d'accepter la configuration.

Limite : un incident de noyau, d'hôte ou de PostgreSQL reste partagé. En cas d'accès indu ou de perturbation significative de l'API, garder le laboratoire fermé et corriger les restrictions ou revoir l'hébergement. Ne jamais présenter un service suspendu comme un laboratoire livré.

Quotas et sincérité

Un run simple consomme une unité ; une comparaison en réserve deux atomiquement. Cinq unités par heure et par membre, une opération active par membre, **un run actif globalement**. La même clé idempotente ne réserve pas deux fois. Les deux enfants d'une comparaison sont successifs, avec données neuves et mêmes entrées attendues. La file est bornée ; un quota dépassé donne une réponse 429 explicite.

Les bornes retenues du cahier sont 15 secondes de traitement visées et 20 secondes maximum par enfant ; 50 secondes après démarrage pour la comparaison. Le temps en file est affiché séparément. Une interruption, un résultat mal formé ou une version indisponible donne erreur/délai dépassé et aucune conclusion favorable. Le réconciliateur libère les réservations d'exécution abandonnées sans fabriquer de test réussi.

Domaines, TLS et navigateur

Usage	Hôte d'exemple	Destination
Application principale	app.haas.example.com	Vercel, projet HAAS
API principale	api.haas.example.com	VPS Systalink
Démonstrateur B2	demo.example.com	Projet Vercel B2 séparé, si B2 est livré

Usage	Hôte d'exemple	Destination
API B2	demo-api.example.com	Vhost/pool/service distinct sur le VPS, uniquement données fictives

Seuls les hôtes de confiance appartiennent à `.haas.example.com`. Les deux hôtes B2 sont hors de ce périmètre : aucun cookie HAAS ne doit y être envoyé. Ils n'appellent pas la base métier. Leurs réglages et artefacts sont distincts ; cela n'ajoute pas un VPS.

Saisir les enregistrements DNS effectivement demandés par Vercel et l'adresse publique réellement attribuée par Systalink : aucune IP n'est inventée. TLS sur les quatre hôtes retenus, renouvellement surveillé. Session Secure/HttpOnly, CSRF lisible par le client selon Sanctum. Ne pas confondre même site avec même origine. Voir `AUTH_CORS_SANCTUM.md`. [D03, D06]

Déploiement Vercel

Projet principal : racine `frontend`, Vite, `npm ci`, `npm run build`, sortie `dist`. Ajouter la réécriture SPA pour les liens profonds et tester les assets : une URL de ressource manquante ne doit pas servir silencieusement un écran de connexion. Pas de réécriture proxy `/api` prévue ; le client emploie l'origine API configurée. [D06]

`VITE_API_URL` contient uniquement l'origine publique. Aucun secret dans `VITE_*`. La séparation Preview/Production porte sur domaines, variables et données. Les previews automatiques `*.vercel.app` ne sont jamais ajoutées en bloc à CORS ou aux domaines Sanctum. Une preview non autorisée reste une revue visuelle sur données fictives ; la recette fonctionnelle se déroule avec une API dédiée de test et des domaines compatibles. Aucune prévisualisation non fiable n'obtient les secrets ni les données de production.

Le forfait doit permettre le dépôt et la collaboration : Hobby n'est pas acquis comme option gratuite admissible ; le déploiement depuis une organisation GitHub privée et la qualité d'auteur des commits sont à vérifier. Ne pas rendre le dépôt public, falsifier un auteur ou partager un compte pour contourner les conditions. [D04, D05]

Livraison coordonnée

1. Exécuter validations, tests, contrats et Qodana sur le SHA retenu ; conserver les rapports réellement produits.
2. Construire les artefacts backend, runner approuvé et frontend pour la configuration cible ; identifier leurs empreintes dans le manifeste. Une reconstruction ultérieure n'est pas réputée identique sans nouvelle vérification.
3. Exécuter la recette sur données fictives ; obtenir la revue de l'autre membre et `GO_PRODUCTION`. Aucun secret de production dans les jobs qui analysent une contribution non approuvée.
4. Sauvegarder. Déployer sur Systalink une release API compatible avec le frontend encore en ligne. Appliquer uniquement les migrations revues et compatibles. Recharger les processus Laravel persistants. [D07, D09]
5. Tester l'API, son contrat, le runner et les droits. Publier ensuite l'artefact React approuvé sur Vercel ; contrôler domaine, variables et liens profonds.
6. Faire une connexion réelle, une requête authentifiée, une déconnexion et un parcours synthétique fictif via les domaines finaux. Lister le résultat de chaque contrôle.

La publication Vercel en production n'est pas un effet automatique non contrôlé de chaque push sur `main`. Choisir une promotion explicite par CI autorisée, ou une branche de release protégée dont la mise à jour attend le backend. Vérifier le comportement réellement offert par le compte. Les contrôles requis ne deviennent jamais facultatifs pour finir plus vite. [D04, D09]

Reprise et sauvegardes

Conserver au moins deux releases API et les déploiements Vercel correspondants. Un manifeste relie SHA, API, frontend, runner, migrations et empreintes. Le retour à un frontend précédent est possible seulement si le backend courant garde le contrat requis. Si le backend doit aussi revenir, vérifier la compatibilité de la base avant toute action. Pas de migration inverse destructive automatique.

Sauvegarder quotidiennement `haas_app` et les artefacts privés nécessaires ; conserver sept générations, chiffrées, sur stockage distant. Les fixtures laboratoire/B2 peuvent être recrées : elles ne remplacent pas une sauvegarde métier. Les clés de récupération sont conservées hors du VPS et transmises uniquement aux personnes autorisées. Tester une restauration hors production. Cibles proposées : perte maximale 24 h, reprise moins de 2 h ; ce sont des objectifs à mesurer, pas un SLA. [D02]

Un stockage hors du VPS mais chez le même fournisseur reste exposé à certains incidents du fournisseur ou du compte : droits distincts et seconde copie hors compte à considérer selon le budget. Ne pas promettre une indépendance totale sans la démontrer.

Supervision minimale

Surveiller disponibilité API/frontend, validité TLS, disque, connexion SQL, queue métier, opérations laboratoire bloquées et présence de sauvegarde. Un `/health` public reste minimal ; les détails de base et version sont protégés. Les logs corrélient par `request_id` sans mot de passe, code privé ou cookie. Les alertes parviennent aux deux développeurs et la personne de permanence est connue.

L'indisponibilité du laboratoire doit être visible mais ne doit pas empêcher de consulter une capsule ou de créer une demande. Le site n'affiche pas de promesse « 99,99 % », de haute disponibilité ou de montée en charge illimitée.

Vérifications avant ouverture

Valider les quatorze points de `docs/quality/DEPLOYMENT_GATE.md`, dont CORS refusé, cookie hors B2, un run global, tentative d'accès aux secrets sous l'identité runner, test de restauration et promotion liée au bon SHA. Le domaine, le forfait Vercel, SMTP, le panier Datacloud et le token Qodana sont des inconnues à lever, pas des valeurs que Codex peut inventer.

34 / EXÉCUTION ET SOURCES

Séquence de développement à deux

Backend complet puis frontend reste la décision de l'utilisateur. Pendant le backend, l'autre membre peut préparer contrats, cas de test, textes et maquettes documentaires ; pas de code React avant GO_FRONTEND. Les ajouts communautaires ne changent pas cet ordre.

Phase	Charge structurée	Sortie attendue
S — préparation	2 lots	Inventaire réel, sources, versions, contraintes, suivi.
B — backend	62 lots	F01-F18, questions, projets, annuaire, atelier et API B2 réellement testés.
Gate backend	Revue humaine	BACKEND_GATE complet et GO_FRONTEND explicite.
F — frontend	35 lots	API réelle, UX01-23, états, mobile/clavier et démonstrateur B2.
R — réception	7 lots	Pilote, artefacts, droits, sauvegarde, déploiement autorisé et preuves.

Répartition sans silos

Pendant B, une personne mène le cas métier, l'autre teste les droits, les contrats et les erreurs ; rôles inversés ponctuellement. Pendant F, une personne réalise l'écran, l'autre vérifie parcours, intégration et accessibilité. Chacun relit l'autre, aucun agent ne signe une revue humaine.

Les 122 lots sont des unités de travail proposées, pas une durée garantie ou un quota de commits. Réestimer disponibilité et reste à faire avant engagement. Si une partie existe, ajouter les lots nécessaires et revalider les gates concernés sans réinitialiser le dépôt. Les correctifs backend découverts pendant le frontend sont permis, testés et tracés.

35 / EXÉCUTION ET SOURCES

Backlog et réestimation

Périmètre ajouté, pas charge cachée

Les 106 identifiants précédents restent dans tasks.json. Dix lots backend BH01-BH10, cinq frontend FH01-FH05 et RH01 ajoutent les coups de main après les compléments communauté. Total **122 lots : S2/B72/F40/R8**. Les tests de chaque fonction appartiennent au lot ; la recette transverse réexécute les parcours et ne remplace pas ces tests.

L'ancienne hypothèse de 180 heures engagées et 216 heures disponibles ne chiffre pas cette extension. Les deux développeurs doivent estimer le reste à faire depuis le dépôt réel : fonctionnalités existantes, migrations, API, interface, tests, revue et marge. Le nombre de fichiers ou commits n'est pas une preuve de qualité.

Dépendances

Schéma/propriété projet précèdent publication et demande liée ; idempotence/audit existants sont réutilisés. Préférences volontaires précèdent annuaire public. Extension ask_question suit le HelpIntent existant. Tous les nouveaux backend précèdent la recette B39-B44 et GO_FRONTEND. Projets/membres/questions côté UI précèdent F25/F26. Les gates de déploiement restent applicables.

Réduction contrôlée

Supprimer les P1 avant les usages communautaires essentiels. Simplifier les raffinements des cartes, filtres secondaires et profils ; ne pas retirer propriété, visibilité parent, questions sans code, propagation des retraits ou tests de sincérité. Tout retrait de B2 ou autre livrable P0 exige décision humaine et mise à jour du pitch/cahier. Le laboratoire n'est pas l'unique critère de réception.

Définition de terminé

Règle métier, accès négatifs, contrat, service, données, écran, erreurs, tests, revue croisée et preuve liée au commit. Pendant la phase backend, la réception de la partie serveur est explicite sans prétendre que son écran est déjà fait.

36 / EXÉCUTION ET SOURCES

Budget simplifié et risques

Tarifs publics consultés le 1er octobre 2026. Ce tableau n'est pas un devis ni une autorisation d'achat. Les prix du panier, la TVA, les options et l'éligibilité à la remise priment.

Infrastructure Systalink

Poste	Référence publique	Tarif normal / mois	Avec 40 % de remise, si éligible
VPS Business	2 vCPU, 8 Go RAM, 100 Go NVMe	31 486 FCFA	18 891,60 FCFA
Stockage distant Starter	250 Go	3 280 FCFA	1 968 FCFA
Sous-total Systalink	Un VPS et sauvegardes	34 766 FCFA	20 859,60 FCFA

Calcul : $(31\,486 + 3\,280) \times 0,60$. Pour deux mois, hypothèse prudente : un mois remisé puis un mois normal, soit **55 625,60 FCFA**. Si seule l'instance et pas le stockage est remisée, premier mois **22 171,60 FCFA** et deux mois **56 937,60 FCFA**. Sans remise du tout, deux mois **69 532 FCFA**. Les 40 % proviennent du règlement fourni, non d'un panier inspecté. Les 70 % après résultats ne sont pas cumulés ni anticipés. [R, art. 9 ; D01, D02]

Vercel et qualité

Poste	Hypothèse budgétaire	Réserve
Vercel Hobby	0 USD seulement si conditions d'usage et de dépôt respectées	Pas une gratuité validée pour l'équipe ou le concours.
Vercel Pro, 1 siège déployeur	20 USD/mois ; 40 USD pour 2 mois	Un siège est inclus ; compatibilité des contributions Git à vérifier.
Vercel Pro, 2 sièges déployeurs	40 USD/mois ; 80 USD pour 2 mois	Ce n'est pas une exigence automatique : dépend du mode de collaboration autorisé.
Qodana Ultimate existant	Aucun nouvel achat budgété	Déclaration utilisateur ; vérifier que le projet et les contributeurs sont couverts.
GitHub Actions	Selon forfait et consommation	Ne pas présumer les minutes privées illimitées.

Hobby est réservé à l'usage personnel non commercial ; une organisation GitHub privée ne peut pas y être déployée via l'intégration documentée. Pro facture un forfait de 20 USD et les sièges déployeurs supplémentaires à 20 USD, hors taxes et consommations additionnelles. Ne pas contourner ces règles en publiant le dépôt ou en changeant l'auteur des commits. [D04, D05, D08]

Ce qui reste à chiffrer

Domaine, envois SMTP/API, taxes, adresses/options facturées, dépassements, éventuel environnement de recette temporaire, travail d'administration et marge de sécurité. Le service de base de données managé, un deuxième VPS LAB, une préproduction permanente et un nouveau chatbot ne figurent pas dans la cible initiale.

Enveloppe de base pour 2 mois : 55 625,60 FCFA Systalink, **plus** le forfait Vercel admissible et les postes ci-dessus. Les USD restent séparés des FCFA tant qu'aucun taux/frais de paiement n'est confirmé. Une provision de 15 % est un choix budgétaire, pas de la TVA calculée.

Aucune garantie de charge : avant l'ouverture, vérifier que cette petite instance assure le parcours principal pendant un run. Les restrictions du lab ne sont pas supprimées pour faire rentrer le produit dans une offre moins chère.

Les dates, droits, capacités et accords budgétaires restent à suivre en section 40.

Effet de l'alignement au mail

Aucun changement automatique de l'offre d'hébergement n'est décidé. Projets et annuaire restent dans le même monolithe. Les montants ci-dessus sont des hypothèses issues des relevés antérieurs, non vérifiés à nouveau dans cette révision de contenu ; valider panier, taxes, forfait Vercel, mail et sauvegardes avant achat. Les ajouts ont un coût de développement à réestimer, même sans nouvelle machine.

37 / EXÉCUTION ET SOURCES

Pilote et indicateurs honnêtes

L'adoption se mesure sur des usages observés, même modestes, pas sur des personnages de démonstration.

Phase 1 — comprendre avant d'ajouter

Interroger cinq développeurs : dernier problème rencontré, informations partagées, aide obtenue, temps perdu et conditions de réutilisation. Demander des exemples anonymisés ; aucun dépôt privé ni secret nécessaire. Les notes distinguent citation, observation et interprétation de l'équipe.

Phase 2 — tester le parcours

Cible proposée : dix testeurs externes au maximum pour garder un suivi qualitatif. Chacun reçoit une tâche : publier un blocage, proposer une aide ou reprendre une brique. Les tests ne doivent pas tous être réalisés par des amis guidés à chaque clic.

Mesurer le temps jusqu'à la première action utile, les points d'abandon et les erreurs de compréhension des badges. La satisfaction seule ne suffit pas ; observer si la tâche est accomplie.

Dictionnaire d'indicateurs

Indicateur	Définition opérationnelle
Demande exploitable	Champs minimaux compris et contexte jugé suffisant par un contributeur.
Résolution réelle	Acceptation par un demandeur non fictif avec note de vérification.
Réutilisation déclarée	Retour renseigné par un autre utilisateur, avec version et environnement.
Test de laboratoire réussi	Run complet avec assertions positives et digests correspondant à la version.
Délai de première aide	Durée entre publication et première proposition structurée ; cas sans réponse conservés.
Conversion en capsule	Nombre de résolutions devenues versions publiées, sans mélanger contenus éditoriaux.

Présentation au jury

Afficher les effectifs bruts : « 4 participants sur 5 », pas « 80 % des développeurs africains ». Indiquer dates, mode de recrutement et limites. Un retour négatif peut être utile s'il explique une amélioration effectivement livrée.

Les inscriptions internes, scénarios de recette, automatisations et comptes de démonstration sont exclus des chiffres d'adoption. Le tableau de bord ne possède pas de bouton permettant d'augmenter ces indicateurs manuellement.

Critère de poursuite : au moins quelques utilisateurs comprennent le parcours et peuvent reprendre une solution. Si ce n'est pas observé, revoir le contenu et l'ergonomie avant d'ajouter une nouvelle technologie.

Test complémentaire de l'atelier

Faire proposer ou reproduire un cas par un développeur extérieur consentant. Observer s'il apporte un point utile, si une amélioration peut être reliée à ce cas et si un autre développeur comprend la fiche sans guidage. « Au moins un cas extérieur utile » est une cible qualitative, pas une adoption déjà acquise. Les démonstrations préparées ne remplissent pas ce critère.

Tester d'abord la communauté

Ajouter au protocole : découvrir un projet sans compte, trouver un membre par technologie, publier une question sans code et identifier une demande ouverte où contribuer. Mesurer la compréhension des libellés et le temps jusqu'à une contribution utile, pas seulement la satisfaction sur le laboratoire. Une disponibilité déclarée n'est pas une promesse d'expert présent. Voir AC67/AC68.

38 / EXÉCUTION ET SOURCES

Démonstration et présentation

Fil narratif proposé — quatre à cinq minutes

Séquence	Ce que l'on montre
Le thème	Se retrouver, échanger, apprendre, présenter et construire ensemble [M].
La découverte	Un projet fictif visible et un développeur volontaire dans l'annuaire.
L'échange	Une demande rattachée, une réponse et un apport identifiable ; aucune messagerie privée nécessaire.
La différence	Pour B1 seulement, un cas documentaire et une comparaison de deux implémentations approuvées.
Le partage	Une capsule liée à la version, aux contributions et aux limites, reprise par un troisième personnage.
La maîtrise	Laravel/React, contrôles GitHub Actions/Qodana et architecture Systalink/Vercel.

Les exemples sont des données de démonstration tant qu'aucun pilote réel n'existe. Les versions peuvent être préparées ; ne pas annoncer de correction écrite en direct. Le laboratoire recalcule ses observations et affiche les incidents, il n'invente aucun succès. La durée proposée n'est pas une exigence du mail.

Pitch

HAAS est une communauté de développeurs où présenter ses projets, trouver de l'aide et construire ensemble. On y découvre des profils, échange sur des questions et partage des solutions. Sa différence : pour certaines solutions, les contributions deviennent des améliorations documentées que l'on peut mettre à l'épreuve et réutiliser.

Trois preuves à obtenir

Une découverte qui mène à un échange utile ; une contribution extérieure documentée avec accord de présentation ; une solution comprise et reprise par une autre personne. Le laboratoire est une preuve technique complémentaire, pas un substitut à l'utilité communautaire.

Présentation simple : neuf diapositives, notes orales et schémas de principe. Aucun résultat de recette ou déploiement n'est affiché comme acquis dans le dossier de conception.

La rencontre par le coup de main dans la démonstration

Un projet sans fil est ouvert volontairement ; une offre limitée mène à un accord, puis à l'échange public consenti. L'apport effectif vient ensuite. Montrer ces événements séparément et ne jamais compter une offre acceptée comme une aide terminée. Le pilote RH01 vérifie cette hypothèse auprès de personnes volontaires.

39 / EXÉCUTION ET SOURCES

Réception et traçabilité

Une version n'est recevable en interne que si le produit, les tests et les preuves concordent.

Matrice consolidée

Les exigences sont conservées ; les références ci-dessous utilisent les identifiants actifs des 122 lots, au lieu des regroupements L de la planification initiale.

Exigence	Lots actifs principaux	Recette
F01 / F11 · identité et profils	B05-B10, B32 ; F06-F08, F17	AC01-AC04, AC26.
F02 · demandes	B11-B16 ; F10-F11	AC05-AC07.
F03 / F04 · collaboration	B17-B21 ; F12-F13	AC08-AC11.
F05 / F06 · capsules	B22-B27 ; F14-F15	AC12-AC15.
F07 / F12 · réutilisation et recherche	B15, B26, B28 ; F10, F18	AC14-AC15, AC27.
F08 · laboratoire / B2	B33-B38 ; F16, F21	AC16-AC24.
F09 / F10 · modération, notifications	B29-B32 ; F19-F20	AC08, AC25-AC26.
F13 · cas de vérification	BV201-203 ; FV201	AC33-AC37, AC45/46/51.
F14 · comparaison	BV204-208 ; FV202-203	AC38-AC44, AC48/49/52.

Exigence	Lots actifs principaux	Recette
F15 · fiche de vérification	BV209 ; FV204	AC45-AC47 ; AC47 conditionnel.
N01-N08 / parcours complet	B39-B44 ; F22-F26 ; R01-R06, RV201	AC27-AC32, AC50 ; DEP-AC01-14.

Conditions de réception

Toutes les fonctions P0 déclarées sont accessibles et testées. Aucun incident de sécurité critique ou élevé confirmé ne reste ouvert. Les limites acceptées sont écrites et les rapports identifient le commit livré.

Bloquant : parcours impossible, accès indu, faux rapport, secret exposé ou non-recevabilité connue. **Majeur** : P0 fortement dégradé sans solution acceptable. **Mineur** : défaut de finition sans perte d'accès ni risque métier.

Dossier et validation croisée

Conserver tag/SHA, URL SPA et API, manifeste et empreintes, comptes d'évaluation transmis en privé, recette, rapports CI/Qodana réels, mesures, restauration, licences, déclaration IA et limites.

A relit le frontend et le parcours global ; B relit le backend, le lab et l'exploitation. Les validations sont humaines et ne remplacent pas les tests. Une nouvelle release réexécute les contrôles affectés ; les anciennes preuves sont conservées.

Une réception ancienne ne valide pas les nouveaux contrats. Backend, frontend et déploiement ont leurs gates ; GO_FRONTEND et GO_PRODUCTION ne sont jamais déduits d'un document rempli par un agent. Le logiciel, la vidéo et le dossier décrivent le même périmètre ; une maquette n'est pas une preuve d'implémentation.

Compléments de réception

F16/F17 : BC01-BC08, FC01-FC04, AC53-AC68 et UX18-20. F02 ask_question : BC07, FC04, AC64/AC68. Valider aussi la navigation revue UX01/04/05/11. Une réception antérieure au mail ne couvre pas les nouveaux contrats. Aucun « tout est terminé » si seul le laboratoire fonctionne.

40 / EXÉCUTION ET SOURCES

Transmission et décisions à clôturer

Le dossier de sortie doit permettre d'évaluer puis de reprendre le projet, sans remettre des secrets ou des droits inexistantes.

Livrables attendus du projet développé

Code source versionné ; URL Datacloud ; scripts d'installation et migration ; .env.example ; architecture ; contrat OpenAPI ; deux kits effectivement retenus au périmètre ; tests ; rapports ; notices tierces ; registre IA ; manuel utilisateur ; runbook de déploiement, sauvegarde, restauration et rollback ; vidéo et notes de version.

Ce cahier des charges décrit ces livrables futurs. Il ne prétend pas fournir aujourd’hui le code ou l’infrastructure de HAAS.

Registres à tenir

Licences : composant, version, origine, licence, usage runtime/dev/service, preuve, décision et réviseur. **IA** : outil, date, tâche, fichiers, relecture et tests. **Droits** : auteur, pièce justificative, périmètre autorisé et limitations. **Décisions** : problème, options, choix, date et conséquences.

Les six arbitrages ouverts

Code	Décision / responsable proposé
ARB01	Confirmer dates, sujet complémentaire et modalités de dépôt — représentant.
ARB02	Demander le modèle de cession et clarifier ses effets — deux membres.
ARB03	Confirmer le périmètre des licences et outils admissibles — A, relu par B.
ARB04	Vérifier le projet Ultimate déclaré existant et les accès GitHub/Vercel — A.
ARB05	Valider diffusion des kits, contribution externe et accès au dépôt — deux membres.
ARB06	Confirmer temps disponible, budget, rôles A/B et clé du prix — deux membres.

En cas de victoire

Préparer un inventaire des éléments cessibles, distinguer les droits tiers et traiter les accès par un canal sécurisé. Ne pas envoyer de clé SSH privée, mot de passe maître ou secret personnel dans le dépôt. Le transfert suit l’acte effectivement signé, pas une supposition tirée du seul nom du concours. [R, art. 8]

Glossaire utile

Capsule : fiche de résolution versionnée. **Brique** : module de code réutilisable. **Laboratoire** : exécution contrôlée d’un scénario autorisé. **Digest** : empreinte du contenu testé. **Idempotence** : répétition d’une opération sans effet supplémentaire indésirable dans le périmètre défini. **P0** : indispensable. **P1** : extension non engagée. **Recette** : vérification formelle de conformité au périmètre retenu.

Contenu de sortie

Le ZIP contient le cahier consolidé, le brief maître, les 122 lots, les dix skills HAAS, les gates, le système de design, les logos/favicons et la présentation. Il ne contient pas d’application prétendument installée. Les registres de suivi conservent des états non exécutés.

ARB07 : réestimer le périmètre atelier à deux et les tâches déjà accomplies ; ARB08 : valider les profils et manifestations de compatibilité avant le premier déploiement du comparateur. Les questions de calendrier, cession, licences, offre et distribution restent ouvertes jusqu’à preuve.

Alignement communautaire

Le mail et son interprétation sont conservés dans MAIL_CADRAGE_CADEV.md et ALIGNEMENT_MAIL.md. Ajouter COMMUNAUTE_ET_PROJETS.md, migrations/API/écrans projets/membres et critères AC53-AC68 aux livrables futurs du code. Le destinataire du mail et les liens de suivi ne sont pas nécessaires au dossier public. Les choix HAAS ne sont pas attribués à l’organisateur.

41 / EXÉCUTION ET SOURCES

Sources et portée du dossier

Les règles fournies, les documentations vérifiées et les choix proposés restent distingués.

Sources fournies par l’équipe

[R] Conditions générales de participation CADEV, texte partagé dans la conversation : articles 3-10 et annexe 1 principalement. Aucune date de version du règlement n’est fournie. Les formulations atypiques du texte ne sont pas corrigées pour en déduire une autre règle.

[C] Capture du tableau de bord Systalink Challenge : thème, équipe Delta, format de deux à cinq membres et période affichée du 29 septembre au 25 octobre 2026. Elle diverge de l’article 3 ; cette divergence reste ouverte.

[U] Décisions de l’équipe : deux développeurs, Laravel et React, usage souhaité de Qodana et GitHub Actions, nom de concours HAAS. Les autres choix de ce dossier sont des propositions, sauf mention contraire.

Documentations officielles consultées le 30 septembre 2026

Repère	Référence	Usage dans ce dossier
S01	Laravel 13 — Release Notes	Compatibilité minimale PHP et choix de base.
S02	Laravel Sanctum	Authentification SPA par cookies et CSRF.
S03	Qodana — PHP	Technologies prises en charge et niveaux de licence.
S04	Qodana — JavaScript/TypeScript	Offre du linter JavaScript/TypeScript.
S05	Qodana — GitHub Actions	Intégration à la chaîne CI.
S06	Qodana — Quality gate	Mécanismes de seuil de qualité.
S07	GitHub — Secure use reference	Permissions et fixation des actions.
S08	MDN — Offline and background operation	Mécanismes hors connexion et limites de contexte.
S09	W3C — WCAG 2.2	Objectifs d’accessibilité de la recette.

Repère	Référence	Usage dans ce dossier
S10	Systalink — offre publique	Existence des familles de services, pas un devis Datacloud.

Ces références éclairent les choix techniques ; elles ne prouvent ni la disponibilité d'un compte, ni un tarif, ni la conformité complète d'une implémentation future. Le modèle de cession, le sujet complémentaire et les conditions de diffusion des kits n'ont pas été fournis.

Références antérieures de l'atelier

Les sources [S01-S10] ci-dessus appartiennent au dossier de référence du 30 septembre. Le 1er octobre 2026, les références suivantes ont été relues pour les usages limités indiqués ; elles ne prouvent ni un abonnement ni une application fonctionnelle.

Repère	Source	Usage limité
T01/T02	OpenAI : skills et AGENTS.md	Organisation du pack et instructions d'agent.
T03	OpenAI : Codex Cloud	Ne pas nier les capacités de développement/test d'un agent.
T04	W3C : WCAG 2.2	Objectifs de lisibilité et accessibilité.
T05	Laravel 13 : releases	Cible proposée conservée ; compatibilité à verrouiller.
T06	JetBrains : Qodana PHP	Ultimate déclaré disponible ; activation sur le projet à vérifier.

Les URL complètes et dates sont dans docs/sources/REFERENCES_V2.md. Le catalogue de skills externes est historique et doit être audité avant installation. Le document SANI-TRACE/SIC visible dans le projet est hors sujet HAAS et n'est pas utilisé.

Références antérieures du déploiement

Les consultations suivantes du 1er octobre 2026 documentent seulement les points actualisés, pas un compte utilisateur. Les choix propres à HAAS restent des prescriptions de conception.

- **[D01] Systalink — Serveur cloud** — <https://systalink.com/fr/server-cloud> — Configuration et prix Business ; gestion non automatiquement comprise.
- **[D02] Systalink — Stockage d'objets** — <https://systalink.com/fr/stockage-objets> — Stockage distant : référence et tarif Starter.
- **[D03] Laravel 13 — Sanctum** — <https://laravel.com/docs/13.x/sanctum> — Cookies, domaine parent, CSRF, client et CORS.
- **[D04] Vercel — Git deployments** — <https://vercel.com/docs/git> — Dépôts privés, membres et déclenchement de production.

- **[D05] Vercel — Hobby** — <https://vercel.com/docs/plans/hobby> — Usage personnel non commercial ; gratuité conditionnelle.
- **[D06] Vercel — Vite** — <https://vercel.com/docs/frameworks/frontend/vite> — Build Vite et réécriture pour SPA.
- **[D07] Laravel 13 — Deployment** — <https://laravel.com/docs/13.x/deployment> — Web root, configuration, processus et livraison.
- **[D08] Vercel — Pro** — <https://vercel.com/docs/plans/pro-plan> — Tarifs USD, sièges et taxes non incluses.
- **[D09] GitHub — Secure use** — <https://docs.github.com/en/actions/reference/security/secure-use> — Secrets, permissions et actions vérifiées.
- **[D10] JetBrains — Qodana PHP** — <https://www.jetbrains.com/help/qodana/php.html> — Linter PHP/JS/TS, Laravel/React, qualité.
- **[D11] JetBrains — Qodana editions** — <https://www.jetbrains.com/help/qodana/pricing.html> — Ultimate et Ultimate Plus distincts.
- **[D12] OpenAI — AGENTS.md** — <https://developers.openai.com/codex/guides/agents-md> — Instructions locales et hiérarchie ; syntaxe vérifiée.
- **[D13] W3C — Contrast minimum** — <https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html> — Contraste texte, objectif de recette ; pas certificat HAAS.

[R] Règlement CADEV copié par l'utilisateur : art. 3-10, annexe 1. [C] Captures du concours. [U] Décision backend Systalink / frontend Vercel ; Ultimate déclaré disponible. [D] Choix d'implémentation explicités dans ADR-004. Les anciennes références du design restent un catalogue historique à revalider avant installation.

Source de cadrage ajoutée [M]

Mail « Félicitations, votre équipe est formée », expéditeur affiché CADEV by Systalink, date affichée 30 septembre 18:11. Texte fourni par l'utilisateur, non recherché dans sa messagerie ; extrait utile conservé dans docs/sources/MAIL_CADRAGE_CADEV.md et en section 53. Il fonde les usages, pas les paramètres de tables, délais, champs ou modules choisis. [H] désigne les prescriptions de conception, notamment F16/F17 et leurs tests.

Cette révision est une consolidation des sources fournies : les références techniques et tarifs antérieurs sont conservés avec leur portée datée, sans affirmation de nouvelle vérification externe. Le règlement reste la source des critères, prix/cession et dates non tranchées par le mail.

42 / ATELIER DISTINCTIF

Une communauté, un atelier distinctif

L'ordre de présentation

La communauté répond au thème : découvrir des personnes/projets, échanger et apprendre. L'atelier enrichit certains échanges : proposer un cas, améliorer la solution, comparer et conserver une vérification. Commencer par le besoin d'un développeur, pas par une liste de runtimes ou de tests.

HAAS n'oblige pas à choisir entre son assistant et une communauté. Le produit ne prétend pas inventer le code, les tests ou les réponses vérifiées. L'hypothèse à valider est la facilité à trouver une contribution extérieure pertinente et à comprendre ce qui en résulte.

Deux parcours également légitimes

Question de connaissance → discussion utile → apprentissage. Projet → demande ciblée → contribution → capsule et, si applicable, comparaison contrôlée. Une question sans résolution machine n'est pas un échec produit. Un comparateur sans contributeurs ne prouve pas à lui seul l'utilité communautaire.

Vocabulaire qui évite la confusion

Une capsule est une fiche versionnée ; une brique est du code réutilisable facultatif ; un cas est un document ; un scénario est un test approuvé ; une comparaison contient deux observations. Une revue humaine, une acceptation et un test ne sont pas interchangeables. Les écrans gardent ces distinctions sans imposer des termes techniques à l'accueil.

43 / ATELIER DISTINCTIF

F13 · Proposer un cas concret

Contribuer ne demande pas forcément de connaître déjà le correctif.

Accès. Membre actif et vérifié, sur une demande accessible ou une version de capsule accessible. Exactement un parent : `help_request_id XOR capsule_version_id`. Le serveur fixe l'auteur et reconstruit la visibilité à la lecture, à la soumission et après retrait. Une version n'est jamais remplacée silencieusement par « latest ».

Données. Titre 15-140 caractères ; contexte 20-2 000 ; étapes 20-4 000 ; résultat attendu 10-2 000 ; résultat observé facultatif, 4 000 maximum ; environnement 500 maximum. Même politique de Markdown/code inerte, de secrets et de liens non récupérés que F02. Un cas peut proposer des étapes sans avoir été exécuté : le libellé doit le dire.

Cycle. draft → submitted → reviewed ou declined ; reviewed → integrated uniquement par un manifeste de release approuvé ; retrait motivé possible. Un auteur corrige un brouillon. Modifier un cas soumis crée une nouvelle révision et le ramène à draft ; la revue précédente ne valide pas les nouveaux mots. Une révision intégrée reste figée ; proposer une révision suivante, ne pas écraser la source d'une preuve.

Revue. Modérateur/admin habilité distinct de l'auteur : motif obligatoire pour refus ou retrait. reviewed signifie « examiné », pas « test réussi ». Un cas n'est marqué integrated que si la release associe case_revision_id, scenario_key, suite_digest et commit dans un manifeste réellement relu/déployé. Pas de bouton permettant au modérateur de saisir un shell ou de transformer le texte en code.

Attribution. L'auteur du cas, la personne qui reproduit, celle qui corrige et celle qui documente peuvent être distincts. Aucun score fabriqué, aucune équivalence entre nombre de cas et niveau de compétence. Une intégration peut être éditoriale ; sa provenance est affichée et ne devient pas une adoption externe.

44 / ATELIER DISTINCTIF

F14 · Comparer sans tricher

Deux observations réelles, une même situation de départ et des limites visibles.

Registre. Un comparison_profile déclaré dans la release identifie la famille B1, le manifeste de référence pédagogique, le manifeste candidat rattaché à une version, la suite de scénarios et un schéma d'entrée fixe. La référence incorrecte porte « Exemple pédagogique incorrect », n'est pas proposée en téléchargement et n'est pas publiée comme une capsule recommandée. Les deux implémentations sont approuvées pour l'exécution, pas déclarées toutes deux correctes.

Entrées API. Seulement comparison_profile_id et scenario_key d'une liste autorisée, avec Idempotency-Key. Pas de code, URL, commande, chemin, nom de classe, scénario dynamique ou paramètre libre. Le serveur fige les versions, code digests, suite digest, environnement et entrées canoniques au lancement. Dans ce P0, le jury choisit le scénario, pas le programme ni un montant réel.

Comparabilité. Les deux côtés utilisent la même charge utile normalisée, le même seed, le même oracle d'attendu, les mêmes identifiants d'assertion et la même configuration d'exécution. Les namespaces de fixtures restent distincts. Une différence de profil/suite/environnement incompatible empêche la comparaison (409), pas une conclusion favorable. Ne pas présenter cette comparaison comme un benchmark de performance.

Exécution réelle. Deux enfants lab_run sont créés avec rôles baseline/candidate et digests figés. Le worker exécute chaque côté à partir de fixtures neuves. Les résultats ne sont ni copiés d'une exécution antérieure ni remplis à partir des attendus. Les étapes préparées et versions déjà déployées sont annoncées comme telles ; aucune prétention à écrire un correctif en direct.

Quotas. L'enveloppe retenue reste cinq unités d'exécution/heure/membre. Une comparaison réserve atomiquement deux unités dès acceptation ; un run simple en réserve une. Un jeu de la même clé ne consomme pas deux fois. Une opération active par membre ; les deux enfants sont séquentiels. Un seul run réellement actif au maximum globalement. Les places et le quota sont partagés entre appels simples et comparaisons : aucune seconde voie ne les contourne. File d'attente bornée ; 429 et Retry-After documentés. Aucun débit réservé n'est remboursé automatiquement après un résultat ambigu.

Délais. Les bornes de 15 s cible et 20 s maximum par enfant sont conservées. Budget global d'une comparaison démarrée : 50 s proposé, sans compter le temps en file. Au-delà : `timed_out` et conclusion inconclusive. Le suivi navigateur est asynchrone ; le timeout HTTP ne tue pas ni ne relance aveuglément l'opération.

Pannes. Claim atomique, `lease/execution_token` et finalisation unique. Une reprise détecte les enfants terminés ; elle ne crée pas une seconde paire et ne mélange pas une autre release. Si le worker disparaît, le réconciliateur attribue `error/timed_out` ; aucune réussite par défaut. Une indisponibilité du lab ne bloque ni les demandes ni la lecture documentaire.

45 / ATELIER DISTINCTIF

Données et API de l'atelier

Le cas humain, le scénario approuvé et le résultat machine restent des objets distincts.

Préfixe `/api/v1` ; erreurs HAAS et UUID ; droits recontrôlés à chaque appel. Le contrat sera développé et testé par Codex, les chemins suivants sont une cible, pas une API existante.

Route	Responsabilité
GET/POST <code>/verification-cases</code>	Liste filtrée par visibilité / créer son brouillon
GET/PATCH <code>/verification-cases/{id}</code>	Lire / réviser ses données autorisées avec <code>lock_version</code>
POST <code>/verification-cases/{id}/submit</code>	Figier une révision soumise
POST <code>/admin/verification-cases/{id}/review</code>	Revue motivée, acteur distinct
POST <code>/verification-cases/{id}/withdraw</code>	Retrait autorisé, historique conservé/expurgé si nécessaire
GET <code>/versions/{id}/comparison-profiles</code>	Profils compatibles et approuvés uniquement
POST <code>/comparisons</code>	Créer une paire contrôlée ; 202 + suivi
GET <code>/comparisons/{id}</code>	État et rapport autorisés, pas de mutation publique
GET <code>/versions/{id}/verification-summary</code>	Fiche versionnée, contributions et limites

Le contexte IA P1 est assemblé côté React à partir des réponses déjà autorisées, sans endpoint d'export massif.

Les enfants réutilisent `lab_runs/lab_results`. Les contraintes de parent exclusif, côtés autorisés, unicité des révisions et clés étrangères sont testées. Manifestes, seeds et versions sont figés ; aucun chemin de code ne vient d'une donnée utilisateur.

Contrat de persistance complémentaire

Table	Champs structurants et contraintes
verification_cases	id, author_id, help_request_id ou capsule_version_id, state, current_revision, lock_version. CHECK un seul parent.
verification_case_revisions	id, case_id, revision_no, body structuré, digest, submitted_at ; unicité case/révision.
case_scenario_links	case_revision_id, scenario_key, suite_digest, release_commit, reviewer_id, integrated_at.
comparison_profiles	id, family_key, baseline_manifest, candidate_version_id, suite_digest, input_schema_version, enabled. Registre lié à la release.
comparison_runs	id, profile_id, actor_id, request_key, state/outcome, input_digest, seed, env_snapshot, started_at, finished_at.
lab_runs (extension)	comparison_id nullable et comparison_side ; unique comparaison/côté ; profils et codes figés.

Les champs exacts, nullabilités et clés seront fixés dans les migrations/OpenAPI et testés. Aucune ligne administrable ne devient un chemin exécutable. Ne pas mélanger l'idempotence du produit, la clé B1 et le lien d'un cas documentaire.

Exemple fictif · Idempotency-Key requis · réponse 202/queued

```
{
  "comparison_profile_id": "uuid-du-profil-approuve",
  "scenario_key": "duplicate_event"
}
```

46 / ATELIER DISTINCTIF

F15 · Une preuve compréhensible

Montrer la version, les observations, les personnes et les limites au même endroit.

États et conclusions

ComparisonState : queued, running, completed, error, timed_out. ComparisonOutcome : improved, unchanged, regressed, mixed, inconclusive.

Une comparaison completed peut conclure à une régression. Un HTTP 200 ne signifie pas que la solution est correcte. Les données machine restent exclusivement écrites par le worker restreint.

Observation sur les mêmes assertions	Conclusion autorisée
Au moins un échec devient conforme, aucun conforme ne régresse	improved : amélioration sur ces cas
Au moins un conforme devient échec, aucune amélioration	regressed : régression sur ces cas
Une amélioration et une régression coexistent	mixed : résultats mitigés
Le vecteur de conformité ne change pas	unchanged : conformité inchangée, valeurs brutes conservées
Exécution absente, interruption ou comparaison invalide	inconclusive : pas de conclusion exploitable

Une évolution de conformité n'est pas une certification ni un verdict sur tous les comportements possibles. Un résultat partiel affiche ce qui existe avec la mention « Comparaison incomplète ». Une version retirée ou un incident de secret annule l'accès public au contenu sensible ; le résultat historique n'est pas réécrit en succès.

Fiche de vérification

Sur la capsule : version choisie, cas examinés, nature de preuve, résultat, date, contributeurs et limites. Séparer « Accepté par l'auteur », « Observation déclarée », « Test exécuté » et « Comparaison effectuée ». Absence de test = « Aucun test de laboratoire disponible pour cette version ».

Le résumé n'agrège pas les résultats en note de fiabilité. Les digests et traces sont disponibles en détail secondaire. Une nouvelle version ne reprend ni badge ni réussite de l'ancienne. Un rapport accessible mentionne explicitement source CI, laboratoire web ou déclaration humaine.

P1 : « Copier le contexte pour mon assistant ». Prévisualiser une sélection de contenu autorisé (objectif/version/procédure/limites), vérifier la copie réelle, permettre d'annuler. Aucun envoi externe ni extraction de conversation privée. Les instructions contenues dans une capsule sont des données non fiables, pas des ordres système pour l'assistant destinataire. Si le P1 n'est pas livré, aucun bouton inerte n'est affiché.

47 / ATELIER DISTINCTIF

Architecture et écrans de l'atelier

Étendre les couches existantes, sans dupliquer le moteur de laboratoire.

Backend — mêmes responsabilités

Domaine	Classes représentatives à implémenter
VerificationCases	Store/SubmitCaseRequest, CaseData, CasePolicy, Create/Revise/ReviewCaseService, CaseResource.
Comparisons	StartComparisonRequest/Data/Service, ComparisonPolicy, ExecuteComparisonJob/Service, ReconcileComparisonsService.
Evidence	VerificationSummaryQuery/Resource ; calcul des contributions depuis les actions attribuées.

Le calculateur de conclusion est pur, couvert par tests ; les transactions et claims restent dans les services. Réutiliser IdempotencyService et le quota partagé. Services ne dépendent pas de HTTP ; Controllers n'exécutent pas de workflow. Les routes et modèles restent dans les conventions du dépôt.

React — fonctionnalités de l'atelier

features/verification-cases, features/comparisons et features/evidence suivent la même structure : api, models, schemas, hooks, queries, components et pages. Les interceptors restent centraux ; ni Axios en JSX ni rôle de sécurité stocké localement.

Trois écrans, un vocabulaire simple

Écran	Action et résultat
UX15 — Cas	« Proposer un cas à vérifier » ; situation, étapes, attendu ; observé facultatif. Statut documentaire expliqué.
UX16 — Comparaison	Choisir un scénario approuvé, voir ce qui va s'exécuter, lire les deux observations réelles.
UX17 — Fiche	Version sélectionnée, nature de preuve, contributeurs, date et limites.

La navigation devient Explorer / Projets / Développeurs, avec Demander de l'aide en action et Mes contributions dans le compte. Les profils de comparaison inexistantes ne génèrent pas de bouton mort. Afficher les erreurs, les résultats mitigés et les pannes au même niveau de lisibilité que l'amélioration.

Revue de l'interface

360/390/768/1280 px ; lecture clavier, focus, zoom, code long, erreurs de presse-papiers si P1. La table devient des cartes empilées. Pas de curseur avant/après masquant une moitié, de pourcentage de progression fictif ou d'animation obligatoire. Les captures du PowerPoint sont des compositions de principe, pas des écrans déjà développés.

Extensions communautaires

Projects ajoute les couches serveur adaptées et features/projects ; Identity conserve les préférences et features/developers porte la découverte. UX18 : catalogue/détail ; UX19 : édition de projet ; UX20 : annuaire et préférences. Le formulaire existant UX05 porte ask_question. Aucun module de messagerie ni autre client HTTP n'est ajouté. Voir section 56.

48 / ATELIER DISTINCTIF

Recette complémentaire AC33-AC52

Les nouveaux contrôles prolongent les 32 cas existants ; aucun résultat n'est acquis.

ID	Attendu atelier	Portée	État / preuve
AC33	L'intention unblock/review_solution/reproduce_behavior valide est conservée ; champ protégé rejeté ; anciennes demandes compatibles.	Backend + formulaire	NON EXÉCUTÉ — aucune preuve
AC34	Un cas référence exactement une demande ou une version accessible ; double parent, absence et parent privé refusés.	Backend	NON EXÉCUTÉ — aucune preuve
AC35	Réviser un cas soumis crée une révision ; la source intégrée reste figée et sa preuve conserve son digest.	Backend + interface	NON EXÉCUTÉ — aucune preuve
AC36	Un cas humain revu ne devient jamais un programme exécutable ; champs code/command/URL de lancement rejetés.	Backend + worker	NON EXÉCUTÉ — aucune preuve
AC37	Revue par personne distincte ; refus/retrait motivés ; intégration liée à un manifeste de release réel.	Backend + administration	NON EXÉCUTÉ — aucune preuve
AC38	Seuls les profils B1 approuvés et compatibles peuvent être comparés ; paire arbitraire ou version retirée refusée.	Backend	NON EXÉCUTÉ — aucune preuve
AC39	Les deux runs partagent entrées/seed/oracle/suite/ environnement comparables et des fixtures distinctes.	Intégration PostgreSQL	NON EXÉCUTÉ — aucune preuve
AC40	Deux runs réels produisent les observations ; le module naïf échoue le cas doublon, le corrigé le satisfait.	Worker + API	NON EXÉCUTÉ — aucune preuve
AC41	Une comparaison réserve deux unités ; même clé ne recharge pas ; concurrence globale/membre partagée avec les runs simples.	Concurrence PostgreSQL	NON EXÉCUTÉ — aucune preuve
AC42	Même clé/payload rejoue la même opération autorisée ; payload différent retourne 409 ; nouveau clic volontaire a un nouvel identifiant.	Backend + frontend	NON EXÉCUTÉ — aucune preuve
AC43	Crash entre enfants, lease expirée et délai dépassé donnent un état explicite sans faux résultat complet ni double finalisation.	Worker/ exploitation	NON EXÉCUTÉ — aucune preuve

ID	Attendu atelier	Portée	État / preuve
AC44	Les cinq conclusions improved/unchanged/regressed/mixed/inconclusive suivent les mêmes assertions ; régression jamais maquillée.	Unitaires + API	NON EXÉCUTÉ — aucune preuve
AC45	Retrait ou secret expurge les vues du cas, de la comparaison et de la fiche, y compris accès direct et cache.	Backend + navigateur	NON EXÉCUTÉ — aucune preuve
AC46	Diagnostic, cas, correction et revue sont attribués à leurs vrais acteurs ; données éditoriales/démo ne deviennent pas adoption externe.	Backend + interface	NON EXÉCUTÉ — aucune preuve
AC47	P1 seulement : aperçu du contexte avant copie ; aucun envoi externe, contenu privé non autorisé absent, échec du presse-papiers explicite.	Frontend ; non applicable si P1 absent	NON EXÉCUTÉ — aucune preuve
AC48	Comparer et lire le rapport à 360 px et au clavier fonctionne ; états nommés, contraste et focus contrôlés sans couleur seule.	Navigateur + revue manuelle	NON EXÉCUTÉ — aucune preuve
AC49	Un ancien rapport reste daté ; relancer produit une nouvelle opération ; polling arrêté au terminal et hors écran.	Frontend + API	NON EXÉCUTÉ — aucune preuve
AC50	Le parcours cas → revue → intégration approuvée → comparaison → fiche relie des IDs réels ; une préparation éditoriale est signalée.	API puis navigateur	NON EXÉCUTÉ — aucune preuve
AC51	Un lien de reproduction dans un cas n'est pas récupéré automatiquement ; aucune instruction de contenu ne modifie l'agent ni le runner.	Sécurité	NON EXÉCUTÉ — aucune preuve
AC52	Acceptations simultanées, deux comparaisons et un run simple ne dépassent ni contraintes de données ni limites atomiques.	Concurrence PostgreSQL	NON EXÉCUTÉ — aucune preuve

Enregistrement

Chaque preuve indique commit, environnement, commande ou procédure, attendu, observé, date et défaut. Le backend est testé avant le frontend ; l'interface se reçoit après intégration réelle. Un objectif pilote non atteint est signalé, pas transformé en test réussi.

49 / ATELIER DISTINCTIF

Les 122 lots de réalisation

Ordre proposé S2/B72/F40/R8 ; les 106 identifiants précédents sont conservés. Les nouveaux lots sont BH01–BH10, FH01–FH05 et RH01. Pas de durée ni de nombre de commits garanti ; chaque intention doit inclure ses tests. Le détail opérationnel est dans PLAN_COMMITS.md et tasks.json.

Ordre	ID	Lot
1	S01	Inventaire et environnement
2	S02	Cadre d'exécution
3	B01	Squelette Laravel
4	B02	Qualité PHP locale
5	B03	CI backend initiale
6	B04	Contrat HTTP et erreurs
7	B05	Identité et référentiels
8	B06	Inscription
9	B07	Sessions et CSRF
10	B08	Courriels de compte
11	B09	Autorisations et compte courant
12	B10	Profils et technologies
13	B11	Schéma de collaboration
14	B12	Audit et révisions
15	B13	Idempotence des commandes
16	B14	Créer une demande
17	B15	Lire et rechercher les demandes
18	B16	Modifier une demande
19	B17	Commentaires
20	B18	Propositions de solution
21	B19	Accepter une proposition
22	B20	Rouvrir une demande
23	B21	Archiver et non-retenir
24	B22	Schéma des capsules
25	B23	Brouillons de capsule
26	B24	Soumettre à la revue
27	B25	Publier une version immuable
28	B26	Catalogue de capsules
29	B27	Téléchargement contrôlé

Ordre	ID	Lot
30	B28	Favoris et réutilisation
31	B29	Notifications internes
32	B30	Signalements
33	B31	Retrait de contenus
34	B32	Suspensions et rôles
35	B33	Registre du laboratoire
36	B34	Lancement et quotas
37	B35	Brique B1
38	B36	Worker et rapports réels
39	B37	Pannes du laboratoire
40	B38	API de démonstration B2
41	BC01	Schéma des projets et découverte volontaire
42	BC02	Brouillons et édition de projet
43	BC03	Publication et catalogue des projets
44	BC04	Demandes liées à un projet
45	BC05	Modération des projets et visibilité liée
46	BC06	API de découverte des développeurs
47	BV201	Schéma des cas et intentions
48	BV202	Créer et soumettre un cas
49	BV203	Revue et intégration des cas
50	BV204	Profils de comparaison B1
51	BV205	Données et contrat des comparaisons
52	BV206	Lancement idempotent et quotas partagés
53	BV207	Exécuter les deux côtés réels
54	BV208	Conclusions et pannes de comparaison
55	BV209	Fiche de vérification et attributions
56	BC07	Questions sans formulaire de panne
57	BC08	Recette API de la communauté
58	BV210	Recette transversale de l'atelier

Ordre	ID	Lot
59	BH01	Schéma et ouverture aux coups de main
60	BH02	Occasions de contribuer et préférences explicables
61	BH03	Proposer un apport avec consentement et limites
62	BH04	Consulter ses offres sans fuite
63	BH05	Accepter et créer un seul échange public
64	BH06	Rattacher une offre à un échange choisi
65	BH07	Refuser retirer expirer et fermer proprement
66	BH08	Projections publiques progrès et notifications
67	BH09	Contrats et documentation des offres
68	BH10	Recette transactionnelle et privée des coups de main
69	B39	Revue de sécurité API
70	B40	Contrat API complet
71	B41	Recette backend complète
72	B42	Exploitation backend
73	B43	Qodana backend
74	B44	Point de validation backend
75	F01	Socle React
76	F02	Tokens de design
77	F03	Composants de formulaire
78	F04	Composants de lecture
79	F05	Client API et types
80	F06	Session et guards
81	F07	Navigation responsive
82	F08	Pages de compte
83	F09	Accueil utile
84	F10	Catalogue
85	F11	Formulaire de demande
86	F12	Discussion de demande
87	F13	Résolution guidée

Ordre	ID	Lot
88	F14	Éditeur de capsule
89	F15	Lecture et preuves de capsule
90	F16	Lancement et rapport du laboratoire
91	F17	Espace personnel et profils
92	F18	Favoris et retours
93	F19	Notifications
94	F20	Back-office
95	F21	Démonstrateur B2
96	FC01	Catalogue et fiche projet
97	FC02	Édition de projet et demande liée
98	FC03	Découverte et préférences des membres
99	FH01	Accueil et occasions concrètes d'aider
100	FH02	Ouvrir son projet volontairement
101	FH03	Proposition et aperçu de publication
102	FH04	Offres reçues envoyées et décision
103	FV201	Formulaire et suivi des cas
104	FV202	Choisir et lancer une comparaison
105	FV203	Lecture comparative accessible
106	FV204	Fiche de vérification et mémoire
107	FV205	Recette visuelle et bout en bout atelier
108	FC04	Parcours communautaire complet
109	FH05	Progrès visibles et recette navigateur
110	F22	Résilience frontend
111	F23	Audit d'accessibilité
112	F24	Finition visuelle
113	F25	Recette navigateur complète
114	F26	Point de validation frontend
115	R01	Inventaires et droits
116	R02	Artefact reproductible

Ordre	ID	Lot
117	R03	Recette et livraison Systalink / Vercel
118	R04	Restauration et retour stable
119	RV201	Preuve d'utilité collaborative
120	RH01	Observer les premières collaborations
121	R05	Pilote et démonstration
122	R06	Réception et transmission

Les douze sous-lots DEP précisent la livraison sans ajouter de serveur. Les gates backend, frontend et production restent humains.

50 / ATELIER DISTINCTIF

Utiliser le dossier et livrer

Partir du dépôt réel

Décompresser le ZIP hors du dépôt. Examiner README_CODEX, AGENTS.md, ADR-005, COMMUNAUTE_ET_PROJETS et le brief maître. Fusionner sans écraser code, historique, instructions ou statuts existants. Conserver les identifiants de tâche ; l'agent commence au prochain lot nécessaire à l'état réel.

Le dossier backend/frontend livré contient des instructions, pas une application déjà installée. Aucun achat, push, publication, déploiement ou partage de donnée réelle n'est autorisé par la seule réception du pack.

Gates

Gate	Exigence
Backend	F01-F18 serveur, B1, API B2, contraintes, OpenAPI, tests négatifs et revue du deuxième membre ; GO_FRONTEND explicite.
Frontend	UX01-UX23 reliés à l'API, parcours communautaire et atelier, erreurs, mobile et clavier.
Release	Artefact, configuration réelle Systalink/Vercel, DEP-AC01-14, restauration, droits et limites ; GO_PRODUCTION humain.

Contenu de cette livraison

Cahier consolidé PDF/HTML/Markdown ; PowerPoint simple avec notes ; instructions Codex ; 122 lots, 90 AC dont AC47 conditionnel, 23 familles UX, 10 skills actualisés et ressources graphiques. Les tests documentaires portent sur cohérence, liens, archive, JSON et palette ; ils ne valident pas l'application.

Le résultat attendu : une communauté compréhensible, un échange utile, des contributions visibles et des solutions que le suivant peut reprendre.

51 / DÉPLOIEMENT

Authentification · Vercel vers Systalink

Choix de conception : sessions de confiance, pas de bearer token persistant dans le navigateur. Configuration illustrative, à adapter et tester. [D03]

Domaines retenus (exemples)

- SPA : `https://app.haas.example.com` ; API : `https://api.haas.example.com`.
- Parent de cookies : `.haas.example.com` ; réservé aux hôtes de confiance.
- B2 : `https://demo.example.com` et `https://demo-api.example.com`, hors de ce parent.

La SPA et l'API sont cross-origin mais same-site. `SameSite=Lax` ne supprime ni les contrôles CSRF ni CORS. Ne jamais élargir le domaine de cookie à `.example.com` si B2 ou des hôtes non fiables y résident.

Variables backend (valeurs publiques d'exemple)

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.haas.example.com
FRONTEND_URL=https://app.haas.example.com
SESSION_DRIVER=database
SESSION_COOKIE=haas_session
SESSION_DOMAIN=.haas.example.com
SESSION_SECURE_COOKIE=true
SESSION_HTTP_ONLY=true
SESSION_SAME_SITE=lax
SANCTUM_STATEFUL_DOMAINS=app.haas.example.com
CORS_ALLOWED_ORIGINS=https://app.haas.example.com
```

`FRONTEND_URL` et `CORS_ALLOWED_ORIGINS` sont des conventions du projet : leur lecture doit être implémentée dans la configuration. Les variables seules ne configurent pas Laravel par magie. Ne pas inclure schéma/protocole dans `SANCTUM_STATEFUL_DOMAINS` ; les ports sont explicites en local. Activer le middleware stateful API de Sanctum dans la version Laravel retenue.

Réglages CORS à implémenter

`allowed_origins` contient uniquement la SPA approuvée ; `supports_credentials=true`. Pas de * avec credentials. Limiter les méthodes aux routes livrées. Autoriser les headers effectivement utilisés : Accept, Content-Type, X-Requested-With, X-XSRF-TOKEN, Idempotency-Key ; exposer Retry-After et X-Request-ID si utilisés pour la lecture frontend.

Les paths couverts comprennent `api/*`, `sanctum/csrf-cookie`, `login`, `logout`, `register`, `forgot-password`, `reset-password`, `email/*` selon les routes effectivement retenues. Tester les prévoils OPTIONS et la présence de CORS aussi sur les erreurs 401/403/419/422/429. CORS est un contrôle navigateur, pas une autorisation métier : Politiques, session, quotas et CSRF restent obligatoires.

Instance HTTP principale

```
// api-client.ts : exemple à compléter par la validation de l'environnement.
export const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
```

```
withCredentials: true,  
withXSRFToken: true,  
timeout: 15000,  
headers: { Accept: 'application/json' },  
});
```

VITE_API_URL=https://api.haas.example.com ; pas de secret. Les fonctions API appellent /api/v1/..., l'authentification /login, /logout, /register et /sanctum/csrf-cookie sur cette même origine API. Ne pas utiliser baseUrl='/' en production Vercel.

Parcours et erreurs

Initialiser CSRF sur l'API, envoyer la connexion, récupérer /api/v1/me. Le login régénère la session ; le logout l'invalidé et renouvelle le token CSRF. Le cookie de session reste HttpOnly ; XSRF-TOKEN doit rester lisible par le client pour le mécanisme documenté. Le HTTPS et les attributs des deux cookies doivent être vérifiés dans le navigateur réel. [D03]

Un 401 attendu à l'ouverture de /me n'est pas une panne. Un 419 mène à une récupération contrôlée, sans renvoyer automatiquement toutes les écritures. Un 503 réseau n'est pas une déconnexion. Annuler les lectures et vider les caches privés à la fin d'une session. Les données de formulaire peuvent rester en mémoire ; aucun secret n'est persisté par défaut.

Vérification du courriel et réinitialisation : construire les liens à partir des origines autorisées. Une URL signée Laravel destinée à l'API ne doit pas être modifiée pour changer son hôte ; l'écran React peut recevoir une redirection autorisée après validation. Interdire les paramètres de redirection externes libres. Le message de réinitialisation ne révèle pas l'existence d'un compte.

Local, CI et previews

Utiliser des hôtes locaux de confiance sous un même parent, ou un proxy local documenté. Ne pas mélanger localhost et 127.0.0.1 dans une session. Les tests des vrais cookies doivent activer le middleware CSRF, souvent neutralisé dans les tests applicatifs ordinaires ; conserver des tests navigateur dédiés.

Les previews Vercel ne pointent pas vers une API privée de production. Pour une recette distante authentifiée, créer temporairement un couple SPA/API de recette et des données/secrets dédiés sous un parent distinct, ou effectuer la recette en CI. Une preview uniquement visuelle est étiquetée comme telle et ne valide pas l'intégration Sanctum.

52 / DÉPLOIEMENT

Recette finale du déploiement

Statut initial : NON EXÉCUTÉ. Aucun résultat applicatif, serveur ou compte fournisseur vérifié par ce pack. **Autorisation : GO_PRODUCTION à obtenir de l'utilisateur**, après revue du second développeur.

Les tests de configuration peuvent démarrer avant le frontend ; les essais réels de domaines et de navigateur se terminent en R03. Ces derniers ne peuvent pas être cochés par simple lecture d'un fichier.

ID	Contrôle	Attendu	État / preuve
DEP-AC01	Offre et facture	Produit Datacloud actif, achat éligible, backend réellement hébergé ; configuration, taxe et remise vérifiées.	NON EXÉCUTÉ
DEP-AC02	TLS et session	SPA Vercel et API Systalink : connexion, lecture/écriture, CSRF et déconnexion réussis sur les hôtes définitifs.	NON EXÉCUTÉ
DEP-AC03	Origine non autorisée	CORS/CSRF refusent une origine non approuvée ; aucune autorisation wildcard de previews.	NON EXÉCUTÉ
DEP-AC04	Cookies et B2	Aucun cookie HAAS envoyé aux hôtes B2 ; API B2 limitée aux données fictives.	NON EXÉCUTÉ
DEP-AC05	Runner et secrets	Sous le compte LAB : lecture .env, secrets APP et sauvegardes impossible.	NON EXÉCUTÉ
DEP-AC06	Rôles PostgreSQL	Compte LAB refusé sur haas_app ; absence de droits super-utilisateur/propriétaire ; fixtures isolées.	NON EXÉCUTÉ
DEP-AC07	Concurrence et quota	Un seul run global ; comparaison réserve deux unités ; double clic/rejeu ne double pas la réservation.	NON EXÉCUTÉ
DEP-AC08	Panne et charge	Runner tué/ralenti : aucun faux succès ; API encore disponible ; latences et ressources mesurées.	NON EXÉCUTÉ
DEP-AC09	Vercel et assets	Rafraîchissement des routes, assets, 404 applicative, bonne API et aucune variable VITE secrète.	NON EXÉCUTÉ
DEP-AC10	Chaîne de livraison	Pas de publication avant tests/revue ; backend compatible avant frontend ; SHA et empreintes vérifiables.	NON EXÉCUTÉ
DEP-AC11	Reprise	Sauvegarde restaurée hors production ; périmètre et temps mesurés ; récupération des clés autorisée.	NON EXÉCUTÉ
DEP-AC12	Retour de release	Ancienne version frontend/API compatible restaurée sans perdre de données ; workers rechargés.	NON EXÉCUTÉ
DEP-AC13	Alertes	Indisponibilité, TLS, disque, queue et sauvegarde : alertes reçues par les responsables.	NON EXÉCUTÉ
DEP-AC14	Accords et accès	Ultimate actif sur le projet, Vercel/dépôt/sièges admissibles, aucune fonction Plus présumée ; GO_PRODUCTION humain.	NON EXÉCUTÉ

Pour chaque ligne : date, commit, environnement, étapes, attendu/observé, lien de rapport et relecteur. Un contrôle documentaire passé ne prouve pas ce test serveur. Les cases cochées dans le dépôt existant ne sont jamais effacées ; réévaluer seulement les changements V3.

53 / COMMUNAUTÉ ET CADRAGE

Alignement sur le mail de l'organisateur

Le cadrage fourni [M]

« Une plateforme d'échange pour les développeurs, par les développeurs et pour les développeurs. »

Le message demande d'imaginer un espace où les développeurs peuvent **se retrouver, échanger, poser des questions, partager leurs connaissances, présenter leurs projets et collaborer entre eux**. Il cite **discuter, apprendre, partager, découvrir et construire ensemble**, puis laisse les équipes décider comment organiser ces usages.

Notre traduction produit [H]

Usage du mail	Réponse retenue dans HAAS	Vérification prévue
Se retrouver / découvrir	Projets publics et annuaire volontaire par technologie.	AC55, AC60-63.
Échanger / discuter	Fils existants, commentaires et propositions ; projet facultatif.	AC08, AC58, AC68.
Poser des questions / apprendre	Mode ask_question sans code ni panne obligatoire.	AC64, AC68.
Partager des connaissances	Capsules, procédures, attributions et retours documentés.	AC12-15, AC32.
Présenter des projets	Fiche légère, phase déclarée et aide recherchée.	AC53-59.
Collaborer / construire ensemble	Projet→demande→contribution ; atelier pour les solutions compatibles.	AC67, AC33-46.
Expérience réellement utile	Navigation simple, états lisibles, pilote observé.	AC31, AC65-68 et observations du pilote.

Le mail n'impose aucun de nos écrans, enums, nombres de tests ou modules précis. Ces propositions rendent les usages visibles dans le périmètre d'une équipe de deux, sans messagerie privée, gestion d'équipe ou terminal. Le calendrier, la cession et la remise Datacloud ne sont pas modifiés par ce mail.

Changement de priorité

L'accueil et le pitch présentent une communauté. Le parcours laboratoire reste P0 à livrer comme démonstrateur, mais facultatif pour chaque utilisateur. Les références détaillées F16/F17 suivent ; les choix initiaux encore valides sont conservés.

54 / COMMUNAUTÉ ET CADRAGE

F16 · Présenter les projets

Données et présentation

Champ	Règle proposée
id	UUID serveur ; adresse stable /projets/:id.
owner_id	Utilisateur authentifié ; jamais accepté du navigateur.
name	3-80 caractères.
summary	30-240 caractères ; description lisible sur une carte.
description	80-4 000 caractères ; Markdown restreint et texte inerte.
project_stage	idea, prototype, in_development, launched ; état déclaré par l'auteur, pas validation de qualité.
technologies	1-5 entrées du référentiel partagé ; aucune nouvelle taxonomie séparée.
help_sought	Facultatif, 1 500 caractères maximum ; ce que l'auteur recherche.
demo_url / repository_url	Facultatifs, HTTPS, 2 048 caractères maximum, sans identifiants intégrés ; jamais récupérés par le serveur.
publication_state	draft, published, archived ; distinct de project_stage et de la modération.
hidden_at / moderation_reason	Champs serveur pour retrait ; raison publique expurgée si nécessaire.
lock_version / timestamps	Conflit 409 si édition obsolète ; UTC, historique et audit.

Aucune image téléversée, capture automatique, iframe de démonstration, prévisualisation OpenGraph, clone Git, import de dépôt ou exécution n'est requis. Utiliser un visuel initiales/technologies. Présenter un projet ne transfère pas les droits sur son dépôt, ne prouve pas sa propriété intellectuelle et n'autorise aucune copie de son code.

Cycle et permissions

Le propriétaire actif et vérifié peut créer un brouillon, modifier ses champs, publier et archiver. La publication exige une fiche complète et une confirmation de visibilité. Les modifications publiées sont historisées ; le nom ou le statut n'est pas modifié silencieusement dans les traces de contribution. Un projet archivé reste consultable s'il était public et non masqué, avec l'étiquette « Archivé » ; il n'accepte plus de nouvelle demande liée et il est exclu par défaut du catalogue.

Un modérateur peut masquer/restaurer avec motif, pas réécrire le contenu au nom du propriétaire. Une restauration ne republie pas un brouillon et ne modifie pas la phase déclarée. Un membre suspendu ne peut pas publier, modifier, relier ou archiver. La suspension masque de la découverte ses projets ; la revue décide de la suite selon les règles de modération existantes.

Créer et publier sont des commandes métier distinctes. La création utilise l'idempotence commune ; les mises à jour et transitions contrôlent `lock_version`. Il n'y a ni publication sur simple PATCH de `publication_state`, ni suppression destructive de conversations.

Catalogue et détail

Catalogue public paginé : recherche dans nom/résumé, technologie, phase déclarée, tri date de publication puis id ; pas de score de popularité. Filtres dans l'URL, page de 20, maximum 50. Indexer seulement les requêtes retenues et mesurer le résultat. Le détail montre le propriétaire, les technologies, le besoin d'aide et les échanges accessibles. Aucun compteur ne révèle des demandes privées.

Lien entre projet et demande

`help_requests.project_id` est facultatif. Dans ce périmètre, **seul le propriétaire d'un projet peut lui rattacher sa propre demande**, et uniquement quand le projet est publié, non masqué, non archivé. Cette restriction simple évite des associations trompeuses. Un visiteur contribue ensuite dans les demandes publiques du projet ; il ne devient pas membre d'une équipe ni coauteur du dépôt.

Le projet ne reçoit pas un second fil de commentaires : toutes les discussions utilisent les demandes, commentaires et propositions existants. Sur une fiche propriétaire : « Demander de l'aide pour ce projet ». Pour un autre membre : « Voir les échanges » et, lorsqu'une demande est ouverte, « Proposer une aide » vers cette demande. Si aucune demande n'est ouverte et `help_open=true`, proposer « Je peux t'aider » vers F18 : une offre bornée, puis création d'un fil après consentement et acceptation du propriétaire. Si les coups de main sont fermés, l'expliquer et proposer d'explorer d'autres besoins ; aucun faux bouton de contact.

Changer le rattachement exige une demande encore en brouillon et aucun commentaire/proposition. Après publication, le lien est stable ; un retrait nécessaire passe par la modération audité. Une demande liée n'hérite jamais d'un droit d'accès depuis une URL.

Propagation de visibilité

Un projet masqué ou un propriétaire suspendu rend les listes et détails des demandes rattachées inaccessibles au public, ainsi que leurs commentaires, cas et propositions. Contrôler l'ancêtre dans Query/Policy, recherches, compteurs, notifications, caches et accès direct. Les personnes habilitées conservent une vue de traitement. Un lien enregistré n'accorde jamais le droit de lire.

Une capsule est un objet documentaire indépendant et soumis à revue : si elle cite ce projet/demande devenu masqué, supprimer les détails et extraits du lien source dans la Ressource. Si elle contient elle-même un secret ou le contenu retiré, la masquer/revoir selon la procédure existante ; ne pas promettre qu'un simple masquage efface un secret déjà divulgué.

55 / COMMUNAUTÉ ET CADRAGE

F17 · Découvrir les développeurs

L'annuaire est un moyen simple de se retrouver, pas un système d'amis, de messagerie ou de recrutement. La publication d'un profil dans l'annuaire est une préférence distincte des contributions publiques déjà attribuées.

Champ dans profiles	Règle proposée
directory_visible	Booléen, false par défaut ; choix explicite modifiable par le membre.
help_availability	not_specified, available, unavailable ; auto-déclaré, jamais présence en ligne.
contribution_interests	Texte facultatif, 300 caractères maximum, sans coordonnées privées obligatoires.
availability_updated_at	Horodatage serveur lors d'un changement de disponibilité ; pas last_login.

Une inscription visible dans l'annuaire exige compte actif, courriel vérifié et `directory_visible=true`. Seules les propriétés publiques autorisées sont retournées : pseudonyme, bio courte, technologies, langue, intention d'aide et liens vers les contributions/projets visibles. Ne pas exposer courriel, IP, sessions, téléphone, date de dernière connexion, statut administratif ou données de modération.

Filtres P0 : pseudonyme/recherche textuelle simple, technologie, disponibilité déclarée. Pagination 20/max50 et tri stable par pseudonyme puis id. Pas d'inférence d'expertise, de recommandation IA, de géolocalisation ou de filtre politique/personnel. Le pays demeure facultatif dans le profil existant, mais aucun filtre pays n'est nécessaire au P0.

« Ouvert à une relecture Laravel » ne promet ni réponse ni délai. La date de mise à jour est consultable. Retirer sa présence de l'annuaire purge les résultats/cache concernés, sans effacer automatiquement l'attribution d'une contribution publique. Un compte suspendu ne figure plus dans l'annuaire, même par filtre ou requête directe. Les paramètres privés du propriétaire restent consultables selon le parcours existant du compte.

La collaboration se poursuit dans une demande publique. Le profil conduit aux projets/contributions ; sur un projet ouvert, F18 permet de proposer un coup de main même sans fil initial. Ce n'est ni un contact privé général ni une invitation d'équipe. Le propriétaire accepte et confirme la projection publique du résumé consenti.

56 / COMMUNAUTÉ ET CADRAGE

Questions, données et contrats de communauté

`help_intent` ajoute **ask_question** aux valeurs existantes `unblock`, `review_solution` et `reproduce_behavior`. Cette extension conserve le même fil, les mêmes autorisations et le même service de discussion ; elle ne crée pas un forum parallèle.

Pour ask_question : titre 15–140, contexte dans goal 30–2 000, question dans observed 30–4 000, 1–5 technologies. Les champs expected, attempts, code et environnement sont facultatifs ; ils restent affichables s'ils existent. Expected/attempts deviennent nullable pour ce mode ; migration non destructive et validation conditionnelle explicite. Les autres intentions conservent leurs règles. Le simple mot « aucune » n'est pas exigé pour remplir artificiellement un champ inutile.

Libellés : « Votre question », « Ce que vous souhaitez comprendre », « Ajouter un exemple — facultatif ». Expliquer avant publication que l'échange sera public. Une discussion peut apprendre quelque chose sans accepter une proposition, publier une capsule ou exécuter un test. Les commentaires restent disponibles ; une réponse structurée et sa résolution utilisent les règles actuelles sans transformer l'absence de laboratoire en erreur.

Extensions ciblées de l'architecture

Laravel

Créer Models/Project, Enums/Projects/{ProjectStage,ProjectPublicationState}, Enums/Identity/HelpAvailability, ProjectPolicy, Data/Projects, Requests/Projects, Resources/Projects, Queries/Projects et Services/Projects. Commandes : CreateProject, UpdateProject, PublishProject, ArchiveProject et Moderation dédiée au type project. Les classes portent le suffixe Service selon la convention existante.

Identity conserve les préférences de profil et l'annuaire : UpdateProfileRequest/Data/Service étendus par liste de champs, ListDevelopersRequest/Query et DeveloperSummaryResource séparée de MeResource. Pas de sérialisation brute de User. HelpRequests vérifie le lien au projet et l'intention ; pas de ProjectService monolithique ni d'accès Eloquent dispersé dans les contrôleurs.

Données

Table projects (UUID, owner_id, données éditoriales, états séparés, lock_version, published_at, hidden_at, timestamps). Pivot project_technologies unique (project_id,technology_id). Ajouter help_requests.project_id nullable avec FK ; aucune suppression en cascade des discussions. Étendre profiles et enum HelpIntent ; migration de données existantes contrôlée.

Index proposés : projects(publication_state,published_at,id), projects(owner_id,created_at), project_technologies(technology_id,project_id), help_requests(project_id,created_at). L'annuaire filtre le statut/vérification dans users et l'opt-in dans profiles ; mesurer avant d'ajouter un index supplémentaire.

API cible

Méthode et route /api/v1	Contrat
GET /projects	Catalogue public filtré ; brouillons/masqués jamais exposés.
GET /projects/{project}	Détail accessible ; capacités calculées selon l'acteur.
POST /projects	Créer son brouillon ; membre actif vérifié ; Idempotency-Key.
PATCH /projects/{project}	Champs autorisés et lock_version ; propriétaire.
POST /projects/{project}/publish	Valider fiche/visibilité puis publier ; propriétaire.

Méthode et route /api/v1	Contrat
POST /projects/{project}/archive	Archiver avec motif et lock_version ; propriétaire.
GET /projects/{project}/requests	Seulement demandes et compteurs visibles à cet acteur.
GET /developers	Liste volontaire, comptes actifs vérifiés, filtrage borné.
PATCH /me/profile	Étendre la route de profil existante ; si son chemin diffère dans le dépôt, garder un seul chemin et le documenter.
POST /requests	Ajouter project_id facultatif et help_intent=ask_question ; toutes les règles existantes s'appliquent.
POST /reports	Type project dans la liste autorisée ; pas de classe libre.

Les actions de modération réutilisent les routes d'administration des signalements. Erreurs HAAS communes (401/403/404/409/422/429), tests de schéma et types OpenAPI générés. Une route ci-dessus est une cible, pas une implémentation déjà fournie.

React

Créer features/projects et features/developers, avec api/models/schemas/hooks/queries/components/pages/tests selon besoin. Le client HTTP, guards, erreurs, forms et composants sont réutilisés. shared n'importe pas ces features. Les modèles de transport viennent d'OpenAPI, pas d'un doublon manuel.

Capacités can.edit, can.publish, can.archive, can.create_request calculées par le serveur. Les boutons suivent ces capacités sans remplacer les Politiques. Invalider catalogue, détail, listes de demandes et profils concernés après mutation. Un retrait de l'annuaire purge la recherche et le cache de profil adapté sans fabriquer un nombre de membres.

57 / COMMUNAUTÉ ET CADRAGE

Recette de l'alignement communautaire

Ces critères sont des prescriptions HAAS [H] fondées sur les usages [M], pas des résultats obtenus. Ils complètent AC01-52 sans les remplacer. AC47 reste le seul conditionnel au P1 de copie de contexte. Chaque preuve future doit mentionner commit, environnement, observé et incident éventuel.

ID	Priorité	Exigence	Situation et résultat attendu
AC53	P0	F16	Compte actif vérifié crée un brouillon : owner_id serveur ; publication refusée si champs requis manquants ; aucun brouillon dans le catalogue public.

ID	Priorité	Exigence	Situation et résultat attendu
AC54	P0	F16	Autre membre modifie/publie/archive le projet, ou fournit owner_id/publication_state dans PATCH : refus et aucune mutation ; modérateur ne réécrit pas le propriétaire.
AC55	P0	F16	Publication valide puis recherche filtrée et pagination : données complètes, ordre stable, phase affichée comme déclaration, aucun projet privé/masqué exposé.
AC56	P0	F16	Modification avec lock_version périmé : 409 et aucune écriture perdue ; archivage ferme les nouvelles demandes, garde l'historique autorisé et retire le projet du catalogue par défaut.
AC57	P0	F16	Idempotency-Key rejouée : un seul projet ; même clé avec contenu différent : 409 ; liens dangereux/identifiants URL rejetés, aucune requête serveur vers les liens saisis.
AC58	P0	F16/F02	Le propriétaire rattache sa demande à son projet publié ; un tiers, un projet privé/archivé/masqué ou un changement après publication sont refusés ; project_id absent reste valable.
AC59	P0	F16/F09	Un projet masqué disparaît du catalogue et de l'accès direct public ; demandes liées, cas, compteurs, notifications et caches suivent ; sources de capsules expurgées/réexaminées si nécessaire.
AC60	P0	F17	Annuaire vide par défaut pour un compte sans opt-in ; seul un compte actif vérifié volontaire apparaît ; courriel/IP/sessions/last_login ne figurent jamais dans la réponse.
AC61	P0	F17	Recherche technologie/disponibilité et pagination retournent uniquement les membres autorisés dans un ordre stable, sans classement d'expertise ni score fabriqué.
AC62	P0	F17/F09	Désactivation directory_visible ou suspension : retrait des résultats et cache annuaire ; anciennes contributions publiques gardent seulement l'attribution autorisée, pas les données privées.
AC63	P0	F17	Disponibilité modifiée par le propriétaire : libellé auto-déclaré et horodatage ; aucun statut « en ligne » inventé ; un tiers ne peut modifier les préférences.
AC64	P0	F02	ask_question sans code, expected ou attempts : enregistrement accepté si titre/contexte/question/technologie valides ; 422 sur les vrais champs manquants ; règles unblock inchangées.
AC65	P0	F16/F17	Parcours mobile 360/390 px et clavier : projet consultable, membres filtrables, formulaire publiable ; focus et erreurs lisibles ; aucun défilement horizontal global.
AC66	P0	F16/F17	Liste réellement vide distincte d'une panne réseau ; 401/403/409/422/429 correctement rendus ; déconnexion/changement de compte ne révèle pas les brouillons ni les préférences.

ID	Priorité	Exigence	Situation et résultat attendu
AC67	P0	F01-F18	A publie projet et demande ; B trouve le projet/membre et contribue ; A accepte si pertinent ; C lit la capsule publiée et les contributions. Origine éditoriale/démo explicite ; branche B1 optionnelle dans ce parcours.
AC68	P0	F02/F03/F17	Un membre pose une question de connaissance ; un autre commente utilement. Aucun dépôt, correctif, test, abonnement payant, messagerie privée ou résultat laboratoire n'est requis pour échanger.

Réception finale

Exécuter les nouveaux tests de contrat, propriété, visibilité parent, liste volontaire, question sans code et UI. Étendre BACKEND_GATE avant GO_FRONTEND, puis FRONTEND_GATE avec UX18-20 et le parcours communautaire. Les sous-lots et contrôles de déploiement restent actifs. Un test de palette ou de structure documentaire ne vaut pas réussite d'un parcours applicatif.

58 / LA RENCONTRE PAR LE COUP DE MAIN

F18 · La rencontre par le coup de main

Choix HAAS approuvé pour la mise à jour du pack ; pas une nouvelle prescription littérale du mail. ADR-006 explicite les règles remplacées.

Résultat visé et périmètre

Un propriétaire ouvre son projet à un petit coup de main. Une autre personne propose un apport concret. Le propriétaire accepte et un échange public contextualisé commence dans les fils existants. Une réponse, un test, une relecture ou une amélioration de documentation peut ensuite contribuer au projet. La capsule et le laboratoire restent facultatifs pour les personnes ; les démonstrateurs prévus restent à livrer.

Deux entrées d'accueil : « **Faire avancer mon projet** » et « **Donner un coup de main** ». Explorer les projets, lire les solutions ou poser une question reste possible sans choisir une entrée. Aucune contribution préalable n'est exigée pour demander de l'aide.

F18 ajoute une offre bornée avec consentement, pas une messagerie générale, un contrat de prestation, une équipe ou un accès au dépôt. Aucun paiement, calendrier, rendez-vous, délai de réponse garanti, chat privé, terminal, classement ou moteur IA. Un seul échange d'offre avant acceptation ; pas de réponses privées imbriquées.

Évolution explicite de F16 : en l'absence de demande ouverte, un membre peut maintenant proposer son aide si le propriétaire a activé l'ouverture aux coups de main. Le tiers ne peut toujours pas créer librement une demande au nom du propriétaire ni la rattacher à son projet. C'est l'acceptation du propriétaire qui autorise le lien, dans une transaction. Les discussions existantes restent utilisables quand les offres spontanées sont fermées.

Ouverture volontaire et types de contributions

Ajouter aux projets `help_open`, faux par défaut, et `help_categories`. Pour ouvrir : propriétaire actif et vérifié, projet publié et non masqué/archivé, `help_sought` décrivant le petit résultat attendu (30-1 500 caractères), 1 à 5 catégories. Aucun projet existant ne devient ouvert lors de la migration. Fermer ne supprime ni projet ni discussion.

Valeur stable	Libellé UI	Exemple de contribution
<code>usability_feedback</code>	Retour sur l'interface	Signaler où le parcours devient difficile.
<code>code_review</code>	Relecture technique	Examiner un extrait volontairement partagé.
<code>reproduce_behavior</code>	Reproduction d'un comportement	Décrire les étapes et l'observation obtenue.
<code>explain_concept</code>	Explication	Clarifier un concept avec son contexte.
<code>documentation</code>	Documentation	Essayer ou améliorer une procédure.

Ces catégories décrivent une intention, pas une expertise certifiée. Dans le profil, `preferred_help_categories` est facultatif ; les technologies existent déjà. La visibilité annuaire (`directory_visible`) reste un consentement distinct : aider n'exige pas l'opt-in à l'annuaire. Un compte suspendu ne peut ni ouvrir, proposer, accepter, retirer ou décliner ; il garde seulement les accès de recours existants.

Découvrir une occasion concrète d'aider

GET `/api/v1/help-opportunities` agrège deux types explicites :

- `request` : demande publique ouverte/en cours, auteur actif, visibilité parent respectée. L'action mène au fil existant pour y contribuer directement, sans offre obligatoire.
- `project` : projet publié, non masqué/archivé, propriétaire actif, `help_open=true`. L'action propose un coup de main. Si une demande publique pertinente existe, la carte propose d'abord de la consulter ; un projet ne doit pas remplir le flux avec des doublons de cartes quasi identiques.

Filtres en liste autorisée : technologie, catégorie, type ; pagination 20/max50, tri `published_at` DESC, `id` ASC stable, pas de score social. Les demandes ne portent pas toutes une catégorie : ne pas en inférer une ; le filtre catégorie ne les retient que si une catégorie a été déclarée dans le besoin. Pour limiter la migration, utiliser la catégorie du projet pour les demandes liées et présenter cette origine explicitement. Les demandes autonomes sans catégorie restent dans la vue non filtrée.

Les préférences ne servent qu'à préremplir les filtres après accord et à expliquer les correspondances exactes. Le visiteur peut modifier/retirer chaque filtre. Afficher par exemple « React figure parmi vos technologies sélectionnées ». Ne pas écrire « Le meilleur développeur pour vous ». Les raisons sont dérivées de champs publics, jamais du dernier login ou d'une inférence d'expertise.

Une liste vide signifie seulement qu'aucun besoin visible ne correspond. Actions : retirer un filtre, explorer les projets, poser une question. Une panne réseau ne devient pas un état vide. Le catalogue public ne révèle aucun nombre d'offres en attente ou refusées.

Proposition limitée et consentement à la publication

Un membre actif vérifié, autre que le propriétaire, remplit une fiche courte. Avant acceptation, seul le proposant, le propriétaire et la modération habilitée sur un signalement peuvent lire l'offre. Il n'existe pas de galerie publique d'offres ni de refus publics.

Champ proposé	Contrat
<code>category</code>	Une catégorie actuellement ouverte sur le projet.
<code>public_summary</code>	30-800 caractères ; apport concret, pas coordonnées personnelles ni secret.
<code>expected_outcome</code>	20-500 caractères ; résultat limité : observation, explication ou procédure.
<code>allow_public_summary</code>	Booléen explicitement true : accord pour afficher ces deux textes dans un échange public si l'auteur accepte. Jamais précoché.

L'aperçu montre exactement les deux textes qui pourront devenir publics, le projet et le pseudonyme attribué. Le formulaire prévient : « Cette proposition est visible par vous et le propriétaire. Si elle est acceptée, ce résumé et le résultat proposé rejoindront un échange public. » Le contact e-mail n'est jamais ajouté. Pas de texte caché importé depuis une conversation privée, pas de fetch de lien, d'image, de fichier ou d'exécutable.

Les textes d'une offre soumise sont figés pour cette décision. Pour les modifier, retirer l'offre encore en attente puis en créer une nouvelle, dans les limites d'usage. Le propriétaire ne modifie pas silencieusement les mots du proposant.

Le refus ne donne pas de malus ; le proposant peut retirer une offre en attente. Un motif de refus est facultatif, court, privé aux deux acteurs, modérable et non recopié dans les notifications. Pas de rappel automatique agressif.

États et transitions

HelpOfferState = pending | accepted | declined | withdrawn | expired.

État de départ	Action / acteur	État d'arrivée	Effet
absent	proposer / membre autorisé	pending	Offre privée, expiration à 7 jours UTC, notification minimale au propriétaire.
pending	accepter / propriétaire	accepted	Crée ou rejoint UN fil autorisé, publie le résumé consenti, inscrit un événement attribué.
pending	décliner / propriétaire	declined	Aucun fil, aucune pénalité publique.
pending	retirer / proposant	withdrawn	Aucun fil ; possibilité de proposer de nouveau dans les limites.
pending	échéance atteinte / système	expired	Aucun fil ni acceptation tardive ; motif interne time_limit.
pending	fermeture, archivage, masquage ou suspension / système	expired	Motif interne adapté ; pas de refus attribué artificiellement au propriétaire.
accepted	consulter le fil	accepted	L'accord est une trace historique, pas un travail achevé.

Les états terminaux sont immuables ; pas de retour automatique à pending. Une offre acceptée ne porte pas de statut « terminé », de score ou de preuve technique. Le travail se poursuit dans le fil. Si quelqu'un s'arrête, il peut le signaler dans le fil ; l'archivage/réouverture existants restent les mécanismes de suivi. Le retrait d'un contenu dangereux peut expurger la projection publique via la modération, sans falsifier l'historique.

L'expiration est évaluée sur le serveur à chaque lecture ou commande critique, pas uniquement par cron. La purge physique et la rétention suivent une décision documentée de confidentialité ; « expiré » ne signifie pas supprimé.

Accepter sans fabriquer un projet, un fil ou un résultat

La page d'acceptation montre l'offre figée et l'espace public de destination. Le propriétaire choisit explicitement :

Créer un échange. Le titre, le contexte, la question/le point à examiner et l'intention sont préremplis pour économiser la saisie, mais restent à relire. La publication valide tous les champs nécessaires au même `CreateHelpRequestService` que les demandes ordinaires. Pour `ask_question`, aucun code, `expected` ou `attempts` n'est exigé. `publish_consent=true` est un accord distinct du propriétaire, non précoché. Le serveur fixe `author_id` au propriétaire actuel ; aucune usurpation du proposant.

Rejoindre un échange. Le propriétaire sélectionne une de ses demandes publiques, ouvertes/en cours, rattachées à CE projet. L'interface affiche le fil exact ; pas de sélection silencieuse « dernier fil ». La commande vérifie de nouveau tous les droits/états. Aucun titre, texte ou contenu déjà présent n'est remplacé.

Dans les deux cas : transaction unique pour décision, création éventuelle du fil, lien unique et projection publique du résumé consenti. Un événement système `help_offer_accepted` porte l'acteur décideur et le proposant distinctement ; il n'est pas présenté comme un commentaire rédigé par ce dernier. Aucune « contribution terminée » n'est ajoutée. Les permissions du proposant restent celles d'un membre ordinaire.

Une seule offre active en attente par couple projet/proposant. Pour éviter la multiplication accidentelle de fils, l'acceptation n'a jamais un mode implicite. Elle crée une nouvelle demande uniquement après confirmation de ce choix ; sinon elle rattache le fil sélectionné. Plusieurs offres acceptées peuvent rejoindre le même fil si le propriétaire le choisit explicitement.

Transactions, doublons et limites d'usage

Toutes les commandes sensibles utilisent actor serveur, DTO validé, Policy, transaction, audit et Resource whitelist. `lock_version` obligatoire sur les transitions ; `Idempotency-Key` obligatoire sur création et acceptation. Le jeu de la même clé/empreinte rend le même identifiant sans nouvelle discussion/notification. Même clé et corps différent : 409. Les droits et la visibilité courants sont revérifiés avant lecture d'une réponse mémorisée ; ne pas rendre un ancien contenu devenu masqué.

Ordre de verrouillage à documenter et partager pour les commandes qui touchent plusieurs objets : lignes de quotas/acteurs nécessaires dans un ordre déterministe, projet, offre, puis demande liée. Ne jamais acquérir ces verrous en ordre inverse. Les services appelés ne doivent ni ouvrir un second commit ni envoyer de mail dans la transaction. Revalider projet ouvert, catégorie, acteurs,

délai, état et lien. Les autres commandes qui peuvent invalider ces invariants participent au protocole de verrouillage et aux tests de concurrence (fermeture, suspension, archivage, acceptation et retrait).

Contraintes minimales : index unique partiel `help_offers(project_id, proposer_id) WHERE state='pending'` ; unicité de la projection publique `source_offer_id` ; FK de `accepted_request_id` avec contrôle du même projet/auteur ; `accepted` implique `request_id/accepted_at` non nuls ; états et dates cohérents. Pas de cascade qui efface les discussions. Expirer les offres périmées avant de libérer/ créer une nouvelle offre ; un index unique ne gère pas le temps automatiquement.

Limites proposées et configurables : **5 créations par 24 heures glissantes et par membre, 5 offres en attente globalement par membre, 1 offre en attente par projet/membre**. Deux créateurs concurrents ne contournent pas ces limites : réservation atomique côté serveur, pas un COUNT puis INSERT non verrouillé. Ces seuils sont des choix à ajuster après essai, pas des garanties de réponse ni un quota de laboratoire. Notification interne une fois après commit par événement, avec rattrapage/clé d'unicité prévus selon les mécanismes existants.

Confidentialité, retraits et modération

Avant acceptation, l'offre ne figure ni dans recherche publique, ni profil, ni compteur, ni payload d'un projet. Après acceptation, seule sa projection consentie est visible dans le fil si tous les parents sont visibles. Les états `declined/withdrawn/expired`, motifs privés et identifiants d'acteurs non publics ne sont jamais divulgués par la fiche de progrès.

Le masquage/archivage ferme les nouvelles offres et empêche leur acceptation ; le masquage rend les fils liés inaccessibles au public selon F16. Une fermeture de `help_open` seule n'efface pas les fils déjà ouverts. La suspension d'un proposant n'invente pas un échec de son travail ; elle bloque ses actions et les décisions en attente. Le traitement respecte la différence entre l'annuaire opt-in et l'attribution publique existante.

La modération peut consulter une offre signalée dans le dossier concerné, masquer sa projection ou traiter le signalement avec audit. Aucun administrateur non propriétaire ne l'accepte au nom de l'auteur. Aucune donnée brute d'offre dans les logs ou notifications ; lien authentifié et message minimal. À la déconnexion : annuler lectures et vider les caches privés, y compris les offres reçues/envoyées. Ne pas mettre ces réponses dans un cache CDN public.

Voir ce que nous avons amélioré ensemble

La fiche projet affiche une vue des échanges et résultats autorisés : contribution liée, résolution validée par l'auteur, capsule publiée et, lorsque disponible, vérification de la bonne version. L'offre acceptée peut apparaître comme « **Collaboration commencée** », jamais comme « **Amélioration réalisée** » avant un résultat enregistré.

Cette vue ne crée ni nouveau score, ni certificat, ni pourcentage de projet terminé. Elle n'affiche pas les offres en attente. Une explication utile sans résolution peut rester visible comme échange, sans être promue automatiquement en correction vérifiée. Les données de démo sont identifiées et exclues des chiffres d'adoption.

59 / LA RENCONTRE PAR LE COUP DE MAIN

Contrats et architecture des coups de main

Routes proposées

Route sous /api/v1	Entrée et accès	Réponse attendue
GET /help-opportunities	Public ; filtres technology_id, category, type, page, per_page en liste autorisée. Les préférences privées restent au client ou dans une lecture privée distincte.	200 data/meta ; uniquement besoins publics visibles ; aucune offre pending.
PUT /projects/{project}/help-settings	Propriétaire actif vérifié ; help_open, help_categories, help_sought, lock_version.	200 ProjectResource ; fermeture expire les offres pending et conserve les fils.
POST /projects/{project}/help-offers	Membre actif vérifié non propriétaire ; category, public_summary, expected_outcome, allow_public_summary=true ; Idempotency-Key.	201 HelpOfferResource privée, state=pending ; Location.
GET /me/help-offers	Acteur connecté autorisé ; direction=sent ou received, state, page, per_page.	200 collection privée ; proposer/owner seulement, pas de filtre user_id arbitraire.
GET /help-offers/{offer}	L'un des deux acteurs ; modérateur uniquement via dossier signalé autorisé.	200 ressource privée autorisée ; 404 si autre acteur.
POST /help-offers/{offer}/accept	Propriétaire actif vérifié, lock_version, publish_consent=true, destination discriminée ; Idempotency-Key.	200 data.offer + data.request minimal avec id/URL autorisés. Même identifiant au rejeu valide.
POST /help-offers/{offer}/decline	Propriétaire, pending, lock_version, reason facultatif max300.	200 état terminal ; répétition hors clé de rejeu =409 si décision déjà prise.
POST /help-offers/{offer}/withdraw	Proposant, pending, lock_version.	200 withdrawn ; pas de retrait d'une acceptation historique par cette route.
GET /projects/{project}/progress	Public si projet visible ; pagination.	200 projections autorisées des fils/résolutions/capsules ; pas d'offres non acceptées.
PATCH /me/profile (étendue)	Préférences de catégories facultatives ; conserver la route existante.	200 profil propre ; pas d'effet sur directory_visible.

Route sous /api/v1	Entrée et accès	Réponse attendue
POST /reports (étendue)	Type autorisé help_offer ; visibilité de la cible contrôlée pour le signaleur.	201 ; traitement de modération via mécanisme existant.

Les commandes decline/withdraw peuvent accepter la clé commune de rejeu ; sans clé, elles doivent rester sans double effet via lock_version et transitions. Ne pas créer deux mécanismes d'idempotence concurrents.

Exemple de proposition (données fictives)

```
{
  "category": "usability_feedback",
  "public_summary": "Je peux essayer le formulaire sur mon téléphone et décrire les étapes difficiles.",
  "expected_outcome": "Une liste d'observations et des étapes reproductibles.",
  "allow_public_summary": true
}
```

owner_id, proposer_id, state, timestamps, accepted_request_id, permissions et champs d'audit sont fixés par le serveur et rejetés en entrée. La proposition ne comporte pas de coordonnées obligatoires, de pièces jointes ou de commande exécutable. Aucune donnée n'est récupérée depuis un lien.

Deux destinations d'acceptation, jamais implicites

Créer : destination.mode=create, destination.request contient les champs validés de CreateHelpRequestData, project_id et author_id étant fixés par le serveur. Prévisualiser et confirmer avant requête. Par défaut suggérer l'intention adaptée, sans forcer ask_question pour contourner les champs requis d'une véritable panne.

```
{
  "lock_version": 1,
  "publish_consent": true,
  "destination": {
    "mode": "create",
    "request": {
      "title": "Un premier retour sur le formulaire de commande",
      "help_intent": "ask_question",
      "goal": "Nous souhaitons savoir quelles étapes du formulaire sont difficiles à comprendre.",
      "observed": "Pouvez-vous décrire les hésitations rencontrées pendant votre essai ?",
      "technologies": [{"technology_id": "9d78ec0c-3e7b-428b-a11a-8dc9a5d62b67"}]
    }
  }
}
```

La forme exacte de technologies est à rapprocher du contrat déjà implémenté ; cet exemple fixe l'intention, pas une seconde représentation contradictoire. Les champs de publication doivent traverser le validateur commun, y compris validation conditionnelle selon help_intent. Aucun new Request fabriqué ni contournement d'autorisation pour réutiliser le service.

Rattacher : destination.mode=attach, request_id UUID. Le fil doit appartenir au propriétaire, au même projet, être public, ouvert/en cours et visible. Aucune modification de son contenu initial. Afficher son titre dans la confirmation.

```
{
  "lock_version": 1,
  "publish_consent": true,
  "destination": {
    "mode": "attach",
    "request_id": "9d78ec0c-3e7b-428b-a11a-8dc9a5d62b68"
  }
}
```

Réponse : `data.offer.id/state/accepted_request_id` et `data.request.id` ; le frontend navigue vers la route interne autorisée existante, pas vers une URL arbitraire fournie par un utilisateur. La projection publique est un événement système explicitement attribué avec le résumé consenti, non un commentaire usurpé. Les champs `private_reason`, `status pending` des autres offres ou email ne sont jamais inclus dans la projection.

Erreurs et reprise

Code HTTP	Code métier proposé	Comportement
401	AUTHENTICATION_REQUIRED	Connexion, aucune mutation automatique après retour.
403	ACCOUNT_NOT_ELIGIBLE	Compte non vérifié/suspendu ou action interdite, selon règle de confidentialité.
404	RESOURCE_NOT_FOUND	Ressource masquée ou acteur non autorisé, sans révélation de l'existence d'une offre.
409	PROJECT_NOT_OPEN_FOR_HELP	Projet fermé/archivé ou catégorie retirée ; ne pas perdre la saisie en mémoire.
409	HELP_OFFER_ALREADY_PENDING	Une offre pending existe déjà pour le couple.
409	HELP_OFFER_STALE_VERSION	Un autre acte a changé <code>lock_version</code> .
409	HELP_OFFER_NOT_PENDING	Déjà acceptée/refusée/retirée/expirée ; aucun nouveau fil.
409	HELP_REQUEST_UNAVAILABLE	Fil cible devenu fermé/masqué ; choisir une destination après relecture.
409	IDEMPOTENCY_CONFLICT	Même clé avec autre contenu.
422	VALIDATION_FAILED	Champs invalides ou consentement absent ; <code>errors.fields</code> exploitable.
429	HELP_OFFER_LIMIT_REACHED	Délai si disponible ; aucune création partielle.
500/503	INTERNAL_ERROR / SERVICE_UNAVAILABLE	Message neutre, <code>request_id</code> ; pas de succès ou échec final inventé.

Après perte réseau, vérifier l'état réel et rejouer seulement la même intention couverte par idempotence. Ne pas produire une nouvelle clé pour masquer une incertitude. Une 409 n'est pas une déconnexion. Les offres et payloads de confirmation ne vont ni dans localStorage ni dans les logs de diagnostic.

Modèle et frontières Laravel

HelpOffer : UUID, project_id, proposer_id, category enum, public_summary, expected_outcome, allow_public_summary_at, state enum, lock_version, expires_at, decided_at, accepted_at, accepted_by nullable, accepted_request_id nullable, declined_reason nullable, closed_reason nullable, timestamps. Ne pas persister le consentement comme un simple booléen sans acteur/date. Projet conserve help_open/help_categories et son historique ; profils conservent preferred_help_categories sans catégorie d'expertise.

Contraintes et indexes : unique pending par project/proposer, indexes owner via projects, (proposer_id,state,created_at), (state,expires_at), FK sans cascade sur les fils, validation cohérence accepted/request/decideur. Les FK et services empêchent de rattacher une offre à la demande d'un autre projet. Le snapshot de publication porte la provenance source_offer_id unique et peut être retiré par modération ; il ne crée pas une deuxième messagerie.

Arborescence cible :

```
app/Enums/HelpOffers/{HelpOfferState,HelpContributionCategory}.php
app/Data/HelpOffers/{CreateHelpOfferData,AcceptHelpOfferData,DeclineHelpOfferData}.php
app/Http/Controllers/Api/V1/HelpOffers/...
app/Http/Requests/HelpOffers/...
app/Http/Resources/HelpOffers/{HelpOfferResource,HelpOpportunityResource}.php
app/Models/HelpOffer.php
app/Policies/HelpOfferPolicy.php
app/Queries/HelpOffers/{ListHelpOpportunitiesQuery,ListMyHelpOffersQuery}.php
app/Queries/Projects/ProjectProgressQuery.php
app/Services/HelpOffers/{CreateHelpOfferService,AcceptHelpOfferService,
    DeclineHelpOfferService,WithdrawHelpOfferService,ExpireHelpOffersService}.php
app/Services/Projects/UpdateProjectHelpSettingsService.php
```

Un controller par commande sensible, services courts par cas d'usage, DTO non HTTP, Policy sans écriture, Resources publiques/privées séparées, événement après commit dédoublé. Pas de HelpOfferManager ou repository universel. Le service d'acceptation garde la transaction englobante ; il utilise le cas de création de demande et l'audit existants sans duplication des règles.

React, cache et tests

features/help-offers contient api/models/schemas/queries/hooks/components/pages/tests. Hooks proposés : useHelpOpportunities, useMyHelpOffers, useCreateHelpOffer, useAcceptHelpOffer, useDeclineHelpOffer, useWithdrawHelpOffer. Un service frontend n'est ajouté que pour une vraie orchestration ; Axios reste dans core/http.

Clés publiques pour catalogue/pagination/filtres ; clés privées avec identité pour offres et permissions. Mutations : invalider projet/catalogue/opportunités, offres envoyées/reçues et notifications ; après acceptation, le fil et la progression. Aucune acceptation optimiste. Annulation et purge à la déconnexion/changement de compte. Guards auth/vérifié/actif pour agir ; catalogue public disponible aux visiteurs. Pas de visibilité publique attribuée par un guard.

Les formulaires de confirmation distinguent deux consentements ; demandes conditionnelles réutilisent les mêmes schémas que la création normale. Tests : propriété, opt-in, compte, publication, limite, TTL, concurrence, idempotence, cache privé, a11y, choix du fil et résultat non simulé. Tous AC69-90 doivent avoir des observations de tests réels avant réception.

60 / LA RENCONTRE PAR LE COUP DE MAIN

Une première collaboration facile à comprendre

UX21 — Trouver une première occasion d'aider

Routes : accueil et /coups-de-main. Deux actions « Faire avancer mon projet » / « Donner un coup de main », plus accès à Explorer et Poser une question sans tunnel. Carte : projet, besoin borné, stack, catégorie, raison factuelle du filtre, action nommée. Un fil public ouvert mène directement à sa discussion ; un projet ouvert aux offres permet de proposer une aide. Aucun indicateur en ligne ni délai garanti.

États : filtres modifiables dans l'URL, choix de catégories facultatif, six cartes maximum au premier écran puis pagination. Vide : enlever un filtre/explorer/poser une question. Erreur réseau distincte, liste sans faux contenu. Projets privés et offres pending absents. Corps 16px, contrôles confortables 44px visés, focus visible, sans colonne qui déborde à 360px.

UX22 — Proposer un coup de main en confiance

Route : /projets/:id/coup-de-main. Formulaire court : catégorie, ce que je propose, résultat recherché. Au-dessus : projet et propriétaire, visibilité du résumé. Aperçu exact de la projection future et case non précochée « J'accepte que ce résumé et le résultat proposé deviennent publics si le propriétaire accepte ». Aucun e-mail demandé ni secret stocké localement.

Action : « Envoyer ma proposition ». Après API : « Proposition envoyée au propriétaire. Vous pouvez la retirer tant qu'elle n'est pas acceptée. » Pas « collaboration réussie ». Les catégories/états sont revalidés côté serveur. 409 explique fermeture ou autre pending ; 422 reste au champ ; 429 explique le délai ; réseau incertain réconcilie l'état avant retry. Texte lisible, labels visibles et aperçu clavier avant envoi.

UX23 — Décider et rejoindre l'échange

Routes : /espace/coups-de-main (Reçus / Envoyés), /espace/coups-de-main/:id ; progression sur la fiche projet et événement sur le fil existant. Il s'agit de propositions ponctuelles, pas d'un chat privé. Montrer projet, résumé, date, état textuel et actions autorisées.

Accepter : aperçu du résumé consenti ; choix explicite « Créer un échange » ou « Rattacher à un échange ouvert ». En création, propriétaire relit les champs préremplis ; en rattachement, titre exact du fil visible. Seconde case non précochée de publication. Bouton « Accepter et ouvrir l'échange » puis navigation interne seulement après 200. Aucun droit équipe/dépôt ajouté. Décliner : motif facultatif privé et sans pression. Retirer : proposant, pending uniquement. Expiration visible sans faux refus humain.

États : accepted signifie « Collaboration commencée », lien vers fil ; progression ultérieure vient des contributions réelles. Pending/declined/withdrawn/expired jamais dans le flux public. Annulation/réseau, 409 et session expirée ne perdent pas discrètement la saisie ; purge du cache privé à la déconnexion. Mobile une colonne, tableaux transformés en listes lisibles, pas de toast seul pour une erreur bloquante.

Pilote proposé

Scénario fictif : Awa présente une application et ouvre un retour d'interface ; Moussa propose de tester ; Awa accepte ; un fil public consenti est créé ; l'observation puis la correction sont documentées. Une comparaison B1 peut ensuite illustrer un cas approuvé. Le résumé d'offre, la contribution réalisée et le rapport machine restent distingués.

Cible qualitative proposée : observer trois petits coups de main entre testeurs volontaires, en consignant aussi refus, abandons et non-réponses. Mesurer découverte→offre, offre→acceptation et acceptation→premier apport utile. Ne pas interpréter une acceptation comme un travail accompli. Un testeur doit pouvoir expliquer ce qui deviendra public avant de confirmer. Les résultats, durées et noms ne sont renseignés que s'ils ont réellement été observés et autorisés à la présentation.

61 / LA RENCONTRE PAR LE COUP DE MAIN

Recette des coups de main AC69-AC90

Référentiel à exécuter sur l'application. Ces contrôles ne sont pas validés par la rédaction de ce dossier. Les 68 scénarios antérieurs sont conservés, AC47 seul conditionnel.

ID	Priorité	Exigence	Situation et résultat attendu
AC69	P0	F18	Ouverture volontaire. Projet existant ou nouveau : help_open=false. Seul propriétaire actif vérifié ouvre un projet publié avec besoin/categories valides ; création ou publication ne coche pas l'opt-in.
AC70	P0	F18	Réglages et droits. Autre membre/admin non propriétaire ne change pas help_open/categories ; catégorie vide, projet brouillon/masqué/archivé ou lock_version ancien refusés sans perte.
AC71	P0	F18	Découverte explicable. Catalogue renvoie demandes publiques et projets ouverts, filtres stables et raisons exactes ; aucun pending privé, score expert, dernière connexion ou obligation annuelle.
AC72	P0	F18	Aucune occasion ou réseau. Aucun résultat propose enlever filtres/explorer/question ; un incident HTTP n'affiche pas « aucun besoin ». Les demandes ordinaires restent accessibles.
AC73	P0	F18	Consentement du proposant. Création active vérifiée avec résumé/résultat et accord explicite : pending privé ; refus 422 si accord absent. Aucun texte public, email ou secret recopié automatiquement.
AC74	P0	F18	Acteurs et champs protégés. Auto-offre, rôle imposé, owner_id/proposer_id/state/accepted_request_id forgés ou compte suspendu/non vérifié : refus et zéro effet.

ID	Priorité	Exigence	Situation et résultat attendu
AC75	P0	F18	Création sans doublon. Même Idempotency-Key/charge : même offre ; contenu différent : 409. Deux clés concurrentes pour même couple : une seule pending et erreur contrôlée.
AC76	P0	F18	Quotas atomiques. Requêtes simultanées sur plusieurs projets ne dépassent pas 5 créations/24h ni 5 pending ; échec ne laisse pas de réservation fantôme ni notification.
AC77	P0	F18	Confidentialité des offres. Seuls deux acteurs lisent leurs offres ; autres utilisateurs et admin non habilité reçoivent refus sans fuite. Modération seulement sur dossier signalé audité.
AC78	P0	F18	Acceptation vers nouveau fil. Deux consentements présents, champs normaux validés : 1 demande du propriétaire, 1 lien, 1 projection consentie. Un échec de validation/transaction laisse pending et aucun fil.
AC79	P0	F18	Acceptation vers fil existant. Choix explicite même propriétaire/projet/état public ouvert : lien créé sans modifier texte. Fil autre projet/auteur, clos/masqué ou choix implicite rejeté.
AC80	P0	F18	Acceptation idempotente concurrente. Double clic, nouvelle tentative même clé et deux acceptations concurrentes : une seule décision, pas deux discussions, projections ou notifications ; vieux lock_version=409.
AC81	P0	F18	Course acceptation/retrait. Accept/withdraw, accept/decline et accept/fermeture réellement concurrentes aboutissent à un seul état cohérent ; aucun fil d'une offre retirée/fermée.
AC82	P0	F18	Expiration et fermeture. Après 7 jours, acceptation refusée même cron arrêté. Fermeture/catégorie retirée/archivage expire pending ; accepted historique conservé ; nouvelles offres impossibles.
AC83	P0	F18	Suspension et masquage. Suspension propriétaire/proposant bloque les actes, masque ce qui doit l'être ; projets/fils/offres/progrès/caches/liens directs respectent le parent après retrait.
AC84	P0	F18	Notifications et projection. After-commit, une notification interne minimale par événement ; échec transaction aucune ; résumé public exactement consenti, source_offer_id unique et acteur système distinct.
AC85	P0	F18	Refus et retrait sans sanction. Decline propriétaire/withdraw proposant pending seulement ; motif privé, pas score ni honte publique ; offre expirée ne devient pas refus du propriétaire.
AC86	P0	F18	Progrès sans faux résultat. Accepted affiche collaboration commencée, pas amélioration réalisée ; seul résultat/commentaire/capsule effectivement enregistré apparaît avec nature réelle et visibilité ; pas faux compteur.

ID	Priorité	Exigence	Situation et résultat attendu
AC87	P0	F18	Autonomie de la communauté. Membre hors annuaire peut aider ; question sans code, contribution à demande ouverte et fermeture des offres coexistent ; aucun laboratoire ou aide préalable imposé.
AC88	P0	F18	Résilience et cache frontend. 401/403/404/409/419/422/429/ réseau/annulation : état juste, pas acceptation optimiste ; saisie en mémoire ; rejeu même intention ; logout/change compte purge offres privées.
AC89	P0	F18	Consentements accessibles. À 360/390px, clavier et zoom200% : deux consentements non précochés lisibles ; erreur proche du champ, focus/restauration dialog corrects, pas piège clavier ni débordement global.
AC90	P0	F18	Première collaboration complète. A ouvre projet sans demande ; B découvre/propose ; A accepte après aperçu et crée un fil ; B apporte une observation ; résultat éventuel attribué sans lab obligatoire. Vérifier aussi rattachement/refus/retrait et aucune permission supplémentaire.